

Tesis

PROTOTIPO PARA EL ANÁLISIS DE SEÑALES MIOELÉCTRICAS EN
PRÓTESIS ELECTROMECAÑICAS DE MIEMBROS SUPERIORES

Sergio Andrés Arenas Hoyos

Documento presentado a
Doctorado en Ingeniería con Énfasis en Ingeniería Eléctrica y Electrónica
como requisito para optar por el título de
Doctor en Ingeniería

Director: Álvaro Bernal Noreña

Escuela de Ingeniería Eléctrica y Electrónica
Facultad de Ingeniería



Agosto 2022

Resumen

En el contexto del análisis de señales EMG en prótesis electromecánicas de miembros superiores controladas a partir del reconocimiento de patrones, cabe mencionar que las etapas de preprocesamiento, extracción de características y clasificación de patrones pueden ser mejoradas en su desempeño a través de estrategias computacionales. Esto tiene el fin de mejorar el tiempo de procesamiento y consumo energético, para así ofrecer una prótesis más eficiente al usuario. Debido a las limitaciones de procesamiento en el hardware que pueden tener los sistemas de bajo consumo, existen estrategias que permiten un mejor uso de los recursos para realizar las operaciones matemáticas y algoritmos propios del procesamiento de señales. De manera más específica, existen antecedentes para la mejora del desempeño de sistemas digitales utilizando, por ejemplo, aritmética modular, el Sistema Numérico de los Residuos (RNS), Transformadas Numéricas Teóricas (NTT), u otros sistemas numéricos no convencionales; por ello se propone explorar el uso de estas técnicas en el preprocesamiento, la extracción de características en señales mioeléctricas, y la clasificación de movimientos con señales EMG. En detalle, la tesis se encamina hacia el uso de sistemas numéricos no convencionales en las etapas de preprocesamiento, extracción de características y clasificación de patrones, para determinar si hay incremento en el desempeño en plataformas de bajo consumo para el análisis de señales EMG en prótesis electromecánicas de miembros superiores.

Palabras clave— Señales mioeléctricas (EMG), Sistema Numérico de los Residuos (RNS), Aritmética Modular, Transformadas Numéricas Teóricas (NTT), Sistemas Numéricos no Convencionales, Procesamiento Digital de Señales (DSP), Hardware Reprogramable.

Abstract

In the context of the analysis of EMG signals in electromechanical upper limb prostheses controlled from pattern recognition, it is worth mentioning that the preprocessing, feature extraction and pattern classification stages can be improved in their performance through computational strategies. This has the purpose of improving the processing time and energy consumption, in order to offer a more efficient prosthesis to the user. Due to the processing limitations that low-power systems may have, there are strategies that allow a better use of hardware resources to perform the mathematical operations and algorithms of signal processing. More specifically, there are antecedents for improving the performance of digital systems using, for example, modular arithmetic, the Residue Number System (RNS), Theoretical Number Transforms (NTT), or other non-conventional number systems; Therefore, it is proposed to explore the use of these techniques in preprocessing, feature extraction in myoelectric signals, and movement classification with EMG signals. In detail, the proposal would cover the use of non-conventional numerical systems in the stages of preprocessing, feature extraction and pattern classification, to determine if there is an increase in performance in low-power platforms for the analysis of EMG signals in electromechanical prostheses of superior members.

Keywords— Myoelectric signals (EMG), Residue Numerical System (RNS), Modular Arithmetic, Numerical Theoretical Transforms (NTT), Non-Conventional Numerical Systems, Digital Signal Processing (DSP), Programmable Hardware.

DEDICATORIA

Esta tesis la dedico a mi familia, porque gracias a ellos he llegado a este punto tan importante de mi existencia, siendo mi fiel apoyo ante cualquier desventura. Principalmente le dedico esta tesis a mis padres y a Aura C. Siempre fueron un apoyo infinitamente importante en todos los momentos buenos y aquellos de mayor complejidad, y me enseñaron a que hay que perseverar y mantener una sonrisa firme ante todo. Gracias por creer en mí siempre y por siempre entregar su amor incondicional.

Agradecimientos

Agradezco a mis padres por siempre estar apoyándome con amor en todo lo que fuera necesario, sin importar nunca lo difícil que fuera el momento.

A mis tíos Sandra, Joaquín, Carmen y Nubia, porque gracias al intachable ejemplo de vida de ellos y a su continuo apoyo pude seguir el sendero actual de mi vida.

A mis hermanos Jaime y Patricia, porque estuvieron siempre atentos y dispuestos a apoyarme y a cuidar de mí cuando fue necesario.

A mi sobrina Camila por creer en mí y motivarme siempre a seguir adelante.

A Pedro, Alejandro, Santiago, Ferney, Oscar y Anderson porque su amistad mantuvo a flote la motivación que requerí ante los momentos difíciles.

Agradezco a mis primos Enrique y Jorge, por ser los primeros en creer en mi potencial y apoyarme en el camino de la vida.

A mi tutor Alvaro Bernal y al profesor Rubén Nieto, por apoyarme con sus sabias palabras, consejos y respaldo durante todo el camino académico que he recorrido desde hace varios años.

Muchas gracias a todos aquellos que de alguna manera me ofrecieron sus palabras de aliento o motivación ante la adversidad.

Tabla de contenido

	Pag.
Resumen	I
Abstract	II
DEDICATORIA	III
Agradecimientos	IV
Glosario	V
TABLA DE CONTENIDO	VI
Lista de figuras	X
Lista de tablas	XIV
1. Introducción.....	1
1.1 Problema de investigación.....	3
1.2 Justificación	4
1.3 Objetivo General.....	5
1.4 Objetivos Específicos	6
1.5 Metodología	6
1.5.1 Actividades del objetivo especifico A.....	6
1.5.2 Actividades del objetivo especifico B.....	7
1.5.3 Actividades del objetivo especifico C.....	7
1.5.4 Actividades del objetivo especifico D.....	7
1.5.5 Actividades del objetivo especifico E.....	8
2. Revisión de antecedentes	11
2.1 Implementaciones SoC para el procesamiento de señales mioeléctricas	13
2.1.1 <i>BioSoC: Highly integrated System-on-Chip for health monitoring</i> [Siwiec et al., 2016]	13
2.1.2 <i>4 channel signal based FPGA architecture for myoelectric features</i> <i>extraction by H.O.S.</i> [Díaz et al., 2010]	14
2.1.3 <i>A Zynq-based dynamically reconfigurable high density myoelectric</i> <i>prosthesis controller</i> [Boschmann et al., 2017]	15

2.1.4	<i>A modular test platform for real-time measurement and analysis of EMG signals for improved prosthesis control</i> [Borbély et al., 2014]	16
2.1.5	<i>A 24 μW, Batteryless, Crystal-free, Multinode Synchronized SoC ‘Bionode’ for Wireless Prosthesis Control</i> [Bhamra et al., 2015]...	17
2.2	Antecedentes para la extracción de características y clasificadores.....	18
2.2.1	<i>Biomedical Signal Processing and Control Current state of digital signal processing in myoelectric interfaces and related applications</i> [Hakonen et al., 2015]	18
2.2.2	<i>EMG feature evaluation for improving myoelectric pattern recognition robustness</i> [Phinyomark et al., 2013]	20
2.3	Antecedentes del uso de RNS en el procesamiento de señales y en arquitecturas digitales	21
2.3.1	Implementation of RNS-based distributed arithmetic discrete wavelet transform architectures using field-programmable logic [Ramírez et al., 2003]	21
2.3.2	A Deep Convolutional Neural Network Based on Nested Residue Number System [Nakahara and Sasao, 2015]	22
2.3.3	A multifunctional Unit for designing efficient RNS-based datapaths [Molahosseini et al., 2017]	23
2.3.4	RNS Hardware Matrix Multiplier for High Precision Neural Network Acceleration: RNS TPU [Olsen, 2018]	23
2.3.5	Rnsnet: In-memory neural network acceleration using residue number system[Salamat et al., 2018]	23
2.4	Antecedentes del uso de NTT en el procesamiento de señales y en arquitecturas digitales	24
2.4.1	Pattern Analysis under Number Theoretic Transforms [Moraga, 2012]	24
2.4.2	Video Filtering with Fermat Number Theoretic Transforms Using Residue Number Systems[Toivonen and Heikkila, 2006]	24
2.4.3	Quadratic modified fermat transforms for fast convolution and adaptive filtering [Radhakrishnan and Jenkins, 2010b]	25
2.4.4	Using Fermat Number Transform to Accelerate Convolutional Neural Network [Xu et al., 2018]	25
3.	Revisión de literatura	29
3.1	Marco Teórico.....	29
3.1.1	Señales mioeléctricas	29
3.1.2	Adquisición y adecuación de señales mioeléctricas	30
3.1.3	Extracción de características.....	32
3.1.3.1	Características del Dominio Temporal.....	33
3.1.3.2	Características del Dominio Frecuencial.....	45

3.1.3.3	Características del Dominio Temporal-Frecuencial	50
3.1.3.4	Características del Dominio Espacial[Hakonen et al., 2015]:.....	54
3.1.4	Reconocimiento de patrones	55
3.1.4.1	Análisis de Discriminante Lineal (LDA):	56
3.1.4.2	Máquinas de Vectores de Soporte (SVM) [Benatti et al., 2017] [Boser et al., 1992]:	59
3.1.5	Sistema de reconocimiento de señales mioeléctricas	66
3.1.6	Aritmética Modular [Abdelfattah, 2011][Apostol, 2013]	68
3.1.6.1	Sistema Numérico de los Residuos (RNS)[Ramírez et al., 2000]:.....	70
3.1.6.2	Transformadas Numéricas Teóricas (NTT).....	73
4.	Desarrollo o solución	75
4.1	Parte 1: Requerimientos algorítmicos	75
4.1.1	Máquinas de Vectores de Soporte	76
4.1.2	Restricciones de implementación de la SVM sobre enteros	78
4.1.2.1	Propuesta de función para el mapeo del Kernel sobre enteros en módulo M	79
4.1.2.2	Propuesta de función para el mapeo de una SVM sobre enteros en módulo M	84
4.1.2.3	Propuesta de modelo de SVM utilizando RNS.....	88
4.2	Parte 2: Requerimientos de la arquitectura en hardware	93
4.2.1	Revisión de la arquitectura	94
4.2.1.1	Bloque del Kernel	96
4.2.1.2	Detección de signo	97
4.2.1.3	Controlador principal	99
4.2.1.4	Registros	103
4.2.1.5	Sumadores RNS	103
4.2.1.6	Multiplicador RNS	106
4.2.1.7	Reducción al módulo $2^\beta - 1$	107
4.2.1.8	Reducción al módulo 2^α	109
4.2.1.9	Reducción al módulo $2^\beta + 1$	109
4.2.1.10	Arquitectura interna del multiplicador RNS	111
4.3	Parte 3: Especificaciones para el análisis de señales EMG de miembros superiores.....	111
4.3.1	Señales utilizadas	112
4.3.2	Extracción de características.....	114
4.3.3	Selección de características	116
4.3.4	Reducción de características	117
4.3.5	Normalización de características.....	121
4.3.6	Conversión de características al dominio RNS.....	121

4.3.7	Selección del clasificador	122
5.	Pruebas y resultados	124
5.1	Prueba 1: Pruebas de concepto de la <i>SVM-RNS</i> en software	124
5.2	Prueba 2: Pruebas de concepto de la <i>SVM-RNS</i> en hardware	130
5.3	Prueba 3: Prueba de concepto de la SVM-RNS en hardware con señales EMG.....	135
5.4	Discusión	152
6.	Conclusiones	160
	Referencias	162

ÍNDICE DE FIGURAS

FIGURA		Pag.
2.1	Esquema ilustrativo de los focos de conocimiento en la búsqueda del estado del arte, Autor	12
3.1	Adquisición de señales mioeléctricas.....	31
3.2	Ejemplo de señal mioeléctrica con segmentación: adyacente a la izquierda y con sobrelape a la derecha [Hakonen et al., 2015]	32
3.3	Bancos de filtros con el algoritmo MALLAT [Duan et al., 2016]	53
3.4	Estructura de datos para el entrenamiento del LDA, basado en [Xu et al., 2016]	59
3.5	Propósito de una superficie de decisión en una SVM, supuesto de características x e y y dos clases(cuadros rojos y círculos azules) [Dong, 2006].	60
3.6	Superficie de decisión en una SVM, mostrando el margen de separación permitido entre clases M^* y los vectores de soporte, enmarcados por círculos negros [Dong, 2006].	62
3.7	Superficies de decisión en una SVM multiclase, mostrando el margen de separación permitido entre tres clases [Xu et al., 2017].	66
3.8	Esquema ilustrativo de las etapas de un sistema de control reconocimiento de señales mioeléctricas (basado en [Hakonen et al., 2015])	67
3.9	Esquema ilustrativo de un sistema utilizando RNS (basado en [Navi et al., 2011])	72
4.1	Función RBF del Kernel, obtenida a partir de los <i>scripts</i> descritos en Python.	80
4.2	Equivalente en enteros de la función RBF del Kernel, obtenida a partir de los <i>scripts</i> descritos en Python.	83

4.3	Equivalente en enteros de la función RBF del Kernel, obtenida a partir de los <i>scripts</i> descritos en Python.	84
4.4	Esquema ilustrativo del hiperplano de la SVM con su región de interés y su límite	85
4.5	Esquema de la búsqueda de g_{lim}	86
4.6	Diagrama de bloques de la arquitectura propuesta para realizar el clasificador SVM-RNS	95
4.7	Diagrama de bloques de la función Kernel	96
4.8	Diagrama de bloques de la función signo	97
4.9	Diagrama de flujo de la función signo	98
4.10	Gráfico de la función signo	99
4.11	Esquema del controlador del sistema	100
4.12	Máquina de estados del controlador del sistema	102
4.13	Registros para el almacenamiento de datos en RNS	103
4.14	Esquema de sumador/restador en módulo m	105
4.15	Esquema compacto de sumador/restador en módulo m	106
4.16	Esquema de sumador/restador para un conjunto de módulos	106
4.17	Esquema de multiplicador en módulo m	107
4.18	Esquema del multiplicador RNS considerando los bloques de reducción al módulo	111
4.19	Ilustración de los remanentes de brazo por sujeto en <i>DB3</i> (basado en [Atzori et al., 2016])	113
5.1	<i>SVM</i> en enteros módulo M con $\alpha = 2, \beta = 6, M = 16380$	127
5.2	<i>SVM</i> en enteros módulo M con $\alpha = 4, \beta = 5, M = 16368$	127
5.3	<i>SVM</i> en enteros módulo M con $\alpha = 5, \beta = 7, M = 524256$	128
5.4	Función signo en RNS $\alpha = 4, \beta = 5, M = 16368$	128

5.5	Ejemplo funcionamiento en RNS $\alpha = 4, \beta = 5, M = 16368$	129
5.6	Esquema de los bloques hardware realizados en <i>Vivado</i>	130
5.7	Esquema de la distribución de características según las categorías	131
5.8	Simulación del punto $\hat{A}_{RNS} = \{\{0, 0, 0\}; \{0, 0, 0\}\}$	133
5.9	Simulación del punto $\hat{B}_{RNS} = \{\{0, 15, 31\}; \{0, 15, 31\}\}$	134
5.10	Simulación del punto $\hat{C}_{RNS} = \{\{1, 15, 30\}; \{1, 15, 30\}\}$	134
5.11	Resultados de la prueba codiseño con las herramientas de <i>Xilinx SDK</i> ...	135
5.12	Segmentación de un movimiento completo.....	138
5.13	Segmentación de una repetición en un movimiento	139
5.14	Características extraídas para un solo canal (electrodo 4 - antebrazo)	139
5.15	Componentes PCA obtenidas de la selección con RFE para un par de movimientos).....	140
5.16	Clasificador de prueba para los movimientos tipo 1 (etiqueta 27) y tipo 2 (etiqueta 28) en la etapa de procesamiento de los datos de la base de datos NinaPro.....	141
5.17	Estadísticas obtenidas para el clasificador de prueba con los movimientos tipo 1 y tipo 2.....	141
5.18	Clasificador RNS para el movimiento tipo 0 vs 1 con un $\beta = 5$ y un $\alpha = 4$.	143
5.19	Clasificador RNS para el movimiento tipo 1 vs 2 con un $\beta = 5$ y un $\alpha = 4$.	143
5.20	Clasificador RNS para el movimiento tipo 2 vs 0 con un $\beta = 5$ y un $\alpha = 4$.	143
5.21	Verificación de los datos de inicialización de la memoria dedicada al bias.	145
5.22	Verificación de los datos de inicialización de la memoria dedicada a los parámetros de vectores de soporte.....	145
5.23	Verificación de los datos de inicialización de la memoria dedicada a la función Kernel	146
5.24	Comparativa de las matrices para la reducción PCA de las características obtenidas de NinaPro para un conjunto de datos en Python, C y su ejecución en el SoC	147

5.25	Comparativa de la normalización en C y su ejecución en el SoC	147
5.26	Clasificación de un punto en el sistema SVM-RNS	148
5.27	Clasificación de dos categorías en el sistema SVM-RNS	149
5.28	WNS reportado y su asociación al reloj de 50 MHz <i>clk_fpga_0</i>	150
5.29	relojes del sistema	150
5.30	Información general de la plataforma y del proyecto diseñado	151
5.31	Desempeño porcentual medido en el sistema por <i>Vivado</i>	151
5.32	Desempeño medido en el sistema por <i>Vivado</i>	152
5.33	Esquema del sistema implementado con las herramientas de <i>Vivado</i>	152

ÍNDICE DE CUADROS

TABLA	Pag.
1.1 Short Caption	9
2.1 Short Caption	26
3.1 Características del dominio temporal [Phinyomark et al., 2012].....	45
3.2 Características del dominio frecuencial [Phinyomark et al., 2012].	49
4.1 Resumen de las funciones de conversión de parámetros.....	85
4.2 Conversión de parámetros enteros a RNS.....	88
4.3 Salidas de la máquina de estados del controlador	103
5.1 Resumen de comparativas de la SVM en módulo M	126
5.2 Resumen de comparativas en implementaciones afines.....	158

1. Introducción

Desde el ámbito social, en Colombia existen dificultades de acceso al trabajo para las personas en condición de discapacidad; para hacerse a una idea, el 55 % de los empresarios en una encuesta realizada por la revista Dinero en el 2015 [Dinero, 2015], afirmaron no contratar personal con discapacidad en su compañía. Para el 2012 [Concha, 2012] se mencionó que “En Colombia, el 61 % de las personas con discapacidad no recibe ningún tipo de ingreso económico para su subsistencia, es necesario que como sociedad les brindemos más oportunidades laborales”. Además, se resalta que en Colombia hay cerca de 3 millones de personas con discapacidad, de las cuales el 52 % está en edad productiva, pero solo el 15,5 % realiza algún trabajo, usualmente mal remunerado. En Cali, existen reportes del periódico El País [Pais, 2011], donde se señala que en el año 2011 el 6,4 % de la población (145.256 de 2.269.630 habitantes) sufría algún tipo de discapacidad. Dentro de este conjunto, el 19 % presentaban limitaciones en la movilidad de brazos y piernas. Para ese entonces, cada día se anunció que un promedio 4 personas discapacitadas instauraban quejas “porque las EPS no les entregan medicamentos, terapias o ayudas técnicas, constituyendo el 15 % del total de quejas que recibe diariamente dicho organismo ”.

Entre las ayudas técnicas para la mencionada población, se encuentran las prótesis electromecánicas que permiten asistir a las personas con discapacidades.

Según el portal el Hospital [Hospital, 2015] se han desarrollado prótesis con impresión 3D hechas en Colombia, para los casos de personas a las que se le han amputado brazos. Los desarrollos mencionados fueron realizados en la Facultad de Ingeniería Mecatrónica de la Universidad Nacional de Colombia; inicialmente utilizaron prótesis hechas de acero inoxidable, pero utilizando impresión 3D argumentaron que se reducen tiempos de fabricación y costos de la pieza.

En el contexto de las prótesis electromecánicas debe incluirse un sistema electrónico, donde se soporte el análisis de las señales bioeléctricas involucradas. Entre las señales bioeléctricas utilizadas para el control de prótesis, se pueden mencionar las señales electroencefalográficas (EEG), provenientes de estímulos cerebrales; o señales mioeléctricas (EMG), que son señales eléctricas producidas en los músculos al producirse una orden de movimiento, y pueden adquirirse en el tejido muscular, o de manera no invasiva (superficial) a través de electrodos. Este último tipo de señales son

denominadas señales mioeléctricas superficiales (sEMG) y son convenientes para este estudio, porque resultan menos invasivas con respecto a las EEG.

Abordando el esquema de control electrónico de prótesis, existen varias etapas que lo conforman: primero se adquieren las señales a través de un transductor y un conversor analógico-digital; después se realiza un preprocesamiento de señales; luego deben extraerse y analizarse las características principales de la señal; finalmente, debe clasificarse el tipo de movimiento para realizar una acción en la prótesis.

De acuerdo a lo mencionado, el análisis de señales está inmerso entre las etapas de preprocesamiento, extracción de características y de clasificación de movimientos, y debe considerarse que este procesamiento en el contexto de una prótesis electromecánica está ligado al uso de sistemas de procesamiento digital de señales (DSP), situados físicamente en la prótesis. El requerimiento de un DSP en un entorno reducido, puede verse contenido en un sistema de bajo consumo de potencia eléctrica para facilidades de uso.

Debido a las limitaciones de procesamiento que pueden tener los sistemas de bajo consumo respecto a sistemas de altas prestaciones técnicas, existen estrategias que permiten un mejor uso de los recursos hardware para realizar las operaciones matemáticas y algoritmos propios del procesamiento de señales. Tal es el caso, por ejemplo, del uso de la aritmética modular [Madan et al., 2016] [Sabbagh et al., 2017], el Sistema Numérico de los Residuos (RNS) [Albicocco et al., 2014] [Chokshi et al., 2009] [Cardarilli et al., 2007]; Transformadas Numéricas Teóricas (NTT) [Moraga, 2012] [Madan et al., 2016]; o sistemas numéricos no convencionales [Yang et al., 2014]. Por ello se propone abordar el uso de estas técnicas en el entorno mencionado, explorando la posibilidad de mejorar el desempeño de plataformas hardware de bajo consumo en el análisis de las señales mioeléctricas. La posibilidad de mejora radica en que existen sistemas de diversa complejidad computacional que se ven beneficiados por el uso de éstos métodos, como sistemas de cálculo tensorial asociados al aprendizaje en máquinas (Machine Learning) [Salamat et al., 2018] [Olsen, 2017], acelerar las capas de convolución en Redes Neuronales Convolucionales [Xu et al., 2018] o procesamiento de imágenes utilizando NTT [Toivonen and Heikkila, 2006].

Por estos motivos se propone una herramienta tecnológica que contribuya a la mejora de las capacidades de movimiento del sector poblacional mencionado. Las mejoras del sistema se realizarían a nivel funcional en las etapas de extracción de características, su posterior análisis, y la clasificación de movimientos; esto será un aspecto central del desarrollo, pues una mejora a nivel computacional del sistema electrónico permitiría

un mejor desempeño en el esquema de control en prótesis. Acorde a lo anterior, una plataforma que contenga lógica reprogramable de bajo consumo es un medio físico para implementar los requerimientos de DSP y los bloques específicos para las funciones pertinentes de aritmética modular u otros sistemas numéricos no convencionales.

1.1 Problema de investigación

En las prótesis electromecánicas de miembros superiores puede incluirse un control electrónico para reconocer intenciones de movimiento, partiendo de la adquisición de señales mioeléctricas. Para lograr procesar la información de estas señales, es posible utilizar técnicas de reconocimiento de patrones en la etapa de control, las cuales ofrecen flexibilidad en su clasificación, debido a que en el entrenamiento de los clasificadores se involucran estas señales. Por otro lado, en las prótesis de brazo se utilizan sistemas embebidos de bajo consumo energético, lo cual implica la búsqueda de arquitecturas que mejoren las figuras de mérito en el procesamiento de este tipo de señales, favoreciendo la portabilidad y tiempo de autonomía del dispositivo.

Cabe resaltar que en un sistema de reconocimiento de patrones en señales EMG, existen varias etapas que lo conforman: primero se adquieren las señales a través de un transductor y un conversor analógico-digital; después se realiza un preprocesamiento de señales; luego deben extraerse y analizarse las características principales de la señal; finalmente, debe clasificarse el tipo de movimiento para realizar una acción en la prótesis.

En las etapas de preprocesamiento y la extracción de características se requieren cálculos para los dominios temporal, frecuencial y temporal frecuencial, lo cual implica complejas operaciones aritméticas que debe manejar el sistema principal. Por otro lado, la etapa de clasificación involucra en las etapas de entrenamiento y ejecución de un clasificador, donde los algoritmos utilizados pueden requerir cálculos sobre vectores o matrices de alta dimensionalidad. Debe tenerse en cuenta que en el preprocesamiento y extracción de características es necesario un análisis que permita determinar un conjunto adecuado de características para una adecuada clasificación.

Estas condiciones suponen, en conclusión, la necesidad de explorar estrategias que permitan realizar mejoras a nivel algorítmico o a nivel físico de los bloques funcionales que están implicados en los sistemas de procesamiento de señales mioeléctricas. Como

se observa en el Estado del Arte, no hay una implementación en plataformas basadas en hardware reprogramable de bajo consumo para la evaluación del desempeño de los bloques de preprocesamiento, extracción de características y clasificación de patrones, específicamente para el análisis de señales EMG en prótesis electromecánicas de miembros superiores, utilizando Sistemas Numéricos no Convencionales. Esto permite plantear una propuesta de investigación que cubra estos aspectos, evaluando la intersección de los tres focos descritos. En detalle, la propuesta abarcaría el uso de sistemas numéricos no convencionales en las etapas mencionadas, para determinar si hay mejora en el desempeño de las plataformas de bajo consumo para el análisis de señales EMG en prótesis electromecánicas de miembros superiores. Todo esto lleva a la siguiente pregunta de investigación:

- ¿Mediante el uso de sistemas numéricos no convencionales en las etapas de preprocesamiento, extracción de características y clasificación de patrones, es posible incrementar el desempeño de plataformas basadas en hardware reprogramable de bajo consumo para el análisis de señales EMG en prótesis electromecánicas de miembros superiores?.

1.2 Justificación

En el contexto del análisis de señales EMG en prótesis electromecánicas de miembros superiores controladas a partir del reconocimiento de patrones, cabe mencionar que las etapas preprocesamiento, extracción de características y clasificación de patrones pueden ser mejoradas en su desempeño a través de estrategias computacionales[Côtéallard et al., 2016][Salim et al., 2011]. Esto tiene el fin de mejorar el tiempo de procesamiento y consumo energético, para así ofrecer una prótesis más eficiente al usuario.

Físicamente, los bloques de procesamiento de señales EMG tienen la tendencia a implementarse en plataformas, reprogramables o de propósito específico (ASIC) de bajo consumo. Por otro lado, para el diseño de prototipos resulta más conveniente utilizar tecnologías reprogramables como los SoC (System on Chip) o los FPGA, donde pueden realizarse tareas de manera concurrente apuntando a una mejora en los

criterios de velocidad y la fragmentación de un problema amplio en rutinas más pequeñas, además de facilitar cambios en la arquitectura del diseño.

Debido a las limitaciones de procesamiento que pueden tener los sistemas de bajo consumo, existen estrategias que permiten un mejor uso de los recursos hardware para realizar las operaciones matemáticas y algoritmos propios del procesamiento de señales. De manera más específica, existen antecedentes para la mejora del desempeño de sistemas digitales utilizando, por ejemplo, aritmética modular [Madan et al., 2016] [Sabbagh et al., 2017]; el Sistema Numérico de los Residuos (RNS) [Albicocco et al., 2014] [Chokshi et al., 2009] [Cardarilli et al., 2007] [Ramírez et al., 2003]; Transformadas Numéricas Teóricas (NTT) [Moraga, 2012] [Madan et al., 2016]; u otros sistemas numéricos no convencionales [Yang et al., 2014]; por ello se propone explorar el uso de estas técnicas en el preprocesamiento, la extracción de características en señales mioeléctricas, y clasificación de movimientos con señales EMG. La posibilidad de mejora radica en que existen sistemas de diversa complejidad computacional: sistemas de cálculo tensorial asociados al aprendizaje en máquinas (Machine Learning) [Salamat et al., 2018] [Olsen, 2017], acelerar las capas de convolución en Redes Neuronales Convolucionales [Xu et al., 2018] o procesamiento de imágenes utilizando NTT [Toivonen and Heikkila, 2006], que se ven optimizados por el uso de los sistemas numéricos no convencionales. Estas técnicas para mejorar el desempeño y que pueden redundar también de la reducción de área, el aumento de la velocidad y la disminución del consumo de potencia eléctrica serán un aspecto central del desarrollo, ya que una mejora a nivel computacional del sistema electrónico podría impactar la calidad del sistema electrónico, lo cual beneficia al usuario.

1.3 Objetivo General

- Desarrollar un sistema electrónico en hardware reprogramable de bajo consumo para el análisis del desempeño en el procesamiento de señales EMG para prótesis electromecánicas de miembros superiores, utilizando sistemas numéricos no convencionales.

1.4 Objetivos Específicos

- A. Determinar conjuntos de características en señales EMG para su uso en el reconocimiento de patrones.
- B. Definir los bloques hardware que permitan realizar las etapas de preprocesamiento, extracción de característica y clasificación de patrones.
- C. Seleccionar un algoritmo de clasificación que permita mejorar su desempeño implementado en hardware reprogramable.
- D. Usar sistemas de numeración no convencional (NTT, RNS) para implementar los bloques funcionales en hardware reprogramable.
- E. Contrastar el desempeño de las unidades hardware concebidas respecto a implementaciones similares previas.

1.5 Metodología

Para el desarrollo del proyecto de investigación, se proponen una serie de actividades que contribuirán al desarrollo de cada uno de los objetivos específicos planteados. Cabe resaltar que las actividades implicadas en el objetivo D y E resultan como las de mayor impacto a nivel del proyecto, debido a que marcan las diferencias respecto a estudios previos, mencionados en el Estado del Arte. Además, para la satisfacción de estos objetivos se debe tomar un tiempo más amplio, ya que implica el desarrollo directo de sistema digitales en hardware.

1.5.1 Actividades del objetivo específico A

- A1. Estudio y selección de las principales características en las señales EMG .
- A2. Estudio y selección de técnicas de preprocesamiento en las señales EMG .
- A3. Determinar los principales bancos de datos de señales mioeléctricas utilizados para reconocimiento de patrones.

A4. Seleccionar el mejor banco de datos para el procesamiento de señales.

A5. Documentación de avances realizados.

1.5.2 Actividades del objetivo específico B

B1. Estudiar los principales bloques utilizados para la implementación de cada una de las etapas del procesamiento de señales mioeléctricas involucradas.

B2. Seleccionar la plataforma hardware adecuada para la implementación del sistema electrónico.

B3. Seleccionar los bloques que serán utilizados, según los criterios de limitación impuestos por la plataforma hardware.

B4. Documentación de los avances realizados.

1.5.3 Actividades del objetivo específico C

C1. Estudio de los principales métodos de clasificación de señales EMG.

C2. Estudio de las principales formas de entrenamiento de los clasificadores.

C3. Determinar el clasificador y el método de entrenamiento con mayor viabilidad para implementar en hardware.

C4. Documentación de avances realizados.

1.5.4 Actividades del objetivo específico D

D1. Estudiar las principales figuras de mérito involucradas en la implementación hardware del procesamiento de señales mioeléctricas

D2. Estudiar y determinar la forma de implementar los bloques funcionales del sistema, de manera que puedan utilizar el sistema numérico de los residuos u otros sistemas numéricos no convencionales para su representación.

- D3. Estudiar y determinar qué mejoras algorítmicas para el procesamiento de señales mioeléctricas son viables para implementar en el hardware seleccionado.
- D4. Seleccionar la forma y el lenguaje de programación adecuado de describir el hardware requerido dentro de la plataforma reprogramable.
- D5. Describir en hardware los bloques funcionales para el procesamiento de señales mioeléctricas.
- D6. Documentación de los avances realizados.

1.5.5 Actividades del objetivo específico E

- E1. Seleccionar implementaciones previas de las unidades hardware concebidas.
- E2. Comparar los resultados de la validación funcional en la implementación hardware concebida, respecto a implementaciones previas.
- E3. Determinar las principales diferencias a nivel de desempeño entre implementaciones.
- E4. Determinar las ventajas y desventajas de la implementación hardware concebida.
- E5. Documentación de los avances realizados.

Para realizar esta metodología se planteó un cronograma establecido para dos años (ver tabla 1.1)

Actividad/ Meses	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
A1	X	X																						
A2	X	X																						
A3	X	X																						
A4			X																					
A5	X	X	X	X																				
B1			X	X																				
B2			X																					
B3			X	X																				
B4			X	X																				
C1			X	X																				
C2			X	X																				
C3				X																				
C4				X	X																			
D1				X	X	X	X																	
D2						X	X	X	X	X	X													
D3								X	X	X	X													
D4			X																					
D5										X	X	X	X	X	X	X	X	X	X					
D6			X								X								X					
E1																	X	X	X					
E2																				X				
E3																					X	X		
E4																						X	X	
E5																							X	X

Cuadro 1.1: Cronograma

El desarrollo de esta tesis consta de 6 capítulos. De manera general, los capítulos muestran consisten de lo siguiente:

- El capítulo 1 muestra el problema de investigación y el alcance de la misma, a la vez que se hace una breve introducción de los temas más relevantes.
- El capítulo 2 muestra los trabajos más relevantes que fueron considerados en la revisión del estado del arte.
- El capítulo 3 provee un marco general entrando en detalle en los temas relacionados la caracterización de las señales mioeléctricas, y los criterios necesarios para la extracción de características y el reconocimiento de patrones en el ámbito de las señales mioeléctricas. También se realiza una descripción sobre los conceptos de aritmética modular involucrados en el trabajo.
- El capítulo 4 muestra en detalle la solución desarrollada, con los planteamientos teóricos correspondientes.
- El capítulo 5 muestra las diferentes pruebas realizadas en software y hardware para el sistema desarrollado. Adicionalmente tiene una sección de discusión de resultados
- El capítulo 6 contiene las conclusiones de la tesis desarrollada.

2. Revisión de antecedentes

La búsqueda del estado del arte se cimentó en la intersección de los siguientes pilares: Sistemas para el procesamiento digital de señales (DSP), sistemas numéricos no convencionales, sistemas digitales de bajo consumo y análisis de señales EMG(ver figura 2.1).

Como se verá más adelante en el desarrollo de esta sección, existen relaciones entre focos, fundamentadas en investigaciones previas:

- Entre el análisis de señales EMG y el DSP existen sistemas de control de prótesis basados en el reconocimiento de patrones.
- Entre los sistemas de procesamiento digital de señales y los sistemas numéricos no convencionales, se pueden encontrar etapas de filtrado, extracción y clasificación de características con sistemas numéricos no convencionales.
- Entre los sistemas digitales de bajo consumo y los sistemas numéricos no convencionales, se han encontrado procesadores basados en el RNS, y bloques aritméticos basados en conceptos afines. En la mayoría de los casos, resulta en una disminución de consumo de área o potencia eléctrica, o en un aumento de la velocidad de procesamiento de operaciones.

Se encontró que no hay una intersección completa de todos los focos especificados, formulando una propuesta de investigación al respecto, cuyo desarrollo implica correr la frontera del conocimiento.

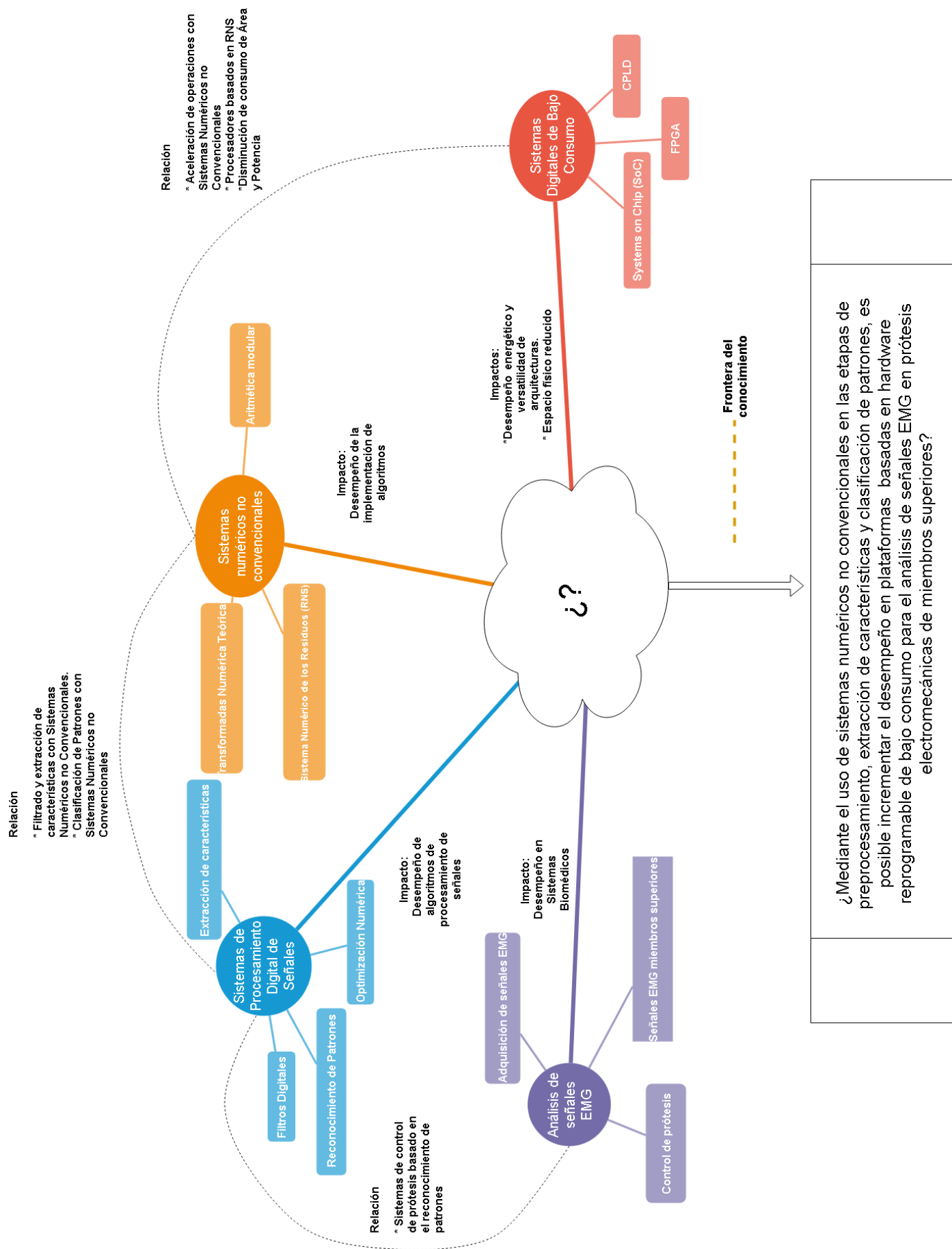


Figura 2.1: Esquema ilustrativo de los focos de conocimiento en la búsqueda del estado del arte, Autor

2.1 Implementaciones SoC para el procesamiento de señales mioeléctricas

2.1.1 *BioSoC: Highly integrated System-on-Chip for health monitoring* [Siwiec et al., 2016]

En su arquitectura se concibe la medición de electrocardiogramas (ECG), electromiogramas(EMG), temperatura y resistencia de la piel, y la Tasa de Respiración (RR). Tiene como núcleo un microprocesador Adelite de 32 bits con varias interfaces seriales, GPIOs y un reloj de 32kHz a 16MHz. Se menciona que el dispositivo tiene altas funcionalidades para el área de telemedicina, respondiendo a las demandas de dispositivos portátiles para el monitoreo de la salud.

Como el sistema implementado debe lograr un manejo de señales eléctricas del cuerpo humano para su procesamiento, se puede observar el BioSoC desde dos secciones: la sección analógica y la sección digital.

Dentro de su sección analógica cuenta con : entradas para señales analógicas con adecuaciones para cada tipo de señal, ADC, referencias de corriente y de voltaje, generadores de señales de reloj y un reset al encendido (POR).

En su sección digital se destacan: un procesador Adelite de bajo consumo de 32 bits con arquitectura RISC, un controlador de interrupciones, unidad de control de potencia(PMU) y 4kB de Scratch-pad RAM (SPRAM). Esta última tiene características similares a una memoria caché.

Este sistema analógico - digital fue fabricado con una tecnología CMOS de 130nm, en un empaquetado de 80 pines QFN, para concretarse finalmente en un área de $22mm^2$; el uso de la tecnología CMOS de 130nm fue satisfactoria para la adquisición de las señales implicadas.

Para confirmar el funcionamiento del circuito fabricado, se tomó el SoC y se utilizaron programas de prueba para verificar el funcionamiento de las interfaces analógicas y el microprocesador Adelite; adicionalmente se realizaron mediciones para verificar condiciones de anchos de banda, error en las medidas y ganancias de los dispositivos.

Además de las verificaciones funcionales, se midió el consumo de corriente tomando señales en todas las interfaces analógicas y con el microprocesador trabajando a 8MHz, dando como resultado una corriente de 1.7mA para la correcta operatividad del dispositivo.

Desde el punto de vista de los bloques para realizar algoritmos de procesamiento de

señales, se incluyó el procesador Adelite, el cual está equipado con varias interfaces de comunicación.

En conjunto, el objeto de realizar este dispositivo fue su uso en el diseño de equipos portables complejos para el monitoreo de la salud.

2.1.2 *4 channel signal based FPGA architecture for myoelectric features extraction by H.O.S. [Díaz et al., 2010]*

En este trabajo se utilizó una arquitectura paralela dentro de una FPGA para realizar la extracción de características de las señales mioeléctricas del codo a través de estadísticas de orden superior (HOS), para lograr la posterior clasificación de este tipo de señales. Para la implementación del sistema mencionado se manejó VHDL en una Xilinx Spartan-3 a una frecuencia máxima de funcionamiento de 44.5 MHz. El tipo de señales involucradas fueron las mioeléctricas superficiales (sEMG).

Para el control de prótesis por reconocimiento de patrones es necesario realizar una correcta clasificación de las señales mioeléctricas. Generalmente, la distribución de probabilidad de las señales puede ser de tipo Gaussiana o Laplaciana, y en el caso de las señales no Gaussianas de este trabajo, se propuso utilizar la extracción de estadísticas de orden superior. Se comentó que en implementaciones previas, los algoritmos se realizaban inicialmente en lenguaje C, para luego abordarse desde la perspectiva del hardware reprogramable para mejorar las tasas de procesamiento. Para lograr abordar el problema de la clasificación de señales mioeléctricas, se hizo una separación en etapas: adquisición, amplificación y filtrado de las señales, extracción de características, reducción de la dimensionalidad y clasificación de patrones.

En la etapa de extracción de características se recomendó no utilizar la Transformada de Fourier para caracterizar este tipo de señales, debido a que no son señales estacionarias. Por ello, se ha propuesto el uso de características en el tiempo, coeficientes autoregresivos, coeficientes cepstrales y coeficientes de la transformada Wavelet para clasificar estas señales no-estacionarias. Por otro lado, en el trabajo realizado se hizo una exploración a estrategias para la extracción de características basadas en estadísticas de orden superior, las cuales son relativamente fáciles de implementar, y requieren operaciones aritméticas sencillas de implementar en FPGA o

DSP.

Para caracterizar un proceso Gaussiano es suficiente utilizar la autocorrelación y la densidad espectral de potencia (primer y segundo momento estadístico). Para utilizar este concepto a sistemas no-Gaussianos, debe extenderse el manejo de momentos de orden superior. En este caso, se utilizaron momentos de diferentes órdenes, y se observó que los porcentajes de clasificación correcta aumentaron conforme se utilizaron los momentos de tercer orden para cuatro tipos de movimientos

Básicamente, la extracción de este tipo de características se basa en la multiplicación matricial iterativa de manera sistólica e iterativa.

Al finalizar el trabajo, se señaló que la complejidad de cálculo incrementa conforme aumenta la longitud de las características, y que la implementación de este sistema basado en HOS mostró una mejora en la clasificación respecto al uso de Wavelets. En concreto, la latencia en el cálculo resultó de unos 23ns.

2.1.3 *A Zynq-based dynamically reconfigurable high density myoelectric prosthesis controller* [Boschmann et al., 2017]

Se presentó un sistema basado en la Xilinx Zynq para realizar la clasificación de señales EMG de una persona amputada. El enfoque del artículo fue mostrar la manera de acelerar las partes más costosas en cómputo: la extracción de características y la clasificación de patrones. Para la adquisición de señales previa a la extracción de características, se usaron 192 canales de alta calidad con retardos de control debajo de los 120 ms, útiles para respuestas en tiempo real en controladores de prótesis. Se utilizó la reconfiguración dinámica de la FPGA para permitir un balance de tiempo de clasificación contra precisión en la clasificación si la calidad de la señal decrecía debido a ruido en los canales. La prótesis se encuentra en el estado del arte en alta densidad de sensores EMG (HD EMG - 192 sensores EMG).

El objetivo principal del trabajo fue lograr un controlador robusto que permitiera la correcta clasificación de señales HD – EMG. Por otro lado, se apuntó a que el controlador debía ser en un sistema embebido para poder permitir la autonomía de la prótesis. Debido a que los requerimientos de tiempo real son críticos en los retardos de control, no se permitió el procesamiento de las señales en un dispositivo externo como un Smartphone o a través de comunicaciones inalámbricas. Este requerimiento hizo de

las FPGA un candidato adecuado para la realización de esta aplicación.

En adición a la extracción de características acelerada en hardware, se aceleró la función de clasificación en la FPGA, mejorando el desempeño de la prótesis.

Por otro lado, de forma heurística se midió el ruido en los canales EMG; si disminuye la calidad se puede reconfigurar la FPGA para lograr una mayor robustez al ruido, a cambio de disminuir la velocidad de procesamiento.

Para los parámetros de tiempo real, se utilizan ventanas deslizantes en las señales de entrada. Las ventanas de procesamiento se superponen medianamente una sobre otra, encolando las decisiones de movimiento de la prótesis que finalmente van al actuador del sistema de control.

Inicialmente, las señales se promedian utilizando RMS y se mapean en una imagen. Posterior a ello, se extraen características del tipo Densas e Invariantes ante el cambio de Escala (DSFIT). Luego se agrupan en diferentes clases utilizando el método de los k-means para unos cuantos conjuntos DSFIT aleatorios. Con ello se entrena una Máquina de Vectores de Soporte (SVM), que es utilizada como clasificador.

Para el enfoque de tiempo real, se utilizó una arquitectura ReconOS, donde trabajan en conjunto un sistema Linux que lanza hilos software en el ARM de la plataforma Zynq, y lanza los hilos hardware en la sección FPGA. Los hilos hardware soportados son la extracción de características DSFIT y la predicción utilizando el SVM entrenado.

Dentro de la validación funcional, se utilizó un arreglo de 192 sensores EMG HD, en conjunto con una Zedboard y una prótesis Otto Bock Michelangelo para la clasificación de 6 tipos de movimiento. Se probaron diferentes combinaciones para el uso de hilos hardware y software.

Al terminar la implementación del sistema, se encontró que el uso de la FPGA para tareas como la multiplicación de matrices en el SVM, la predicción de patrones con el SVM y la extracción de características DSFIT, mejora el rendimiento del sistema en términos de velocidad y potencia.

2.1.4 *A modular test platform for real-time measurement and analysis of EMG signals for improved prosthesis control [Borbély et al., 2014]*

Muestra una plataforma para el procesamiento de señales sEMG. Su estructura es diseñada de manera modular para facilitar la integración de tareas de medición,

procesamiento de señales y control de actuadores. Para medir el desempeño del sistema, se probaron dos métodos diferentes en la clasificación de patrones dentro de una Xilinx Zynq-7000. Este hardware reconfigurable para el prototipado cuenta con un SoC que permite una flexibilidad a la hora de implementar algoritmos de reconocimiento de patrones. La modularidad también facilitó la reconfiguración del hardware, teniendo en cuenta que cada etapa tuviera el menor consumo posible de potencia.

Dentro del trabajo se mencionó que en el control de prótesis electromecánicas superiores son muy utilizadas las señales sEMG como vectores de entrada, procedentes de la suma superficial de señales eléctricas en la piel al contraer los músculos.

Para el desarrollo de la unidad de medidas, se estableció el uso de un microcontrolador AVR, cuyas interfaces permitieron adquirir 8 señales EMG bipolares a 1kHz de frecuencia de muestreo y 16 bits de resolución. Las señales adquiridas se tomaron de manera que pudieran guardarse en una tarjeta SD y pudieran transmitirse a través de una interfaz Bluetooth.

En el bloque de análisis de señales, se utilizó una ZedBoard con un SoC Xilinx Zynq 7020, y un módulo Bluetooth para obtener las señales de la unidad de medidas, donde el bloque de Sistema de procesamiento (PS) se utilizó para interactuar con el módulo Bluetooth y el módulo PL se utilizó para el preprocesamiento de señales.

La implementación del primer algoritmo se basa en aprendizaje supervisado permite el entrenamiento fuera de línea del clasificador y posteriormente clasifica los datos en línea con lo obtenido del entrenamiento. El segundo tipo de clasificador se basa en el primero, pero con una variación en el método de entrenamiento, lo cual provee una cierta ventaja respecto al control de posición.

Este sistema puede ser utilizado con gran variedad de unidades de medida y de actuadores, debido a su estructura modular.

2.1.5 *A 24 μ W, Batteryless, Crystal-free, Multinode Synchronized SoC ‘Bionode’ for Wireless Prosthesis Control [Bhamra et al., 2015]*

Se propone un sistema sin requerimiento de batería, sin cristales para reloj, con uso de TDMA que está conformado por una red inalámbrica de sensores corporales (WBSN) en System on Chip (SoC). Cada uno de los nodos se basa en el uso de la

recolección de energía en supercondensadores a través de señales de radiofrecuencia, con una interfaz de adquisición de señales, un trancceptor RF de muy bajo consumo y circuitería de relojes. Los nodos sensores se conectan a una estación base, montada en la prótesis de brazo. Cada nodo se implementó con tecnología CMOS de $0.18\ \mu\text{m}$, con un consumo menor a $24\ \mu\text{W}$.

Los biosensores pueden ubicarse encima de la piel, o por debajo directamente en los músculos y obtener una mejora en la señal adquirida.

Básicamente todas las etapas fueron concebidas: Los ADCs, la fuente Boost para brindar eficiencia en la alimentación, reguladores y referencias de voltaje y corriente, los protocolos de comunicación utilizando OOK y TDMA, las interfaces analógicas y la circuitería de compensación para el reloj. Las antenas fueron meticulosamente escogidas para cumplir con un patrón de radiación que no superase los 26,47 mm de radio, y que trabajaran en la banda de 915 MHz. Los supercondensadores son externos y de montaje superficial de 3 mm x10 mm, con una capacitancia de 120 mF. Las ganancias de las señales de entrada son ajustables.

En las pruebas experimentales se observó que la probabilidad de error de bit (BER) en los enlaces de comunicación con una estación emulada, era menor a 0.01 %, con un error en la medida menor a $35\ \mu\text{V}$.

Los nodos cumplen de forma satisfactoria los requerimientos para crear una WBSN autónoma, además de ser el nodo WBSN con menor consumo de potencia reportado.

2.2 Antecedentes para la extracción de características y clasificadores

2.2.1 *Biomedical Signal Processing and Control Current state of digital signal processing in myoelectric interfaces and related applications* [Hakonen et al., 2015]

Esta revisión discute los aspectos críticos y las prácticas recomendadas en los sistemas con interfaces de señales mioeléctricas. También trata de mostrar las nuevas tendencias en este campo.

Se muestran las ubicaciones donde deben situarse los electrodos, las condiciones que deben cumplir, y además se hace un análisis detallado de cada uno de los pasos a seguir en este tipo de sistemas.

Haciendo énfasis en la extracción de características, se presentan cuatro categorías: las características del dominio del tiempo (TD), las características del dominio frecuencial (FD), las características de tiempo-frecuencia (TFD), y las características espaciales (SD) (ubicación de los electrodos, diferenciación de posturas, fuerza de una contracción).

Para la adquisición de señales EMG, se señala el uso de electrodos dispuestos en los músculos a analizar. La forma en que se configuren los electrodos es importante para lograr una buena clasificación de señales a largo plazo. Para facilitar esta tarea, existen brazaletes que permiten adquirir señales mioeléctricas de varios canales, entre ellos destaca la Myo Armband de Thalmic Labs.

Las señales EMG adquiridas presentan un comportamiento relacionado a sus componentens frecuenciales, y se menciona que la mayor información de las señales EMG se encuentra por debajo de los 400 Hz, por lo tanto, una buena frecuencia de muestreo ronda 1kHz.

A partir de las señales EMG se extraen características TFD, entre ellas se menciona que la transformada Wavelet puede mejorar al robustez del sistema respecto a las características temporales y frecuenciales, las cuales pueden ser tomadas como subespacios de las transformadas Wavelet. Debe acompañarse a la transformada Wavelet con un proceso de reducción de dimensionalidad para incrementar la velocidad y la precisión de la clasificación. Entre estos métodos se menciona el Análisis de Componentes Principales (PCA) como uno de los más comunes. Por otro lado, se pueden aplicar técnicas TD a los coeficientes Wavelet para hacer la reducción de dimensionalidad.

Un conjunto recomendable de características para la clasificación de movimientos es: Cruce por Cero (ZC), Amplitud de Willison (WAMP) y el Valor Absoluto Medio (MAV) calculado del segundo nivel de la señal de reconstrucción de una Wavelet Daubechies Db7 y la rata porcentual de miopulsos (MYOP) del primer nivel de la señal EMG reconstruida con una Wavelet Daubechies Db8. Una característica importante de la Transformada Wavelet Discreta (DWT), es que resulta muy útil para remover el ruido blanco presente en la señal EMG.

Dentro de los clasificadores se menciona que los Discriminadores Lineales (LDA), las Máquinas de Vectores de Soporte (SVM) y los Modelos Ocultos de Markov (HMM) son los más típicos para realizar estas clasificaciones, debido a que son relativamente sencillos de entrenar e implementar. Destacan al LDA como uno de los más comunes, logrando porcentajes de éxito de clasificación de 82.37 % cuando se entrena con datos

de 21 días. En caso de requerir un mayor porcentaje de clasificaciones correctas a largo plazo es recomendable utilizar clasificadores de mayor complejidad.

Los algoritmos de preprocesamiento como cPCA (classwise Principal Component Analysis) e ICA (Independent Component Analysis) mejoran el desempeño del sistema, en cuanto a la reducción del tamaño de segmentos en ms y la identificación de componentes independientes en las señales EMG.

2.2.2 EMG feature evaluation for improving myoelectric pattern recognition robustness [Phinyomark et al., 2013]

Aunque se muestra que las clasificaciones multiclase para señales mioeléctricas son buenas, presentan problemas para seguir trabajando adecuadamente a largo plazo. En este artículo se muestran 50 diferentes características TD y FD para lograr clasificar 10 movimientos del antebrazo, con un conjunto de datos tomado durante 21 días. Se dice que la entropía muestreada (SampEn) es una característica que utilizada en conjunto con un clasificador LDA, logra un desempeño de 93.37 % en promedio, con un decrecimiento de solo 2.3 % en la clasificación luego de 21 días. Esto mejora si se añaden tres características: coeficientes cepstrales de cuarto orden, RMS y WL, ya que el desempeño incrementa a un 98.87 %. Se logra una clasificación en tiempo real. Se señala que el efecto del uso a largo plazo/ uso prolongado de controles basados en EMG es estudiado en pocos trabajos, debido a que las señales capturadas para el entrenamiento son solamente pruebas de un día. Esto hace que difieran los resultados reportados en artículos con los reportes clínicos de uso de prótesis.

En la extracción de características se prefiere el uso de características de tiempo (TD) debido a que presentan un mayor porcentaje de acierto al utilizar clasificadores LDA que las características de frecuencia-tiempo (TFD). Las características TFD (Wavelets y similares) presentan un mejor comportamiento ante los clasificadores SVM que las características TD, pero debido a la robustez del LDA ante el SVM, se prefiere adoptar estrategias que utilicen características TD.

Por otro lado, tanto los estados transitorios como los de estado estacionario de las señales EMG son mejor detectados por características TD utilizando LDA que utilizando TFD y LDA. Se resalta que los coeficientes Wavelet por sí solos no pueden clasificar señales EMG, deben acompañarse por procedimientos como PCA y ULDA (uncorrelated LDA) para lograr un buen desempeño. Esto también depende del nivel de

descomposición con Wavelets y del tipo de Wavelets madre que sean utilizadas.

Se implementaron todas las características enunciadas (50) y se verificaron cuales combinaciones resultaron mejores en la clasificación. Por otro lado, también se probaron diferentes tipos de clasificadores (cuadráticos, lineales y Random Forest - RFS), pero solo se mostraron los resultados con LDA.

Las ventanas recomendadas van desde los 125 ms con variaciones que deben ser menores a 300 ms para garantizar el funcionamiento en tiempo real.

Los segmentos que mejores desempeños mostraron fueron de 500 ms, pero con sobrelapes entre ellos.

Se cree que las características encontradas como mejores serán de utilidad para trabajos futuros, ya que la forma de realizar los experimentos y los conjuntos de datos para el entrenamiento fueron los mismos.

2.3 Antecedentes del uso de RNS en el procesamiento de señales y en arquitecturas digitales

2.3.1 Implementation of RNS-based distributed arithmetic discrete wavelet transform architectures using field-programmable logic [Ramírez et al., 2003]

Actualmente existen ciertas barreras para realizar procesamiento de señales de alta precisión con dispositivos de lógica reprogramable FPL. En este artículo se explora el uso de aritmética distribuida junto al Sistema Numérico de los Residuos (RNS) para lograr diseños centrados en el uso de FPL. Este paradigma se utiliza en el contexto de un banco de filtros de alto desempeño y una Transformada Wavelet Discreta. Se comparan posteriormente los desempeños de las implementaciones del filtro con Complemento a Dos – Aritmética Distribuida (2C-DA) y RNS-DA, donde se muestra que el sistema RNS-DA goza de mayor precisión.

Los sistemas típicos 2C poseen problemas en el procesamiento debido a que la propagación del acarreo limita los canales de las arquitecturas. Por ello, se muestra que con el RNS y la fragmentación de las palabras en otras más pequeñas e independientes, se pueden crear múltiples canales que pueden trabajar de una mejor manera que el complemento a dos.

Se realiza una introducción al manejo de filtros para Wavelets, conceptos de RNS y algunos de aritmética distribuida. Luego de ello, se presentan tres modelos base para realizar el banco de filtros: uno con complemento a dos, otro con complemento a dos y aritmética distribuida y el tercero con aritmética distribuida y RNS:

En el momento de lograr hacer las implementaciones, se hicieron mejoras para lograr reducir el hardware implicado en el diseño, principalmente en los acumuladores RNS basados en Carry Save Adders.

Se lograron realizar mejoras significativas respecto a la velocidad utilizando RNS y aritmética distribuida, hasta con un 156.27% de incremento en la frecuencia de trabajo.

También se muestran algunos procedimientos para la conversión binario-RNS y RNS-binario y la cantidad de hardware consumido, mostrando algunas mejoras para este cuello de botella.

Se muestra que el uso de RNS-DA resulta ventajoso en aplicaciones DSP de alta precisión respecto a la aritmética convencional de complemento a dos.

2.3.2 A Deep Convolutional Neural Network Based on Nested Residue Number System [Nakahara and Sasao, 2015]

En el sistema propuesto se toma una Red Neuronal Convolutiva Profunda (DCNN) pre-entrenada para su uso en sistemas embebidos en visión artificial. Para su implementación física, utilizan una plataforma FPGA Xilinx Virtex VC707, debido a su menor consumo de potencia respecto a plataformas GPU (Graphics Process Unit). Se resalta principalmente que en las DCNN el 90% del tiempo de cómputo se usa para calcular convoluciones de dos dimensiones (2D), lo cual puede ser mejorado al utilizar el Sistema Numérico de los Residuos Anidado (NRNS). El NRNS es una variante del RNS donde se hace una descomposición recursiva del RNS, que conlleva a fragmentar unidades de multiplicación-acumulación (MAC) de 48 bits a unidades de 4 bits con *lookup-tables*.

En términos de desempeño, el número de Giga Operaciones por Segundo por cada *slice* es 5,86 veces superior a realizaciones previa, citadas en el mismo trabajo.

2.3.3 A multifunctional Unit for designing efficient RNS-based datapaths [Molahosseini et al., 2017]

En este trabajo se proponen estrategias para implementar arquitecturas de procesamiento basado en RNS en ASIC, a partir de VHDL sintetizable. Para ello, se muestran esquemas específicos en dos unidades: Unidad Multifuncional y Unidad Aritmética. En la Unidad Aritmética se realizan las operaciones de adición, sustracción, multiplicación. Por otro lado, la Unidad Multifuncional realiza operaciones más complejas en RNS: comparar magnitudes, comparar signos, operaciones de escalamiento y conversión entre números enteros y RNS. Finalmente, ambas unidades se manejan a través de una Unidad de Control. Al realizarse medidas de desempeño, se observó que presenta ventajas en consumo de área y potencia, incluso al realizar operaciones de la Unidad Multifuncional.

2.3.4 RNS Hardware Matrix Multiplier for High Precision Neural Network Acceleration: RNS TPU [Olsen, 2018]

Para este trabajo se muestra una implementación en hardware de un circuito aritmético que permite realizar multiplicaciones matriciales basado en el RNS. Como ventajas se establece un incremento en la precisión de los cálculos en las etapas de entrenamiento, debido a la aritmética en residuos que presenta el RNS. A través de la implementación de este sistema en una FPGA Cyclone- IV, se estableció que el desempeño del multiplicador matricial en RNS era de siete a ocho veces superior a un equivalente en aritmética binaria típica. Por ejemplo, para un multiplicador de matrices 9x9, la máxima frecuencia de trabajo en RNS era de 287 MHz, mientras que el equivalente binario solo alcanzaba los 76 MHz.

2.3.5 Rnsnet: In-memory neural network acceleration using residue number system [Salamat et al., 2018]

Se presenta la realización de una Red Neuronal Profunda (DNN) implementada con Procesamiento en Memoria (PIM) en conjunto con RNS. El soporte hardware

utilizado fue una plataforma basada en arreglos de memristores: Memristor Aided loGIC (MAGIC), la cual fue simulada y llevada a ASIC en un nodo tecnológico de 45 nm.

Aprovechando la plataforma MAGIC, se utilizó un modelo de arquitectura de Red Neuronal en RNS, la cual parte de convertir las características al espacio de los residuos RNS, para posteriormente utilizar una función de activación utilizando una expansión de Series de Taylor, y finalmente una comparación en RNS para determinar las clases. El uso de series de Taylor permite aproximar con operaciones MAC, funciones de activación no lineales, como la sigmoidea, ReLU (*rectified linear unit*). Cabe resaltar que RNSnet fue evaluada en aplicaciones de reconocimiento de texto manuscrito, reconocimiento de voz, reconocimiento de objetos, y localización en entornos cerrados.

Al realizar pruebas, se comprobó que RSNnet puede realizar sus funciones 35,4 veces más rápido y con un consumo energético 145,5 veces menor que implementaciones en GPU.

2.4 Antecedentes del uso de NTT en el procesamiento de señales y en arquitecturas digitales

2.4.1 Pattern Analysis under Number Theoretic Transforms [Moraga, 2012]

En este artículo se documentó el uso de las NTT en dos dimensiones espaciales (2D-NTT) para realizar análisis de geometrías y cambios cromáticos en el procesamiento de imágenes. Se concluye a través de un análisis teórico, que las NTT son útiles para detectar ruido en patrones simétricos.

2.4.2 Video Filtering with Fermat Number Theoretic Transforms Using Residue Number Systems[Toivonen and Heikkila, 2006]

En este trabajo se exploró la forma de realizar operaciones de convolución en dos dimensiones a través de transformadas NTT, para aplicaciones de procesamiento de imágenes y video. Adicional a ello, se utilizó el RNS para lograr incrementar la

máxima longitud en los operadores de la NTT, fraccionando los datos en partes de menor longitud de bits. Para realizar las convoluciones, se acudió a bloques mariposa compatibles con el RNS. Al implementar todo el sistema en un ASIC de tecnología de $0,180\text{ }\mu\text{m}$, se encontró que el circuito resultante era eficiente en área y velocidad respecto a una implementación equivalente utilizando solo las NTT.

2.4.3 Quadratic modified fermat transforms for fast convolution and adaptive filtering [Radhakrishnan and Jenkins, 2010b]

En este trabajo se utiliza la Transformada Numérica de Fermat Modificada para realizar operaciones de convolución en un filtro adaptativo tipo BLMS (*Block Least Mean Square*). La arquitectura resultante muestra un incremento en la eficiencia de cómputo y un potencial uso en filtros que requieran bajo consumo de potencia.

2.4.4 Using Fermat Number Transform to Accelerate Convolutional Neural Network [Xu et al., 2018]

En este trabajo se mostró que la Transformada Numérica de Fermat (FNT) en dos dimensiones es útil en la aceleración de convoluciones para las CNN. Para su implementación se utilizaron módulos mariposa, cuya arquitectura interna posee bloques aritméticos de suma y desplazamiento, a diferencia de los bloques de Transformadas Rápidas de Fourier (FFT) donde hay sumas y multiplicaciones. Es de resaltar que las operaciones de desplazamiento son menos complejas de implementar en hardware que los bloques de multiplicación. Para observar su comportamiento experimental, se acudió a implementar el diseño en la plataforma Xilinx Virtex-7 XC7VX485t. Al evaluar su desempeño con implementaciones similares, se observó que su desempeño incrementó en 1,41 veces, utilizando 3,05 veces menos bloques DSP. Además, se mostró las FNT requieren una complejidad de cómputo mucho menor que las FFT en tareas de convolución.

Fuente			DSP				EMG				SDBC		SNNC	¿Implementado?
Ref.	Foco Principal	Foco Secundario	Preproce	Extracci de caracter	Clasifica de Patrones	Selección de caracter	Adquisición	Selección del clasificador	EMG miembro superior	Reprogr	otros	Tipo		
[Siwiec et al., 2016]	DSP	EMG					x				ASIC	No	Sí	
[Díaz et al., 2010]	EMG	DSP	x	x	x	x	x	x	x	FPGA		No	Sí	
[Boschmann et al., 2017]	EMG	DSP	x	x	x	x	x	x	x	SoC		No	Sí	
[Borbély et al., 2014]	EMG	SDBC	x	x	x	x	x	x	x	SoC		No	Sí	
[Bhamra et al., 2015]	SDBC	EMG					x				ASIC	No	Sí	
[Hakonen et al., 2015]	EMG	DSP	x	x	x	x	x	x	x	x	x	No	Sí	
[Phinyomark et al., 2013]	EMG	DSP	x	x	x	x	x	x	x			No	Sí	
[Ramírez et al., 2003]	DSP	SNNC		x						PLD		RNS	Sí	
[Nakahara and Sasao, 2015]	DSP	SNNC		x	x					FPGA		RNS	Sí	
[Molahosseini et al., 2017]	SNNC	SDBC									ASIC	RNS	Sí	
[Olsen, 2018]	SNNC	DSP		x	x					FPGA		RNS	Sí	
[Salamat et al., 2018]	SNNC	DSP		x	x						ASIC	RNS	Sí	
[Moraga, 2012]	SNNC	DSP		x								NTT	No	
[Toivonen and Heikkila, 2006]	SNNC	DSP	x								ASIC	NTT/RNS	Sí	
[Radhakrishnan and Jenkins, 2010b]	SNNC	DSP	x									NTT	Sí	
[Xu et al., 2018]	SNNC	DSP		x	x					FPGA		NTT		
PR.	Los tres focos		x	x	x	x		x	x	x		x	x	

Cuadro 2.1: Tabla comparativa para el estado del arte

Partiendo de los artículos principales que se han descrito, se puede hacer una tabla comparativa (ver tabla 2.1) entre los tres focos: Análisis de señales mioeléctricas (EMG), Sistemas de Procesamiento Digital de Señales (DSP), Sistemas Digitales de Bajo Consumo (SDBC) y Sistemas Numéricos No Convencionales (SNNC). Cada uno de estos focos fue especificado a través de categorías de incidencia:

- **Sistemas de Procesamiento Digital de Señales (DSP):** Se detalló la incidencia de las investigaciones en: Preprocesamiento de señales, extracción de características y clasificación de señales.
- **Análisis de señales EMG (EMG):** Se detalló la incidencia de las investigaciones en: Adquisición de señales EMG, selección de características, aplicación en miembros superiores y selección para el clasificador.
- **Sistemas Digitales de Bajo Consumo (SDBC):** Se observó la tendencia en la implementación hardware, clasificando las investigaciones en aquellas que utilizaron dispositivos reprogramables (como FPGA, SoC, PLC) o en otros dispositivos (como ASIC's).
- **Sistemas Numéricos No Convencionales (SNNC):** Para este caso se buscó la tendencia de uso de los SNNC, principalmente en los RNS y las NTT, puesto que son utilizadas para aplicaciones DSP de diversa complejidad computacional.

Finalmente, se resaltó que los sistemas propuestos en los artículos fueran implementados, puesto que algunos de ellos solamente ofrecen planteamientos teóricos para afirmar criterios de mejoras de desempeño.

A partir del análisis de antecedentes, se observa que dentro de los referentes revisados de las últimas dos décadas no hay una implementación en hardware de bajo consumo para la evaluación del desempeño de los bloques de preprocesamiento, extracción de características y clasificación de patrones, específicamente para el análisis de señales EMG en prótesis electromecánicas de miembros superiores utilizando los sistemas numéricos mencionados. Esto permite plantear una propuesta de investigación que cubra estos aspectos, corriendo la frontera del conocimiento en la intersección de los focos descritos. En detalle, la propuesta abarcaría el uso de sistemas numéricos no convencionales en las etapas de preprocesamiento, extracción de características y

clasificación de patrones, para determinar si hay incremento en el desempeño en plataformas de bajo consumo para el análisis de señales EMG en prótesis electromecánicas de miembros superiores.

La adquisición de señales EMG no se contempla en el trabajo, puesto que realizar una adquisición EMG directamente en Sistemas Numéricos No Convencionales requiere el desarrollo de dispositivos ASIC para la conversión de señales analógicas, como los *Analog to Residue Converter* (A/R) [Abdelfattah et al., 2010] [Zhu et al., 2012] o los *Residue to Analog Converter* (R/A) [Abdelfattah, 2011]. Esto implica la búsqueda de bases de datos de señales EMG para la realización de la propuesta.

Debido a que la población de interés son personas amputadas en miembros superiores, deben buscarse bases de datos que incluyan la adquisición de señales EMG en pacientes amputados, con variedad de tipos de movimientos. Existe un proyecto llamado Nina Pro [Nina, 2019], que ofrece varios conjuntos de datos para señales sEMG. En el proyecto mencionado, existe una comparativa entre los diferentes conjuntos de datos y su desempeño en la clasificación de movimientos de mano [Pizzolato et al., 2017].

3. Revisión de literatura

3.1 Marco Teórico

3.1.1 Señales mioeléctricas

Las señales mioeléctricas (EMG) son señales eléctricas que provienen de la actividad muscular al realizar una contracción voluntaria, y las cuales pueden ser capturadas de diversas formas, dependiendo del tipo de interfaz que sea utilizada. Los métodos utilizados para medir este tipo de señales incluye:

- **Adquisición superficial (sEMG):** La señal se toma a través de electrodos puestos sobre la piel en el área de interés.
- **Adquisición intramuscular (iEMG):** Los electrodos se insertan a través de la piel, en el tejido muscular.
- **Adquisición percutánea (pEMG):** Se inserta una aguja o alambre debajo de la piel y tejido subcutáneo sobre la aponeurosis del músculo.

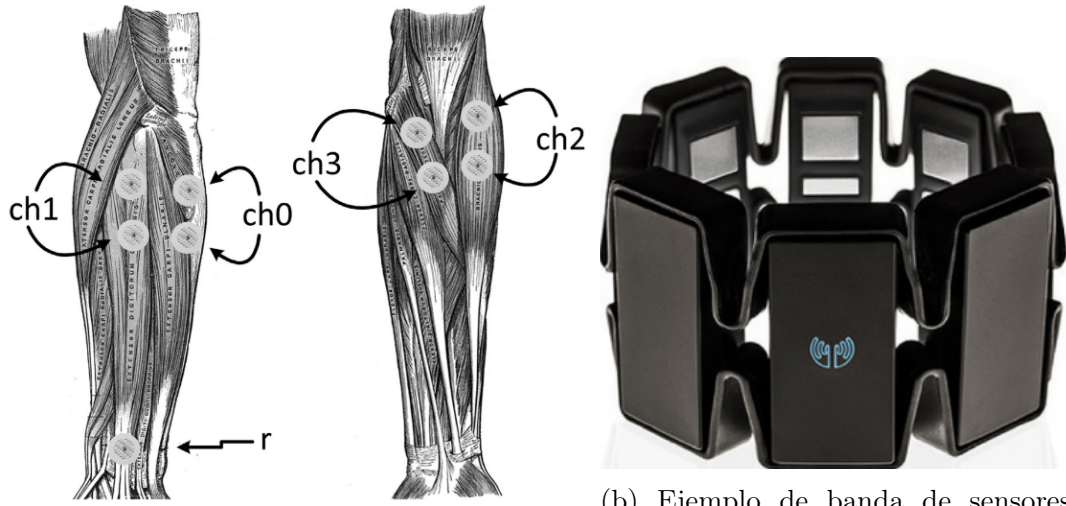
Realizando una comparativa de estos métodos, se observa que el uso de sEMG tiene ciertas ventajas: es no invasivo y sus costos son inferiores. Además, las señales tipo sEMG de un electrodo obtienen la superposición de señales de múltiples músculos en un solo canal, a diferencia de las iEMG o pEMG, que son mucho más selectivas debido a su condición subcutánea o intramuscular. Esto significa que las señales tipo sEMG pueden reunir mayor cantidad de información muscular con una menor cantidad de elementos de medición. Como desventajas, los desplazamientos de los electrodos o los cambios en la impedancia de la piel podrían representar problemas en el análisis de este tipo de señales [Hakonen et al., 2015].

3.1.2 Adquisición y adecuación de señales mioeléctricas

La adquisición de señales SEMG para el control de prótesis basadas en el reconocimiento de patrones, típicamente utiliza electrodos ubicados aproximadamente en las regiones de los músculos *flexor digitorum superficialis*, *flexor capri ulnaris*, *extensor capri radialis longus o brevis*, y *extensor capri ulnaris* [Phinyomark et al., 2013] (ver figura 3.1.a).

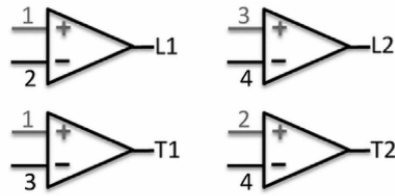
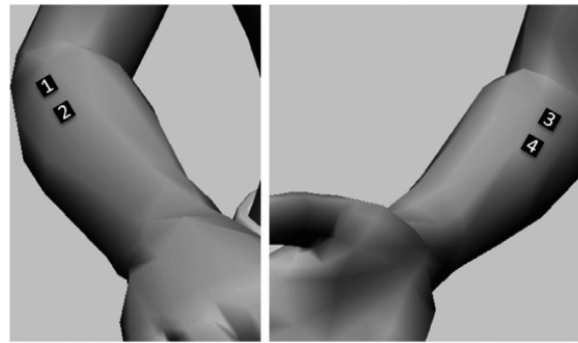
El número de electrodos para la toma de muestras puede variar, existiendo el caso de cuatro sensores de canal bipolar (ver figura 3.1.c) hasta casos donde se utilizan arreglos de sensores en forma de brazalete (ver figura 3.1.b) o incluso arreglos de 192 sensores.

La cantidad de información incrementa al aumentar la cantidad de sensores, pero el procesamiento de las señales es más complejo. Esto puede afectar el desempeño de una plataforma basada en sistemas embebidos, debido al esfuerzo computacional que realiza [Boschmann et al., 2017].



(a) Ubicación aproximada de los electrodos en combinada con unidades de medida el antebrazo para cuatro canales [Phinyomark de inercia (IMU) para nueve ejes de et al., 2013]

(b) Ejemplo de banda de sensores la marca Myo [Hakonen et al., 2015]



(c) Arreglo de sensores para la adquisición bipolar[Hakonen et al., 2015]

Figura 3.1: Adquisición de señales mioeléctricas

Este tipo de señales presentan una forma no-estacionaria, y por lo general se utilizan técnicas de enventanado para lograr analizar la información que almacenan. Usualmente sus amplitudes se encuentran por debajo de los 10 mV, y el 95 % de su contenido energético se encuentra debajo del rango de frecuencias de los 400 - 500 Hz, lo cual se

limita con un filtro paso bajo con una frecuencia de corte entre los 400-450 Hz. Por lo general, se debe utilizar un filtro paso alto con frecuencia de corte de 60 Hz para eliminar significativamente el ruido por el movimiento de cables y electrodos, y la interferencia de 60 Hz de la corriente alterna. También se ha establecido que el contenido energético en la banda de frecuencias de 20-100 Hz no mejora mucho la exactitud de la clasificación (entre un 0.25 % y un 1.6 %). Para realizar el muestreo de las señales EMG, se utilizan ventanas de tiempo que pueden presentar una duración variable, estableciendo que una duración de $t \geq 200\text{ms}$, puede recolectar suficiente información para la clasificación de movimientos (ver figura 3.2)

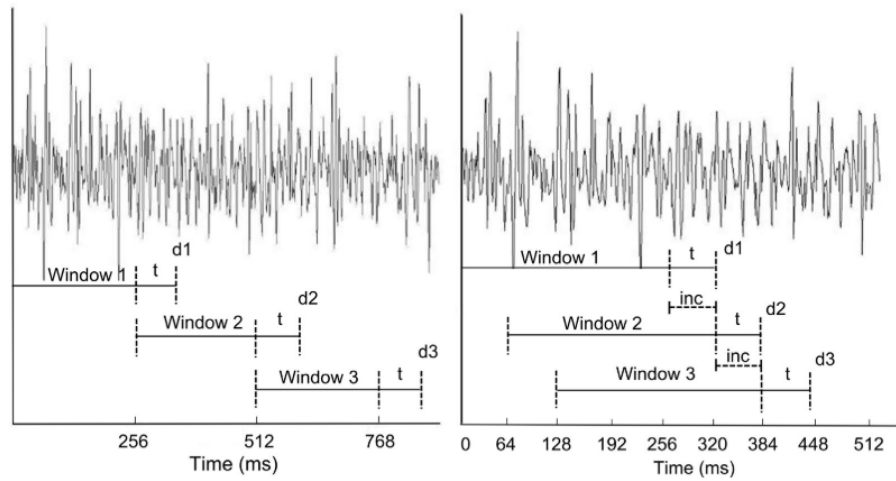


Figura 3.2: Ejemplo de señal mioeléctrica con segmentación: adyacente a la izquierda y con solapamiento a la derecha [Hakonen et al., 2015]

3.1.3 Extracción de características

Las características de las señales mioeléctricas se toman para poder reducir la dimensionalidad de las señales de entrada, extrayendo la información más importante utilizando diversos tipos de transformaciones. De esta manera, los vectores resultantes para posteriores análisis muestran una menor complejidad, mejorando el desempeño en términos de velocidad. Por otro lado, las características pueden ser agrupadas en cuatro categorías:

- Características del dominio temporal (TD).
- Características del dominio frecuencial (FD).
- Características del dominio temporal-frecuencial (TFD).
- Características del dominio espacial (SD).

Existen variantes de las características TD, utilizando Momentos Estadísticos de Orden Superior (HOS) que resultan como una buena forma de extracción de características en términos de facilidad de cómputo [Díaz et al., 2010]. Es recomendable utilizar un conjunto de características que pertenezcan a diferentes dominios para obtener una mejor clasificación [Hakonen et al., 2015].

Por otro lado, aunque existen diversas maneras de realizar la extracción de características y dado que es un campo de investigación muy amplio, solo se entrará en detalle sobre algunas técnicas relevantes.

3.1.3.1 Características del Dominio Temporal

Las características en el dominio del tiempo son usualmente rápidas y fáciles de implementar, debido a que no se necesita ninguna transformación, y se calculan a partir de las secuencias EMG crudas. Existe una amplia variedad de este tipo de características (ver tabla 3.1), y son ampliamente utilizadas tanto en las prácticas e investigaciones médicas y de ingeniería [Phinyomark et al., 2012]. Como desventaja, las características TD son sensitivas al ruido y a interferencias.

Característica	Definición matemática	Anotación
EMG Integrada (IEMG)	$IEMG = \sum_{i=1}^N x_i $	En este caso, x_i corresponde a cada muestra en el segmento i -ésimo y N denota la longitud de la señal EMG.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Valor Absoluto Medio (MAV)	$MAV = \frac{1}{N} \sum_{i=1}^N x_i $	Es una de las características más populares en el análisis de señales EMG. Se define como el promedio del valor absoluto de la amplitud de la señal EMG en un segmento.
Valor Absoluto Medio Modificado tipo 1(MAV1)	$MAV1 = \frac{1}{N} \sum_{i=1}^N w_i x_i $ $w_i = \begin{cases} 1, & \text{si } 0.25N \leq i \leq 0.75N \\ 0.5, & \text{otro caso} \end{cases}$	Es una extensión de la característica MAV. En este caso se asignan unos pesos w_i para ciertos segmentos de la señal, mejorando la robustez de la característica MAV.
Valor Absoluto Medio Modificado tipo 2(MAV2)	$MAV2 = \frac{1}{N} \sum_{i=1}^N w_i x_i $ $w_i = \begin{cases} 1, & \text{si } 0.25N \leq i \leq 0.75N \\ \frac{4i}{N}, & \text{cuando } 0.25 < i \\ \frac{4(i-N)}{N}, & \text{otro caso} \end{cases}$	Es una extensión de la característica MAV similar a la MAV1. Se diferencia en que realiza un mayor suavizado con los pesos w_i , y esta ecuación de ponderación resulta en una función continua.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Integral Cuadrática Simple (SSI)	$SSI = \sum_{i=1}^N x_i^2$	Utiliza la energía de la señal como característica. Es la sumatoria de los valores cuadráticos de la amplitud de la señal. Generalmente, es tratada como un índice energético.
Varianza de la señal EMG (VAR)	$VAR = \frac{1}{N-1} \sum_{i=1}^N x_i^2$	Es definida como el promedio de los valores cuadráticos de la desviación de la señal. Aún así, como el valor esperado de la señal EMG es cercano a cero ($\sim 10^{-10}$), se simplifica la expresión.
Valor absoluto de los tercer, cuarto y quinto momento	$TM3 = \left \frac{1}{N} \sum_{i=1}^N x_i^3 \right $ $TM4 = \left \frac{1}{N} \sum_{i=1}^N x_i^4 \right $ $TM5 = \left \frac{1}{N} \sum_{i=1}^N x_i^5 \right $	Se toma el valor absoluto de los momentos para reducir la separación dentro de la clase para el caso del momento impar. El primer momento y el segundo momento son similares a las características MAV y VAR, respectivamente.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Media cuadrática (RMS)	$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$	Es una característica popular utilizada en el análisis de señales EMG, modeladas como un proceso aleatorio Gaussiano de amplitud modulada que se relaciona con la fuerza constante y la contracción sin fatiga. Es similar a la desviación estándar.
El Orden V (<i>v-Order</i> , V)	$V = \left(\frac{1}{N} \sum_{i=1}^N x_i^v \right)^{\frac{1}{v}}$	Cada muestra de la señal EMG, puede modelarse como: $x_i = (\gamma m_i^\alpha) n_i$. m_i representa la fuerza de contracción, γ y α son constantes, y n_i es la clase de los procesos gaussianos ergódicos. Teóricamente, α es de valor 0.5, aunque en la práctica oscila entre 1 y 1.75. La forma del orden v es similar al RMS cuando $v = 2$, considerado como el valor óptimo.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Detector logarítmico (LOG)	$LOG = e^{\frac{1}{N} \sum_{i=1}^N \log(x_i)}$	Similar al orden v, esta característica también provee información acerca de la fuerza de contracción muscular. Aún así, la definición del detector no-lineal se cambia para basarse en el logaritmo.
Longitud de la Forma de Onda (WL)	$WL = \sum_{i=1}^{N-1} x_{i+1} - x_i $	Es una medida de la complejidad de la señal EMG. Es definida como la longitud acumulativa de la forma de onda de la señal EMG. Cierta literatura se refiere a esta característica como (WAVE).
Cambio Promedio de Amplitud (AAC)	$AAC = \frac{1}{N} \sum_{i=1}^{N-1} x_{i+1} - x_i $	El Cambio Promedio de Amplitud es cercanamente equivalente a la característica WL, excepto que la longitud de la señal es promediada.
Diferencia del valor de la desviación estándar absoluta (DASDV)	$DASDV = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N-1} (x_{i+1} - x_i)^2}$	Es similar a la característica RMS, o en otras palabras, es la desviación estándar de la longitud de señal.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Cruce por Cero (ZC)	$ZC = \sum_{i=1}^{N-1} [sgn(x_i \cdot x_{i+1}) \cap x_i - x_{i+1} \geq \text{threshold}]$ $sgn(x) = \begin{cases} 1, & \text{si } x \geq \text{threshold} \\ 0, & \text{otro caso} \end{cases}$	Es una medida de la frecuencia de la información definida en el dominio del tiempo. Es definida como el número de veces que la amplitud de la señal cruza el nivel cero. Para evadir fluctuaciones o ruido de fondo, se implementa una condición umbral (<i>threshold</i>).
Tasa de Porcentaje de Miopulso (MYO)	$MYOP = \frac{1}{N} \sum_{i=1}^N [f(x_i)]$ $f(x) = \begin{cases} 1, & \text{si } x \geq \text{threshold} \\ 0, & \text{otro caso} \end{cases}$	Es un valor promedio de la salida mioeléctrica, la cual es definida como 1 cuando el valor absoluto de la señal excede un valor umbral predefinido.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Amplitud de Willison (WAMP)	$\text{WAMP} = \sum_{i=1}^N [f(x_i - x_{i+1})]$ $f(x) = \begin{cases} 1, & \text{si } x \geq \text{threshold} \\ 0, & \text{otro caso} \end{cases}$	Es una medida de la frecuencia en la información de la señal EMG, similar a la característica ZC. Esta característica indica el número de veces en que la diferencia de amplitud de dos segmentos adyacentes exceden un umbral predefinido. Además, está relacionado con el disparo de los potenciales de acción de las unidades motoras (MUAP) y la contracción muscular.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Cambio de Signo en la Pendiente (SSC)	$SSC = \sum_{i=2}^{N-1} [f[(x_i - x_{i-1}).(x_i - x_{i+1})]]$ $f(x) = \begin{cases} 1, & \text{si } x \geq \text{threshold} \\ 0, & \text{otro caso} \end{cases}$	Es otro método para representar información frecuencial de la señal EMG. Es el número de veces en que la pendiente de la señal EMG cambia de signo. El numero de cambios entre pendientes positivas y negativas sobre tres segmentos secuenciales se realiza con una función de umbral para evitar el ruido de fondo en la señal EMG.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Pendiente de valor absoluto medio (MAVSLP)	$\text{MAVSLP}_k = \text{MAV}_{k+1} - \text{MAV}_k$ $k = 1, \dots, K - 1$	Es una versión modificada de la característica MAV utilizada para establecer múltiples características. Las diferencias entre MAV's de segmentos adyacentes son determinados. K representa el número de segmentos que cubren la señal EMG. Si el número de segmentos incrementa, puede mejorar la representación de la señal respecto a una característica MAV tradicional. Se recomienda el uso de tres segmentos ($K = 3$).

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Ventanas Hamming Múltiples (MHW)	$\text{MHW}_k = \sum_{i=0}^{N-1} (x_i \cdot w_{i-i_k})^2, k = 1, \dots, K$	La idea principal del uso de múltiples enventanados es lograr evidenciar el cambio energético en la señal EMG con respecto al tiempo. En este procedimiento, se segmenta la señal utilizando ventanas Hamming en toda la secuencia y posteriormente se hace el cálculo de energía sobre cada una de las ventanas. Se recomienda el uso de tres segmentos con un 30 % de solapamiento.
Ventanas Trapezoidales Múltiples (MTW)	$\text{MTW}_k = \sum_{i=0}^{N-1} (x_i^2 \cdot w_{i-i_k}), k = 1, \dots, K$	Esta característica es un tipo de MHW. Similarmente, se busca el contenido energético dentro de ventanas, pero la función de enventanado w cambia de la ventana Hamming a la ventana trapezoidal. Se recomienda el uso de tres segmentos con un 30 % de solapamiento.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Histograma de la señal EMG (HIST)	$h_k = \sum_{i=1}^N x_i,$ $\text{si } x_i \in [(k-1)\frac{N}{B}, k\frac{N}{B}],$ $k \in [0, B]$	En esta característica, se toma la señal EMG cruda y sus elementos son divididos en B segmentos espaciados de manera equidistante y se retorna el número de elementos de la señal por cada segmento.
Coefficientes autorregresivos (AR)	$x_i = \sum_{p=1}^P a_p \cdot x_{i-p} + w_i$	El modelo autoregresivo es un modelo predictivo que describe cada muestra de la señal EMG como una combinación lineal de las muestras anteriores x_{i-p} y un término de ruido blanco w_i . En la clasificación de señales EMG, los coeficientes de del modelo AR, denominados a_p son utilizados como vector de características. P es el orden del modelo AR. Se recomienda utilizar un modelo de cuarto orden $P = 4$.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Coeficientes Cepstrales (CC)	$c_1 = -a_1;$ $c_p = -a_p - \sum_{l=1}^{p-1} \left(1 - \frac{l}{p}\right) a_p \cdot c_{p-l}$	Se definen como la Transformada Inversa de Fourier del logaritmo de la magnitud del espectro de potencia de los datos de la señal. Los Coeficientes Cepstrales han sido utilizados como una característica en conjunto con los coeficientes del modelo AR. Además, los coeficientes cepstrales pueden ser derivados del modelo AR. c_p es el coeficiente de p-ésimo orden del análisis CC y $1 \leq l \leq P$. De esta definición, se puede considerar que se trata de una característica del dominio del tiempo debido a que no requiere una Transformada de Fourier para ser determinada. De la sugerencia del modelo AR, se sugiere que el análisis CC sea de cuarto orden.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
----------------	-----------------------	-----------

Cuadro 3.1: Características del dominio temporal [Phinyomark et al., 2012].

Cabe resaltar que los valores umbral típicos para las características ZC, MYOP, WAMP y SSC oscilan entre los $50\mu V$ y los $100mV$

3.1.3.2 Características del Dominio Frecuencial

Las características del dominio frecuencial (ver tabla 3.2) son muy utilizadas para estudiar la fatiga de los músculo y el análisis de enganche en unidades motoras (*MU recruitment analysis*). En este listado aparece la densidad espectral de potencia (PSD) como una herramienta de peso en el dominio frecuencial, ya que de ella surgen las principales características. Para esto, se utilizan diversas técnicas estadísticas sobre la PSD, la cual es definida como la Transformada de Fourier de la función de autocorrelación de la señal EMG, aunque también puede ser estimada con el Periodograma o los modelos paramétricos, por ejemplo el modelo AR. Dos de las características más ampliamente utilizadas que parten del PSD son las frecuencia promedio (*Mean Frequency, MNF*) y la frecuencia mediana (*Median Frequency, MDF*). Existen otras características, como la frecuencia pico (PKF), la potencia media (MNP) y la potencia total (TTP)[Phinyomark et al., 2012].

Característica	Definición matemática	Anotación
Frecuencia Promedio (MNF)	$MNF = \frac{\sum_{j=1}^M f_j P_j}{\sum_{j=1}^M P_j}$	Es una frecuencia promedio calculada como la suma de los productos del Espectro de Potencia de la señal EMG con el segmento de frecuencia, dividido por la suma total de la intensidad del espectro. Otros nombres que recibe son: Frecuencia Central (f_c) y Centro de Gravedad Espectral. Donde f_i es la frecuencia del Espectro en el segmento (<i>bin</i>) de frecuencia j , P_j es el Espectro de Potencia en el segmento j , y M es la longitud del segmento de frecuencia.
Frecuencia Mediana (MDF)	$\sum_{j=1}^{MDF} P_j = \sum_{j=MDF}^M P_j$ $= \frac{1}{2} \sum_{j=1}^M P_j$	Es la frecuencia en la cual el espectro es dividido en dos regiones de igual amplitud. En otras palabras, resulta de dividir por la mitad la potencia total (TTP).
Frecuencia Pico(PKF)	$PKF = \max(P_j)$ $j = 1, \dots, M.$	La PKF es la frecuencia en la cual se presenta la mayor potencia.
Potencia Media (MNP)	$MNP = \frac{1}{M} \sum_{j=1}^M P_j$	Es la potencia promedio del Espectro de Potencia de la señal EMG.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Potencia Total (TTP)	$TTP = \sum_{j=1}^M P_j = SM0$	Se define como la suma de las componentes del Espectro de Potencia de la señal EMG. Otros nombres que recibe son: Momento del Espectro de orden Cero (SM0) y Energía.
Momentos Espectrales de primer, segundo y tercer orden	$SM1 = \sum_{j=1}^M P_j f_j$ $SM2 = \sum_{j=1}^M P_j f_j^2$ $SM3 = \sum_{j=1}^M P_j f_j^3$	Los momentos espectrales son alternativas para realizar un análisis estadístico, extrayendo características de las señales EMG. Los primeros tres momentos (SM1-SM3), son considerados de mayor importancia.

Sigue en la próxima página.

Característica	Definición matemática	Anotación
Relación de Frecuencia (FR)	$FR = \frac{\sum_{j=LLC}^{ULC} P_j}{\sum_{j=LHC}^{UHC} P_j}$	Se propone para lograr distinguir entre la contracción y la relajación del músculo usando la razón entre las componentes de baja frecuencia y de alta frecuencia de la señal EMG. <i>LLC</i> y <i>ULC</i> son las frecuencias alta y baja de corte que definen la banda de frecuencias bajas y <i>LHC</i> y <i>UHC</i> son las frecuencias alta y baja de corte que definen la banda de frecuencias altas. El umbral que divide a ambas bandas puede definirse de dos maneras. La primera forma es determinar experimentalmente estas bandas. Por ejemplo, se han utilizado bandas de baja frecuencia entre los 30 – 250 Hz y la componente de alta frecuencia entre 250 – 1000 Hz. La segunda forma toma como valor divisorio de las bandas la frecuencia mediana MDF. De la definición matemática, la característica FR es el caso inverso de la relación alta a baja (<i>H/L ratio</i>), la cual es utilizada ampliamente en el estudio de la fatiga diafragmática.

Característica	Definición matemática	Anotación
Relación de espectro de potencia (PSR)	$PSR = \frac{P_0}{P} = \frac{\sum_{j=f_0-n}^{f_0+n} P_j}{\sum_{j=-\infty}^{\infty} P_j}$	Puede ser vista como una extensión de las características de frecuencia pico (PKF) y la relación de frecuencia (FR). Se define como la relación entre la energía P_0 la cual es tomada en cercanías al valor máximo del Espectro de Potencia de la señal EMG y la energía P la cual es la energía completa del Espectro de Potencia de la señal EMG. n representa los límites de integración, donde el valor recomendado de n es 20 y la energía P se toma entre los 10 Hz y los 20 Hz, donde se sitúa el mayor contenido energético de la señal EMG.
Varianza de la Frecuencia Central(VCF)	$VCF = \frac{1}{SM0} \sum_{j=1}^M P_j \cdot (f_j - f_c)^2$ $= \frac{SM2}{SM0} - \left(\frac{SM1}{SM0}\right)^2$	La VCF es una característica importante de la PSD, la cual puede definirse utilizando unos pocos momentos espectrales.

Cuadro 3.2: Características del dominio frecuencial [Phinyomark et al., 2012].

3.1.3.3 Características del Dominio Temporal-Frecuencial

Las características TFD utilizadas para la caracterización de las señales EMG incluyen la Transformada de Fourier de Tiempo Reducido (STFT), la Transformada Wavelet de Tiempo Continuo (CWT), la Transformada Wavelet Discreta (DWT), la Transformada Wavelet por Paquetes (WPT), y la Transformada Wavelet Estacionaria (SWT). Un problema de la STFT es la imposibilidad de manejar incrementos simultáneos tanto en el dominio temporal como en el frecuencial. Como alternativa, las diversas Transformadas Wavelet (CWT, DWT, WPT) pueden resolver este con el análisis multiresolución: una baja resolución de tiempo acompaña una buena resolución de frecuencia en la banda de bajas frecuencias, y una alta resolución de tiempo acompaña una baja resolución de frecuencia en la banda de altas frecuencias. Se puede destacar que la DWT es computacionalmente más eficiente que la CWT, resulta ser la más común de las características tiempo-frecuencia más utilizadas en las interfaces EMG. Aunque el cálculo de las características TFD es más complejo comparado con las características TD, pueden ser implementadas con algoritmos eficientes para lograr cumplir con los requerimientos de tiempo real en el procesamiento de las señales EMG. Cabe resaltar que un análisis basado en Wavelets puede mejorar la robustez del sistema respecto a las características TD y FD, debido a que si es utilizado un subconjunto de coeficientes Wavelet, el análisis puede restringirse a las bandas de frecuencia deseadas. Por otro lado, debido al análisis multiresolución, con las TFD se obtienen vectores de características que resultan en una alta dimensionalidad. Para reducir dimensionalidad en los vectores de características, es recomendable realizar proyecciones de características como el método del Análisis de Componentes Principales (PCA) o el uso del Análisis Discriminante Lineal no Correlacionado(uLDA), pero en el caso del análisis Wavelet, puede utilizarse una técnica del dominio del tiempo sobre los coeficientes Wavelet, o sobre la señal EMG reconstruida [Hakonen et al., 2015].

3.1.3.3.1. Transformada de Fourier de Tiempo Reducido: La STFT consiste en una versión modificada de la Transformada de Fourier, donde la señal es analizada en diversos intervalos con una función de ventana g , la cual se desplaza en las diferentes regiones de interés por medio de un coeficiente k . Matemáticamente se describe así:

$$\text{STFT}(k, m) = \sum_{r=1}^{N-1} x(r)g(r-k)e^{\frac{-j2\pi mi}{N}} \quad (3.1)$$

Donde g corresponde a la función ventana, x es la señal EMG, k es el tiempo de muestra, m corresponde a los segmentos (*bins*) de frecuencia y $j = \sqrt{-1}$.

3.1.3.3.2. Transformada Wavelet de Tiempo Continuo (CWT) [Duan et al., 2016]: Una Wavelet $\psi_{a,b}(t)$ se deriva de su Wavelet Madre $\psi(\cdot)$ a través de dilataciones y desplazamientos en el tiempo. Esta relación puede ser definida por:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}} \psi\left(\frac{t-b}{a}\right) \quad (3.2)$$

Donde a y b representan los factores de escala y traslación, respectivamente. La familia de funciones generadas por $\psi(\cdot)$ pueden ser definidas como:

$$\Omega = \{\psi_{a,b}(t), a \in \mathbb{R}, b \in \mathbb{R}, a \neq 0\} \quad (3.3)$$

Ω es llamado como una captura (*frame*) de $L^2(\mathbb{R})$ (espacio vectorial normado) si cumple con la condición de admisibilidad C_ψ :

$$C_\psi = \int_0^{+\infty} \frac{|\psi(\omega)|^2}{\omega} d\omega < +\infty \quad (3.4)$$

Donde $\psi(\omega)$ se genera de $\psi(t)$ utilizando la Transformada de Fourier. De aquí, si se tiene una función $f(t) \in L^2(\mathbb{R})$, se define la Transformada Wavelet de Tiempo Continuo como el producto interno entre la función $f(t)$ y las funciones $\psi_{a,b}(t)$:

$$\begin{aligned} CWT(a, b) &= \langle f(t), \psi_{a,b}(t) \rangle \\ &= \frac{1}{\sqrt{|a|}} \int_{-\infty}^{+\infty} \psi^*\left(\frac{t-b}{a}\right) f(t) dt, a > 0 \end{aligned} \quad (3.5)$$

donde $\psi^*\left(\frac{t-b}{a}\right)$ corresponde al complejo conjugado de $\psi\left(\frac{t-b}{a}\right)$.

3.1.3.3.3. Transformada Wavelet Discreta (DWT) [Duan et al., 2016]: De manera general, la Transformada Wavelet Discreta (DWT) descompone la señal en unos coeficientes de detalle y unos coeficientes de aproximación, pasándola a través de filtros pasa-bajos y pasa-altos complementarios. Estos coeficientes de aproximación se pasan de nuevo por otra descomposición a través de un segundo nivel de filtros. Repitiendo tal proceso, la señal puede ser fragmentada en varias componentes de resolución más baja [Hakonen et al., 2015].

Esta característica resulta de considerar la versión discreta de la Transformada Wavelet para una señal discreta $f(n)$, definida matemáticamente por [Duan et al., 2016]:

$$DWT(j, k) = \frac{1}{\sqrt{|a_0^j|}} \sum_n \psi \left(\frac{n - kb_0 a_0^j}{a_0^j} \right) f(n), a > 0 \quad (3.6)$$

Donde $a = a_0^j$, $b = kb_0 a_0^j$, j , k y n son números enteros y j corresponde a la profundidad de la descomposición. El análisis tiempo-frecuencia basado en la DWT es muy propio para las señales no estacionarias, cuyo espectro de frecuencia es variante en el tiempo. Algunos enfoques muestran el uso de las DWT para el manejo de señales EMG, y se muestra que tienen un mejor desempeño que las características TD. Para simplificar los cálculos, se impone que $a_0 = 2$ y $b_0 = 1$, lo cual es llamado muestreo diádico en el tiempo, y resulta en las llamadas Wavelets Diádicas:

$$\psi_{j,k}(t) = \frac{1}{\sqrt{2^j}} \psi \left(\frac{n}{2^j} - k \right) \quad (3.7)$$

Donde j, k y n son números enteros. Para muchos casos, la solución analítica no puede ser resuelta por la DWT, solo puede obtenerse por métodos numéricos. Por lo tanto, es requerido utilizar un método rápido de cálculo para lograr que la DWT se use de forma práctica. Un método para lograr esto es el algoritmo MALLAT, que puede utilizarse en señales EMG. La señal original A_0 es descompuesta en aproximaciones A_{-1} y detalles D_{-1} , luego la señal A_{-1} es descompuesta en las aproximaciones A_{-2} y detalles D_{-2} , y así sucesivamente, hasta llegar a un nivel predeterminado. Esta relación puede verse como un conjunto de filtros digitales, definidos por las siguientes ecuaciones en diferencias:

$$\begin{aligned} A_m &= \sum_k h(2n - k) A_{m+1}(k) \\ D_m &= \sum_k g(2n - k) A_{m+1}(k) \end{aligned} \quad (3.8)$$

Donde $m = -1, -2, \dots, -M$. Tomando G como un filtro paso alto, representado por la respuesta al impulso $g(k)$, y H como un filtro paso bajo, representado por la respuesta al impulso $h(k)$, se puede establecer una arquitectura MALLAT(ver figura 3.3) y una equivalencia entre filtros:

$$g(k) = (-1)^{1-k} h(1 - k) \quad (3.9)$$

Con el algoritmo MALLAT, el operador $\downarrow 2$ mantiene una de cada dos muestras, y A_0 es la señal original [Duan et al., 2016]

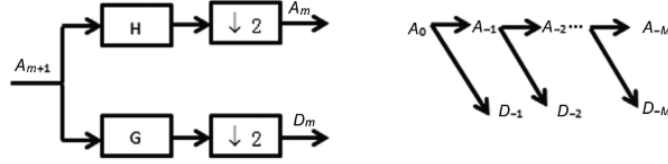


Figura 3.3: Bancos de filtros con el algoritmo MALLAT [Duan et al., 2016]

3.1.3.3.4. Coeficientes Cepstrales en las Frecuencias de Mel (MFCC) Los Coeficientes Cepstrales en las Frecuencias de Mel corresponden a una técnica de procesamiento de señales que permite la extracción de información de interés en una señal de entrada y elimina componentes innecesarias, como ruido.

Aunque el método es bastante utilizado para sistemas de reconocimiento de voz, también se puede emplear en sistemas relacionados con el reconocimiento de patrones para señales EMG [Amor et al., 2019].

Para la obtención de los coeficientes mencionados, se realiza una serie de pasos que involucran filtros digitales, enventanados y Transformadas de Fourier [Muda et al., 2010]:

- **Paso 1 (Pre-énfasis):** Se pasa la señal de entrada por un filtro que realiza las componentes de alta frecuencia, incrementando la energía presente en ellas.
- **Paso 2 (Segmentación):** Cada señal se segmenta en ventanas más pequeñas.
- **Paso 3 (Enventanado Hamming):** Para cada segmento se realiza una multiplicación por una ventana tipo Hamming para reducir la distorsión espectral.
- **Paso 4 (FFT):** Se realiza la FFT para cada uno de los segmentos enventanados.
- **Paso 5 (Banco de Filtros de Mel):** Se realiza una conversión logarítmica a los espectros obtenidos del paso anterior y se pasan por un banco de filtros triangulares de Mel
- **Paso 6 (Transformada Discreta del Coseno):** Se utiliza la Transformada Discreta del Coseno (DCT) al espectro logarítmico que ha pasado por el banco de filtros de Mel. Los coeficientes obtenidos son los MFCC y la aproximación de su primera derivada resulta en una tabla de coeficientes que es utilizada en reconocimiento de patrones.

3.1.3.4 Características del Dominio Espacial/[Hakonen et al., 2015]:

El uso de señales EMG de Alta Densidad (HD-EMG) permite no solamente extraer información del dominio temporal y espectral, sino también de la ubicación espacial. Las características del Dominio Espacial (SD) pueden mejorar la diferenciación entre posturas y niveles de fuerza, al proveer información de la distribución de los potenciales de acción de las unidades motoras (MUAP) y de la distribución de carga entre los músculos. La relevancia de la información espacial está soportada en que diferentes regiones musculares serán activadas dependiendo de la articulación, como la duración y fuerza de la contracción. Como las interfaces HD-EMG solo han sido utilizadas recientemente en las interfaces EMG, hay muy pocos estudios sobre el diseño y la implementación de características SD para este tipo de análisis. Un ejemplo de característica SD es el variodograma experimental. Utilizando este modelo, el rendimiento es similar al uso de características TD, pero con una mayor robustez ante desplazamientos transversales y longitudinales de los electrodos. Por otro lado, el

número de electrodos en la etapa de entrenamiento y la etapa de pruebas, lo que permite excluir electrodos que tengan una débil calidad en la señal detectados al calcular el variodograma de la señal EMG, sin tener que re-entrenar el clasificador.

3.1.3.4.1. Periodograma Experimental: Matemáticamente se puede determinar de la siguiente forma:

$$\gamma(h) = \frac{1}{2n(h)} \sum_{i=1}^{n(h)} [x(z_i) - x(z_i + h)]^2 \quad (3.10)$$

Donde h representa un vector de distancia, $x(z_i)$ representa la medida en la ubicación z_i , $n(h)$ es el número de pares h aparte en la dirección del vector h .

3.1.4 Reconocimiento de patrones

Para realizar el control en interfaces EMG, se acude a dos estrategias: el control basado en reconocimiento de patrones y el que no utiliza reconocimiento de patrones. De manera general, el uso de reconocimiento de patrones resulta preferido en el campo de las señales mioeléctricas, debido a su versatilidad en el esquema de control comparado con métodos donde no se utiliza el reconocimiento de patrones.

El enfoque hacia el uso de reconocimiento de patrones reside en el supuesto de que el clasificador es capaz de reconocer los valores de entrada colocados en su etapa de entrenamiento, y asignarle a cada valor de entrada una clase dentro de un conjunto de categorías. Los valores de entrada corresponden a vectores de características calculados a partir de las señales EMG. El reconocimiento de patrones ha implicado una mejora en el desempeño de los sistemas de control basados en señales EMG. Estudios comparativos sugieren que al escoger un conjunto adecuado de características y un número suficiente de canales de entrada, el desempeño de los diversos tipos de clasificadores puede ser similar. Esto significa que bajo una representación apropiada de las características, la tarea de clasificación puede resultar en un problema lineal. La tendencia actual es utilizar clasificadores sencillos de implementar, rápidos de entrenar, y que cumpla con los requerimientos de tiempo real, como los clasificadores por Análisis de Discriminante Lineal (LDA), las Máquinas de Vectores de Soporte (SVM), los Modelos Ocultos de Markov (HMM), de los cuales

es recomendado el uso de LDA.

Por otro lado, muy pocos estudios se han hecho respecto a la habilidad de los clasificadores EMG para continuar con un buen desempeño para el uso extendido, debido a la adición de ruido y los desgastes y disturbios en los sistemas de adquisición. Existen estudios que señalan a los LDA como uno de los clasificadores con mayor robustez ante los efectos a largo plazo respecto a otros tipos de clasificadores. Se ha encontrado que el LDA supera al perceptrón multicapa (*MLP*) cuando hay cambios en la posición de los electrodos. Aún así, se recomiendan clasificadores de mayor complejidad cuando se trata de concebir sistemas que tengan un buen desempeño ante los efectos de largo plazo [Hakonen et al., 2015].

Al igual que las técnicas relacionadas con extracción de características, la clasificación de patrones muestra un espectro muy amplio y diverso, por lo cual solo serán citadas algunas de las técnicas revisadas en la literatura.

3.1.4.1 Análisis de Discriminante Lineal (LDA):

En el Análisis por Discriminante Lineal (LDA) se busca una transformación lineal óptima por la cual los datos originales sean transformados a un espacio de una dimensionalidad mucho menor. El objetivo principal de un LDA es el de encontrar una transformación que maximice la separabilidad de clases en el espacio de dimensiones reducidas. Por ello, el criterio de reducción de dimensionalidad del LDA se formula para maximizar la dispersión entre las clases y minimizar la dispersión de los datos en una clase [Park and Park, 2008].

Un clasificador por Análisis de Discriminante Lineal (LDA) toma un espacio conformado por los vectores de características y reduce su dimensionalidad, mediante el cálculo y el uso de vectores de proyección que permiten separar mejor las categorías. Luego de obtener los vectores de proyección, se proyectan sobre ellos los demás vectores del espacio de características, conllevando a una reducción de dimensiones, ya que la cantidad de vectores de proyección es menor que las dimensiones originales del conjunto de características [Bence J. Borbély et al., 2013]. Debido a ello, el LDA puede verse como un candidato para trabajar con sistemas embebidos donde se tenga un recurso limitado de memoria y de cómputo, comparado con técnicas como las SVM [Xu et al., 2016].

En un nivel más detallado, el funcionamiento del LDA parte de una etapa de

entrenamiento (ver figura 3.4) donde se deben hallar las matrices de proyección y nivel de compensación (*offset*). Siendo Σ_c la matriz de covarianza (dispersión en una sola clase), la cual se puede definir como:

$$\Sigma_c = \frac{1}{n_c - 1} \sum_{i=1}^{n_c} (x_{ic} - \mu_i)^T (x_{ic} - \mu_i) \quad (3.11)$$

Donde x_{ic} representa el vector completo de características, μ_i indica el vector promedio de características por cada una de las muestras de una clase c , y n_c corresponde al total de muestras de entrenamiento de una clase c . La matriz de covarianza combinada (dispersión entre las clases) se define como:

$$\Sigma = \sum_{c=1}^C \frac{n_c - 1}{N - C} \Sigma_c \quad (3.12)$$

Donde C simboliza el número de clases y N el total de muestras de entrenamiento. De aquí, se puede obtener la matriz de proyección como:

$$W_i = \Sigma^{-1} \mu_i^T \quad (3.13)$$

Si se agrupan los términos en formas de arreglo:

$$\begin{aligned} W &= (W_1, W_2, \dots, W_n) \\ \mu &= (\mu_1^T, \mu_2^T, \dots, \mu_n^T) \end{aligned} \quad (3.14)$$

Se llega a una forma matricial:

$$W = \Sigma^{-1} \mu \quad (3.15)$$

Por otro lado, se debe hallar una matriz de compensación (*bias* u *offset*):

$$w = (w_1, w_2, \dots, w_n)^T \quad (3.16)$$

Donde cada componente se define como:

$$w_i = -\frac{1}{2}\mu_i\Sigma^{-1}\mu_i^T \quad (3.17)$$

Teniendo estos parámetros, se logra la clasificación con una función $g(x)$, definida como:

$$g(x) = W^T x + w \quad (3.18)$$

Donde $g(x)$ es un vector fila, donde cada una de sus componentes representa una clase. La componente que resulte con mayor valor en $g(x)$ para un vector de características x indica la clase a la que tiene mayor posibilidad de pertenecer.

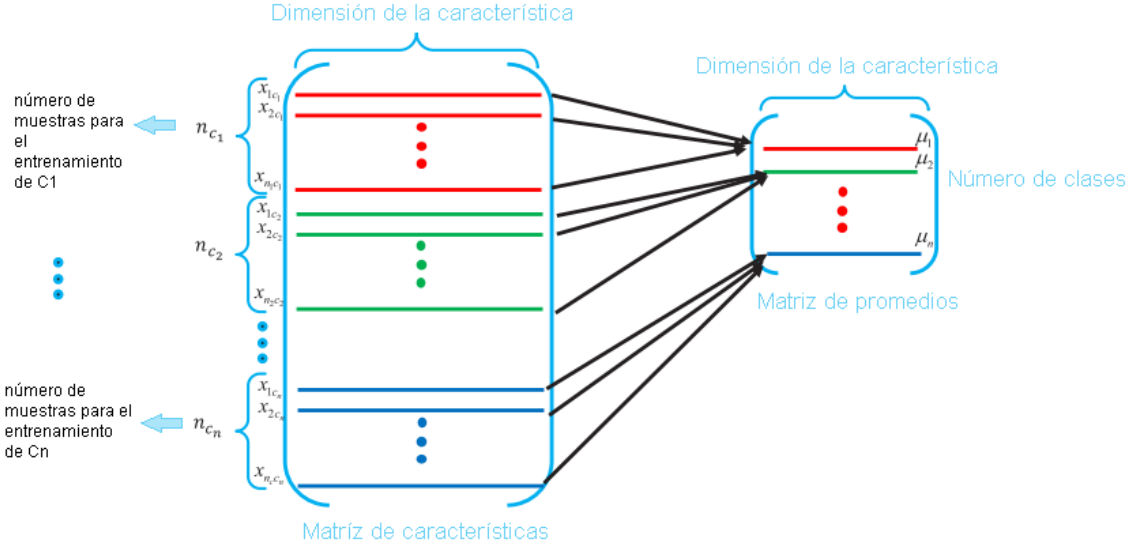


Figura 3.4: Estructura de datos para el entrenamiento del LDA, basado en [Xu et al., 2016]

3.1.4.2 Máquinas de Vectores de Soporte (SVM) [Benatti et al., 2017] [Boser et al., 1992]:

Las SVM consisten en un algoritmo de aprendizaje supervisado que mejora el desempeño de los métodos de regresión logística a través de un largo margen de clasificación. En la etapa de entrenamiento de las SVM, se utilizan datos etiquetados para calcular el hiperplano óptimo de separación entre dos clases de datos, a través de la solución de un problema de optimización convexa. Estos planos de separación son representados por un conjunto de vectores de datos llamados Vectores de Soporte, los cuales pertenecen a los bordes de las dos clases y son utilizados para clasificar las nuevas instancias de datos.

El funcionamiento básico de un clasificador SVM para dos clases parte de crear una función de decisión $D(x)$, cuya dependencia respecto a las características x de n

dimensiones, puede tomar una forma variada (lineal o no lineal) según la función *kernel* que se escoja. La función de decisión puede verse como una superficie multidimensional que maximiza el mínimo margen entre datos que corresponden a diferentes clases (ver figura 3.5) [Dong, 2006].

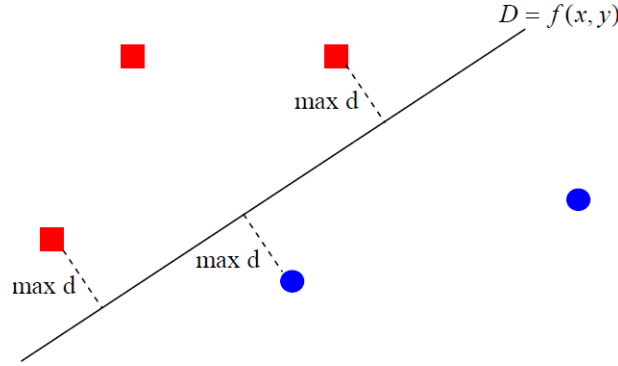


Figura 3.5: Propósito de una superficie de decisión en una SVM, asumiendo dos características x e y y dos clases (cuadros rojos y círculos azules) [Dong, 2006].

La clasificación de características parte de obtener los parámetros de $D(x)$ a través de una etapa de entrenamiento, donde se toma un conjunto de p características entrenamiento con una etiqueta de clase l_i :

$$(\mathbf{x}_1, l_1), (\mathbf{x}_2, l_2), \dots, (\mathbf{x}_p, l_p), \quad (3.19)$$

$$\begin{cases} \text{Si } l_k = 1, x \in \text{clase } C_1 \\ \text{Si } l_k = -1, x \in \text{clase } C_2 \end{cases} \quad (3.20)$$

Después del algoritmo de entrenamiento se puede clasificar un nuevo vector de características x , a través de $D(x)$:

$$\begin{cases} \text{Si } D(\mathbf{x}) > 0, \mathbf{x} \in C_1 \\ \text{Si otro caso, } \mathbf{x} \in C_2 \end{cases} \quad (3.21)$$

La superficie de separación debe expresarse como una dependencia lineal de sus

parámetros, sin implicar una dependencia lineal de x :

$$\begin{aligned}
D(\mathbf{x}) &= \sum_{i=1}^N \omega_i \phi_i(x) + b \rightarrow \text{Espacio primal} \\
D(\mathbf{x}) &= \sum_{k=1}^p \alpha_k K\langle x_k, x \rangle + b \rightarrow \text{Espacio dual} \\
\text{Donde:} & \\
\omega_i &= \sum_{k=1}^p \alpha_k \phi_i(x_k) \\
K\langle x, x' \rangle &= \sum_i \phi_i(x) \phi_i(x')
\end{aligned} \tag{3.22}$$

En la anterior descripción, ϕ_i son funciones predeterminadas del vector x , y ω_i junto a b son parámetros ajustables de la función de decisión. En el espacio dual, los coeficientes α_k son parámetros que deben ajustarse, x y x' son vectores de características, y x_k son patrones de entrenamiento, y la función kernel $K\langle x, x' \rangle$ permite un mapeo de los vectores de características para mejorar la separabilidad de las clases. Esta función kernel puede tomar una forma polinómica:

$$K\langle x, x' \rangle = (x^T \cdot x' + 1)^d \tag{3.23}$$

Cabe resaltar que los valores ω_i , α_k y b son parámetros obtenidos en un proceso de optimización respecto a una margen M , que surge inicialmente de caracterizar adecuadamente la función $D(x)$, con unos vectores en la margen llamados Vectores de Soporte (ver figura 3.6).

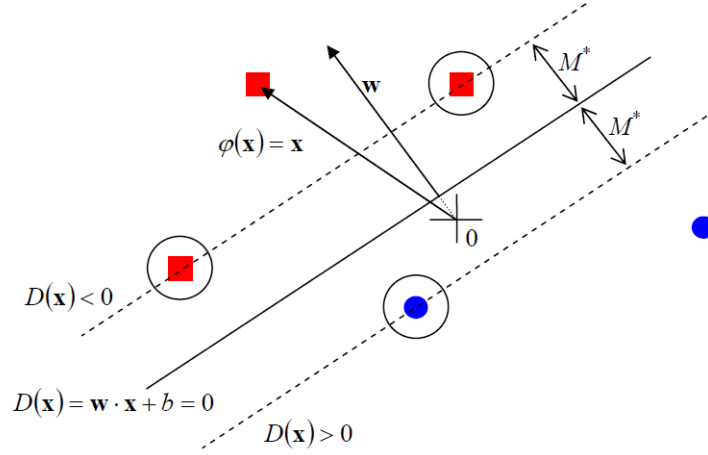


Figura 3.6: Superficie de decisión en una SVM, mostrando el margen de separación permitido entre clases M^* y los vectores de soporte, enmarcados por círculos negros [Dong, 2006].

La descripción primal del plano $D(x)$ se plantea para hallar la margen M :

$$D(\mathbf{x}) = \sum_{i=1}^N \omega_i \phi_i(x) + b \rightarrow D(\mathbf{x}) = \mathbf{w} \phi(x) + b \text{ (Forma vectorial)} \quad (3.24)$$

$\frac{D(\mathbf{x})}{\|\mathbf{w}\|} \rightarrow$ Distancia entre la superficie de decisión y el vector x

Donde w y ϕ son vectores de N dimensiones. Recordando la distancia de los vectores de entrenamiento $\mathbf{x}_k$ al plano $D(\mathbf{x})$ deben superar la margen M :

$$l_k \frac{D(\mathbf{x}_k)}{\|\mathbf{w}\|} \geq M \quad (3.25)$$

De esta desigualdad, se requiere maximizar la margen entre clases en función de los

pesos $\mathbf{w}$, es decir:

$$\begin{aligned}
M^* &= \max M \\
\mathbf{w}, \|\mathbf{w}\| &= 1 \\
\text{Sujeto a:} & \\
l_k D(\mathbf{x}_k) &\geq M \\
k &= 1, 2, \dots, p
\end{aligned} \tag{3.26}$$

El límite M^* se alcanza para los x_k que satisfacen:

$$\min_k l_k D(\mathbf{x}_k) = M^* \tag{3.27}$$

A estos vectores que alcanzan la distancia M^* se les llama vectores de soporte, y al utilizarlos en la expresión de maximizar la margen, se llega al siguiente problema:

$$\begin{aligned}
\max_{\mathbf{w}, \|\mathbf{w}\| = 1} \quad & \min_k l_k D(\mathbf{x}_k)
\end{aligned} \tag{3.28}$$

Por otro lado, la norma unitaria del vector de pesos $\|\mathbf{w}\|$ puede cambiarse por una imposición de la siguiente relación:

$$\begin{aligned}
M \|\mathbf{w}\| &= 1 \\
\rightarrow M^* &= \frac{1}{\|\mathbf{w}^*\|}
\end{aligned} \tag{3.29}$$

De esto, se minimiza a $\|\mathbf{w}\|$, lo cual es equivalente a maximizar M :

$$\begin{aligned}
\min f(\mathbf{x}) &= \frac{1}{2} \|\mathbf{w}\|^2 \\
\text{Con la condición:} & \\
l_k D(\mathbf{x}_k) &\geq 1 \\
\rightarrow g_k(\mathbf{x}) &= l_k D(\mathbf{x}_k) - 1 \\
g_k(\mathbf{x}) &\geq 0
\end{aligned} \tag{3.30}$$

Este problema tiene una forma de minimización con restricciones de desigualdad, y es viable utilizar Multiplicadores de Lagrange:

$$\begin{aligned}
\min L(\mathbf{x}, \mathbf{a}) &= f(\mathbf{x}) - \sum_{i=1}^p a_i g_i(\mathbf{x}) \\
\text{sujeto a:} \\
a_i &\geq 0 \\
\text{cumpliendo con:} \\
a_i g_i(\mathbf{x}) &= 0 \\
i &= 1, \dots, p
\end{aligned} \tag{3.31}$$

Conllevando a una función de costo equivalente que absorbe las condiciones de desigualdad:

$$\begin{aligned}
\min L(\mathbf{w}, b, \boldsymbol{\alpha}) &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{k=1}^p \alpha_k (l_k D(\mathbf{x}_k) - 1) \\
\text{sujeto a:} \\
\alpha_k &\geq 0 \\
\text{cumpliendo con:} \\
\alpha_k (l_k D(\mathbf{x}_k) - 1) &= 0 \\
k &= 1, \dots, p
\end{aligned} \tag{3.32}$$

Donde α_k son los Multiplicadores de Lagrange. Como hay requerimientos de obtener una minimización, se debe lograr un gradiente en cero para la función de Lagrange:

$$\begin{aligned}
\nabla L(\mathbf{w}, b, \boldsymbol{\alpha}) &= 0 \\
\Rightarrow \frac{\partial L}{\partial \mathbf{w}} &= \mathbf{w}^* - \sum_{k=1}^p \alpha_k l_k \boldsymbol{\phi}_{\mathbf{k}} = 0 \\
\rightarrow \mathbf{w}^* &= \sum_{k=1}^p \alpha_k l_k \boldsymbol{\phi}_{\mathbf{k}} \\
\frac{\partial L}{\partial b} &= \sum_{k=1}^p \alpha_k l_k = 0 \\
\rightarrow \sum_{k=1}^p \alpha_k l_k &= 0
\end{aligned} \tag{3.33}$$

Si se reemplazan las condiciones de gradiente cero (condiciones de primer orden de optimalidad) en la función de Lagrange, se puede reformular el problema de optimización sin una dependencia directa de $\mathbf{w}$ ni de b :

$$\begin{aligned}
\min L(\boldsymbol{\alpha}) &= \sum_{k=1}^p \alpha_k - \frac{1}{2} \boldsymbol{\alpha} \mathbf{H} \boldsymbol{\alpha} \\
\text{sujeto a: } \alpha_k &\geq 0
\end{aligned} \tag{3.34}$$

En esta expresión, $\mathbf{H}$ es una matriz de dimensiones pxp , y cada una de sus componentes está asociada a la función kernel:

$$H_{kj} = l_k l_j K\langle x_k, x_j \rangle \tag{3.35}$$

De este problema resulta una búsqueda de los valores óptimos de $\boldsymbol{\alpha}^*$, que pueden hallarse mediante técnicas de penalización o de programación cuadrática (QP). Los vectores de entrenamiento donde $\boldsymbol{\alpha}^*$ es diferente de cero, son los vectores de soporte, y usualmente la cantidad de ellos es menor que la de vectores de entrenamiento. El último parámetro es el *bias* óptimo b^* , que corresponde a:

$$\begin{aligned}
b^* &= \frac{1}{2} \sum_{k=1}^p l_k \alpha_k^* (K(\mathbf{x}_A, \mathbf{x}_k) + K(\mathbf{x}_B, \mathbf{x}_k)) \\
\mathbf{x}_A &\rightarrow \text{vector de soporte en clase A} \\
\mathbf{x}_B &\rightarrow \text{vector de soporte en clase B}
\end{aligned} \tag{3.36}$$

Finalmente, el plano clasificador queda determinado como:

$$D(\mathbf{x}) = \sum_{k=1}^p l_k \alpha_k^* K(\mathbf{x}_k, \mathbf{x}) + b^* \quad (3.37)$$

$$\text{sgn}(D(\mathbf{x})) \rightarrow \text{clasificación con el plano} \quad (3.38)$$

Para expandir el concepto de SVM a múltiples clases, existen varios enfoques [Xu et al., 2017] y crear múltiples superficies de decisión. Uno de ellos consiste en maximizar la margen por cada par de clases (ver figura 3.7)

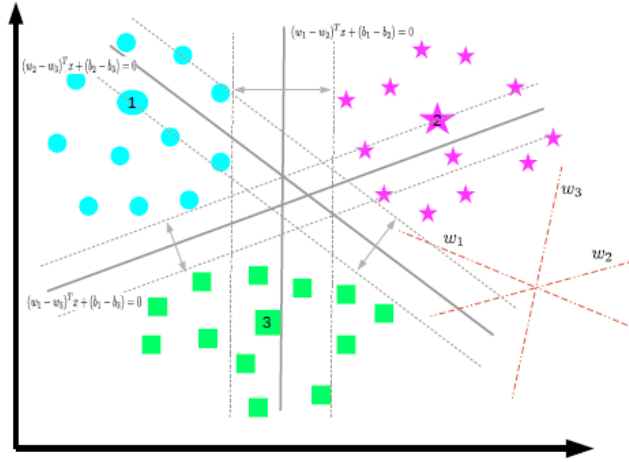


Figura 3.7: Superficies de decisión en una SVM multiclase, mostrando el margen de separación permitido entre tres clases [Xu et al., 2017].

Por lo general, la etapa de entrenamiento no se realiza en un sistema embebido por ser computacionalmente demandante, sino que se utiliza un entrenamiento fuera de línea en un PC [Benatti et al., 2017].

3.1.5 Sistema de reconocimiento de señales mioeléctricas

Los sistemas modernos de reconocimiento de señales mioeléctricas, basados en reconocimiento de patrones, estructura su funcionamiento en cuatro etapas básicas. El primer bloque consta de la segmentación de los datos en diferentes ventanas de tiempo, para agrupar características en bloques de tiempo. Posteriormente se extraen los vectores de características por cada uno de los segmentos, que pueden ser la

concatenación de características TD, FD o TFD.

En el tercer paso, las características extraídas del conjunto de datos pasan por una etapa de clasificación, donde se puede realizar la proyección de características y posteriormente pasan a un clasificador (SVM, LDA, redes neuronales, etc).

El paso final consiste en utilizar el vector de salida del clasificador como elemento de referencia en un controlador digital que puede utilizar sus salidas para manejar actuadores en una prótesis de brazo.

Para ilustrar el sistema, se puede observar la figura 3.8:



Figura 3.8: Esquema ilustrativo de las etapas de un sistema de control reconocimiento de señales mioeléctricas (basado en [Hakonen et al., 2015])

De estas cuatro etapas, se tomarán como puntos centrales para el estudio la extracción y la clasificación de características, donde se han formulado estrategias computacionales como el Sistema Numérico de los Residuos (RNS) que permiten acelerar las operaciones de multiplicación-acumulación (MAC) involucradas [Nakahara and Sasao, 2015]. Existen múltiples ejemplos en el procesamiento de señales, donde se muestra que el desempeño mejora al utilizar el RNS: Filtros de Respuesta al Impulso (FIR) implementados en un Circuito Integrado de Aplicación Específica (ASIC) con mejor desempeño en velocidad, área y consumo respecto a FIR convencionales; mejoras en velocidad y potencia para implementaciones de filtros FIR en Arreglos de Compuertas Programables en el Campo (FPGA) [Albicocco et al., 2014]; mejoras en velocidad para bancos de Transformadas Wavelet [Ramírez et al., 2003].

Otra estrategia para acelerar la obtención de características en el Dominio de la Frecuencia, son las Transformadas Numéricas Teóricas (NTT). Las NTT ofrecen propiedades similares a la Transformada de Fourier, ya que cumplen con los Teoremas de Convolución y Parseval, logrando incrementos de velocidad en aplicaciones de procesamiento de señales [Moraga, 2012]. Entre las NTT existen variantes como la Transformada Numérica de Fermat, la cual permite simplificar el hardware existente en módulos mariposa implicados en operaciones de convolución, vitales en sistemas

como las Redes Neuronales Convolucionales [Xu et al., 2018].

Debe resaltarse que ambas estrategias (RNS y NTT) y sus variantes tienen sus fundamentos de funcionamiento en la Aritmética Modular [Jenkins et al., 2018] [Sousa et al., 2016], basada en anillos de números enteros.

3.1.6 Aritmética Modular [Abdelfattah, 2011][Apostol, 2013]

La aritmética modular permite representar un número convencional (por ejemplo, en complemento a 2) en su equivalente en un anillo módulo m , tomando en cuenta ciertas propiedades básicas.

- **Congruencia:** La relación básica entre números convencionales y su representación modular, es la siguiente congruencia:

$$a \equiv b \pmod{m} \quad (3.39)$$

Se dice que a es congruente a b módulo m , donde m es el módulo, y b es el residuo. El residuo es definido como el menor número positivo remanente de la división de a entre b .

Esto indica que m divide la diferencia entre a y b :

$$m | (a - b) \quad (3.40)$$

- **Equivalencia:** La congruencia es una relación de equivalencia:

$$\begin{aligned} a &\equiv a \pmod{m} \text{ (reflexividad)} \\ a &\equiv b \pmod{m} \rightarrow b \equiv a \pmod{m} \text{ (simetría)} \\ a &\equiv b \pmod{m}, b \equiv c \pmod{m} \rightarrow a \equiv c \pmod{m} \text{ (transitividad)} \end{aligned} \quad (3.41)$$

- **Valores Base:** Cualquier número X puede ser representado respecto a un módulo m_i , por una combinación lineal de un valor base B_i y un residuo r_i :

$$\begin{aligned} X &= B_i + r_i \\ B_i &= k.m_i \end{aligned} \tag{3.42}$$

- **Adición y Substracción:** Si $a \equiv b(\text{mod } m)$ y $\alpha \equiv \beta(\text{mod } m)$ entonces se cumple:

$$ax + \alpha y \equiv bx + \beta y, \text{ para todo entero } x \text{ y } y \tag{3.43}$$

- **Multiplicación y Exponenciación:** Si $a \equiv b(\text{mod } m)$ y $\alpha \equiv \beta(\text{mod } m)$ entonces se cumple:

$$\begin{aligned} a\alpha &\equiv b\beta(\text{mod } m) \\ a^n &\equiv b^n(\text{mod } m) \text{ para todo entero positivo } n \end{aligned} \tag{3.44}$$

- **Inverso Aditivo:** La relación entre el residuo b , ($a \equiv b(\text{mod } m)$) y su inverso aditivo $\bar{b}$, se define por la congruencia:

$$(b + \bar{b})(\text{mod } m) \equiv 0(\text{mod } m) \tag{3.45}$$

El inverso aditivo $\bar{b}$ puede obtenerse utilizando la siguiente operación:

$$\bar{b}(\text{mod } m) \equiv (m - b)(\text{mod } m) \tag{3.46}$$

- **Inverso Multiplicativo:** El inverso multiplicativo b^{-1} del residuo b ($a \equiv b(\text{mod } m)$) está definido por la congruencia:

$$(b.b^{-1})(\text{mod } m) \equiv 1(\text{mod } m) \tag{3.47}$$

Donde b^{-1} existe solo si b y m son coprimos.

3.1.6.1 Sistema Numérico de los Residuos (RNS)[Ramírez et al., 2000]:

De la aritmética modular han surgido formas de representación numérica como el Sistema Numérico de los Residuos (RNS), el cual se ha estudiado desde épocas antiguas en China (100 A.C), donde personas como Sun Tzu elaboró preguntas como [Jenkins et al., 2018]:

Tenemos cosas de las que no sabemos el número.

Si los contamos de a tres, el resto es 2.

Si los contamos por cinco, el resto es 3.

Si los contamos por siete, el resto es 2.

Cuántas cosas hay

La respuesta, 23.

La solución a este tipo de cuestionamientos implica el uso de un sistema RNS, manipulando un conjunto de **residuos** respecto a un conjunto de **módulos**, a través del Teorema Chino del Resto (CRT).

Entrando en mayores detalles, el RNS está definido por un conjunto de L números enteros positivos y coprimos por parejas $\{m_1, m_2, \dots, m_L\}$ llamados módulos. El producto de los módulos, $M = \prod_{i=1}^L m_i$ representa el rango dinámico del RNS. En consecuencia, cualquier número entero $X \in \mathbb{Z}(M)$, el anillo de los enteros módulo M , es representado de manera única por la L -tupla $[x_1, x_2, \dots, x_L]$ de sus residuos, donde $x_i = X \bmod m_i$.

Dados dos números $X, Y \in \mathbb{Z}(M)$ y sus correspondientes representaciones en residuo, $[x_1, x_2, \dots, x_L]$ y $[y_1, y_2, \dots, y_L]$, la aritmética en RNS está definida por:

$$X \circ Y \leftrightarrow [|x_1 \circ y_1|_{m_1}, |x_2 \circ y_2|_{m_2}, \dots, |x_L \circ y_L|_{m_L}] \quad (3.48)$$

Donde $\circ$ representa las operaciones de adición, substracción o multiplicación. De esta forma, la aritmética RNS está definida sobre la suma directa de los anillos módulo m_i , $\mathbb{Z}(m_i)$ y se pueden realizar operaciones en paralelo sin dependencias entre los residuos. La conversión de una representación en residuos $[x_1, x_2, \dots, x_L]$ a su valor entero original se basa en el Teorema Chino del Resto (CRT). El CRT establece que existe un

único $X \in \mathbb{Z}(M)$ para una representación residual dada y que puede calcularse como:

$$\begin{aligned} |X|_M &= \left| \sum_{j=1}^L \widehat{m}_j \left| \frac{x_j}{\widehat{m}_j} \right|_{m_j} \right|_M \\ \widehat{m}_j &\equiv \frac{M}{m_j} \end{aligned} \tag{3.49}$$

De aquí se observa que el RNS provee una metodología fundamental para el particionamiento de rangos dinámicos amplios, en un número de canales con rangos más pequeños sobre los cuales los cálculos computacionales pueden ser realizados en forma paralela.

El esquema básico de un procesador con RNS contiene un convertidor de binario a RNS o convertidor directo, luego realiza las operaciones aritméticas necesarias para cada uno de los canales independientes, y finalmente utiliza un convertidor de RNS a binario o convertidor inverso para retomar los valores de salida en la representación binaria convencional [Navi et al., 2011]. Esto se puede observar en la figura 3.9.

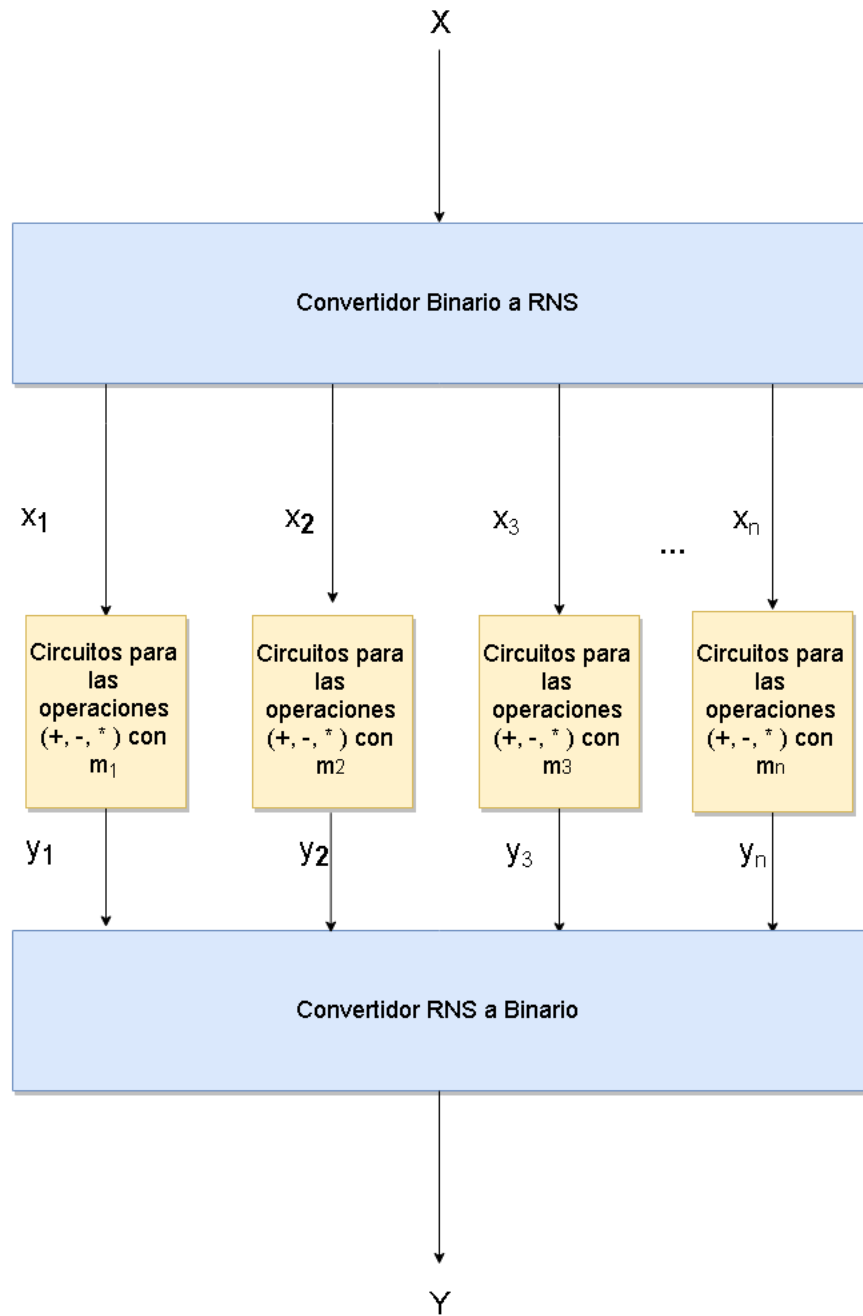


Figura 3.9: Esquema ilustrativo de un sistema utilizando RNS (basado en [Navi et al., 2011])

Por esta forma de paralelización de operaciones, con aislamiento de operaciones entre módulos m_i , se afirma [Jenkins et al., 2018] que el RNS tiene tolerancia a fallos

de cálculo inherente a su estructura algebraica, siendo atractivo para diseñar aplicaciones en hardware para procesamiento digital de señales en operaciones de multiplicación-acumulación, principalmente en extracción de características y procesamiento de imágenes [Jenkins et al., 2018] [Sousa et al., 2016] [Chang et al., 2015] e incluso en filtros adaptativos [Radhakrishnan and Jenkins, 2009].

3.1.6.2 Transformadas Numéricas Teóricas (NTT)

Las NTT surgieron en la década de los 60's como una alternativa a las Transformadas de Fourier para el procesamiento de señales. La razón principal de esto, es que las NTT trabajan sobre números enteros en aritmética modular, en lugar de trabajar con el conjunto de los números complejos. Por otro lado, los números de Fermat y de Mersenne están relacionados con potencias de 2, lo cual puede ser útil para la reducción en la complejidad de cómputo de productos, por ejemplo en operaciones de convolución, donde se reducen a operaciones de desplazamiento a nivel de bit. De estas condiciones proviene el interés en el uso de este tipo de transformadas para realizar cálculos intensivos de manera eficiente y con bajos recursos energéticos y computacionales, como los sistemas embebidos [Moraga, 2012]. Un caso específico de NTT es la Transformada Numérica de Fermat (FNT) [Xu et al., 2018], la cual puede calcularse para una secuencia de longitud N como:

$$X(k) = F(x(n)) = \sum_{n=0}^{N-1} |x(n)\alpha^{nk}|_{F_t} \quad (3.50)$$

Donde $k = 0, 1, \dots, N-1$. F_t representa el t -ésimo número de Fermat, $|a|_b$ representa la operación $a \bmod b$ y F es la Transformada de Fermat unidimensional. Al utilizar $\alpha = 2$ permite facilitar cálculos en hardware a través del uso de registros de desplazamiento. Para hallar la Transformada Inversa Numérica de Fermat F^{-1} (IFNT), se utiliza la siguiente ecuación:

$$x(n) = F^{-1}(X(k)) = \frac{1}{N} \sum_{k=0}^{N-1} |X(k)\alpha^{-nk}|_{F_t} \quad (3.51)$$

Los números de Fermat pueden obtenerse de la relación:

$$\begin{cases} b &= 2^t \\ F_t &= 2^b + 1 \end{cases} \quad (3.52)$$

b es el número de bits de datos y pesos que no causan desbordamiento. Bajo la situación $\alpha = 2$, la máxima longitud de la transformación será $N = 2^{t+1}$.

Dentro de las Transformadas Numéricas de Fermat, existe una variante denominada Transformada Numérica de Fermat Cuadrática Modificada (QMFNT), la cual en el dominio de los Números Complejos Cuadráticos. En el contexto de la tolerancia al error, debido a la eliminación de productos cruzados, permite reducir la complejidad computacional e incrementar la tolerancia a fallos [Radhakrishnan and Jenkins, 2010b]. Por otro lado, al utilizar la Transformada Numérica de Fermat Modificada se aceleran las operaciones de convolución y aplicarse en filtros adaptativos [Radhakrishnan and Jenkins, 2010a].

4. Desarrollo o solución

Para la solución de todo el proyecto planteado, se realizaron tres partes fundamentales que serán descritas en detalle en el presente capítulo:

- Parte 1: Realización algorítmica en software para obtener parámetros de funcionamiento del sistema.
- Parte 2: Realización de un equivalente en hardware programable para utilizar los parámetros obtenidos en software.
- Parte 3: Aplicación de los modelos al análisis de señales mioeléctricas para miembros superiores.

4.1 Parte 1: Requerimientos algorítmicos

Inicialmente, se planteó una forma de obtener parámetros del funcionamiento del para el análisis de señales que pudieran exportarse posteriormente al sistema embebido. Para lograr esto, se optó por el uso de Máquinas de Vectores de Soporte (SVM) como herramienta para clasificar características dado que se ha reportado un buen desempeño de las SVM para el problema propuesto.

Por la facilidad de uso, la curva de aprendizaje, la cantidad de librerías gratuitas disponibles para procesamiento de señales y reconocimiento de patrones, se escogió Python como lenguaje de programación base.

Debe mencionarse que para el desarrollo de esta sección, fue necesario concebir una nueva forma de realizar las SVM para integrar en su funcionamiento el uso de Sistemas Numéricos no Convencionales.

4.1.1 Máquinas de Vectores de Soporte

Debido a que en las plataformas embebidas de bajo consumo existen limitaciones en recursos computacionales, la implementación de algoritmos sofisticados puede dificultarse por requerimientos de operaciones de alta complejidad. Por ello se planteó el uso del RNS para simplificar operaciones de punto flotante a números enteros en conjunto con el uso de SVM como algoritmo de clasificación.

Se tuvo en cuenta que el RNS es rápido para operaciones como la adición, substracción y multiplicación, pero tiene algunos inconvenientes en operaciones no lineales como la comparación y la división. Esto definió un enfoque: primero, encontrar un método para representar características de entrada en enteros módulo M ; segundo, mapear funciones no lineales, como el Kernel de la SVM, a enteros módulo M ; y tercero, usar el modelo obtenido para diseñar un equivalente RNS.

Como punto de partida, se tomó la naturaleza de las SVM que consiste en un hiperplano de decisión basado en encontrar un margen óptimo: maximizar el mínimo margen entre dos clases. Este problema puede definirse en términos de optimización usando la formulación dual:

$$\begin{aligned} \max L(\lambda) &= \max \sum_{i=1}^N \lambda_i - \frac{1}{2} \cdot \sum_{i,j} \lambda_i \lambda_j l_i l_j K \langle \vec{x}_i \vec{x}_j \rangle \\ \text{s.t: } 0 &\leq \lambda_i \leq C; \sum_{i=1}^P \lambda_i l_i \end{aligned} \quad (4.1)$$

Donde cada λ corresponde a los Multiplicadores de Lagrange, K es la función Kernel, C es un límite máximo impuesto para λ proveniente de la flexibilización de la SVM para aceptar puntos con una clasificación incorrecta (soft-margin SVM), $\vec{x}$ corresponde al vector de características de entrada del conjunto de entrenamiento y l corresponde a la etiqueta de clase. Utilizando métodos de optimización numérica, como la versión simplificada del algoritmo de Optimización Mínima Secuencial (SMO)[Tymchyshyn and Khlevniuk, 2020], se obtuvieron los Multiplicadores de Lagrange y los Vectores de Soporte (S.V) que formaron un hiperplano:

$$g(\vec{x}_j) = \sum_{i \in \text{S.V}}^T \lambda_i l_i K \langle \vec{x}_i, \vec{x}_j \rangle + b \quad (4.2)$$

La función Kernel se utilizó para mapear categorías no separables en categorías separables, como por ejemplo el Kernel Gaussiano, el cual toma características de

dimensiones finitas y realiza un producto interno que proviene de un espacio vectorial de infinitas dimensiones. Este producto está basado en la distancia Euclidiana respecto a los Vectores de Soporte:

$$\begin{aligned} K\langle \vec{x}_i, \vec{x}_j \rangle &= e^{-\gamma \|\vec{x}_i - \vec{x}_j\|^2} \\ \Rightarrow 0 < K\langle \vec{x}_i, \vec{x}_j \rangle &\leq 1 \end{aligned} \quad (4.3)$$

Luego de la etapa de entrenamiento, se obtuvo que los Multiplicadores de Lagrange yacían en el intervalo:

$$0 \leq \lambda_i \leq \lambda_{\text{MAX}} \quad (4.4)$$

Las etiquetas de clase se definieron como 1 para la clase 1 y -1 para la clase 2, por lo cual las etiquetas pertenecen al conjunto $\{-1, 1\}$ es decir:

$$l_i \in \{-1, 1\} \quad (4.5)$$

Para propósitos de clasificación, el hiperplano $g(\vec{x}_j)$ se usó como argumento de entrada a una función signo:

$$g(\vec{x}_j) = \begin{cases} \geq 0 & \Rightarrow \vec{x}_j \in \text{Clase 1} \\ < 0 & \Rightarrow \vec{x}_j \in \text{Clase 2} \end{cases} \quad (4.6)$$

Para integrar en la SVM el RNS, se escogió un conjunto de módulos específicos para la representación de variables. Entrando en detalles, teniendo un número entero X y un número entero Y, el sistema se definió de la siguiente manera:

$$\begin{aligned} X &\mapsto \widehat{X} = \langle x_1, x_2, x_3 \rangle \\ Y &\mapsto \widehat{Y} = \langle y_1, y_2, y_3 \rangle \\ \widehat{X} \circ \widehat{Y} &= \langle x_1 \circ y_1, x_2 \circ y_2, x_3 \circ y_3 \rangle \end{aligned} \quad (4.7)$$

Donde los símbolos $\circ$ hacen referencia a cualquier operador del conjunto $\{+, -, *\}$ y el conjunto de módulos es:

$$\begin{aligned} \text{mod set} &= \{m_3, m_2, m_1\} = \{2^\beta - 1, 2^\alpha, 2^\beta + 1\} \\ \alpha &< \beta ; \alpha, \beta \in \mathbb{N} \end{aligned} \quad (4.8)$$

De lo anterior, el rango dinámico del sistema puede expresarse como:

$$\begin{aligned} M &= \prod_{i=1}^3 m_i = m_1 \cdot m_2 \cdot m_3 = 2^{2 \cdot \beta + \alpha} - 2^\alpha \\ M &\text{ es par} \end{aligned} \quad (4.9)$$

Cada entero X representado en este campo finito, debía satisfacer la siguiente condición:

$$\frac{-M}{2} \leq X \leq \frac{M}{2} - 1 \quad (4.10)$$

Lo anterior representa la restricción principal que fue analizada en el proyecto, ya que impactó todos los aspectos de la SVM como se discutirá en las próximas secciones.

4.1.2 Restricciones de implementación de la SVM sobre enteros

Retomando análisis previos, pudo inferirse que el hiperplano de la SVM cumplía con:

$$\begin{aligned} G_1 \cdot g(\vec{x}_j) &= \begin{cases} \geq 0 & \Rightarrow \vec{x}_j \in \text{Clase 1} \\ < 0 & \Rightarrow \vec{x}_j \in \text{Clase 2} \end{cases} \\ G_1 &> 0 \\ G_1 &\text{ es una constante} \end{aligned} \quad (4.11)$$

Donde G_1 se utilizó como una ganancia que permitió el mapeo de la función $g(\vec{x}_j)$ a una escala determinada. Para una implementación en números enteros, las restricciones son muy específicas para cada parte del hiperplano. Relacionado con lo anterior, debe tenerse en cuenta que los campos finitos solo permiten representar un

subconjunto P de un anillo de enteros:

$$\begin{aligned} P &= \left\{ \frac{-M}{2}, \dots, \frac{M}{2} - 1 \right\} \\ P &\subseteq \mathbb{Z} \end{aligned} \tag{4.12}$$

Esto implicó que para cada parte de la SVM (Multiplicadores de Lagrange, vectores de características, función Kernel, etiquetas de clase y la función de comparación) se tuvieron que obtener equivalentes en subconjuntos de P .

4.1.2.1 Propuesta de función para el mapeo del Kernel sobre enteros en módulo M

Las funciones Kernel en las SVM típicamente corresponden a funciones que permiten mapear características a espacios de mayor dimensionalidad. En el caso concreto de las Funciones de Base Radial (RBF), primero se mide la distancia entre dos vectores de dimensiones finitas y posteriormente se conforma un producto interno entre dos vectores de dimensiones infinitas. Esto puede llegar a ser útil en la separabilidad de las clases, y usando el Truco del Kernel (*Kernel Trick*) en las SVM de Kernel RBF se puede calcular el hiperplano de decisión evitando el producto interno al utilizar:

$$K\langle \vec{x}_i, \vec{x}_j \rangle = e^{-\gamma \|\vec{x}_i - \vec{x}_j\|^2} \tag{4.13}$$

De lo anterior, el rango de la función Kernel está en el intervalo $(0, 1]$, y tiene una dependencia de la distancia entre vectores:

$$\begin{aligned} K\langle \vec{x}_i, \vec{x}_j \rangle &= z(d) = e^{-\gamma d^2} \\ d^2 &= \|\vec{x}_i - \vec{x}_j\|^2 \end{aligned} \tag{4.14}$$

De acuerdo con la forma de la función kernel, el comportamiento puede verse representado en la figura 4.1.

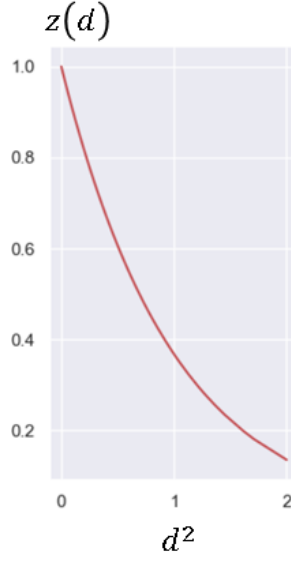


Figura 4.1: Función RBF del Kernel, obtenida a partir de los *scripts* descritos en Python.

Se puede apreciar que la función kernel $z(d)$ tiene su dominio y rango en el conjunto de los números reales, pero pudieron mapearse en números enteros encontrando sus respectivos límites.

El primer paso consistió en encontrar los límites del dominio, que en esencia es la distancia Euclidiana entre dos vectores de k dimensiones $\vec{x}_i$ y $\vec{x}_j$:

$$\begin{aligned}\vec{x}_i &= [x_{i0}, x_{i1}, x_{i2}, \dots, x_{i(k-1)}] \\ \vec{x}_j &= [x_{j0}, x_{j1}, x_{j2}, \dots, x_{j(k-1)}] \\ d^2 &= \sum_{m=0}^{k-1} (x_{im} - x_{jm})^2\end{aligned}\tag{4.15}$$

Al tener vectores de entrada normalizados, sus componentes estarían contenidas en el intervalo $[0, 1]$. Bajo este supuesto, la máxima distancia entre cada par de componentes fue unitaria, por lo cual se pudo inferir la máxima distancia entre vectores:

$$d_{\text{MAX}}^2 = \sum_{m=0}^{k-1} 1^2 = k\tag{4.16}$$

Con ello, se pudo observar el rango de la distancia entre vectores normalizados:

$$\begin{aligned} 0 \leq d^2 \leq k \\ 0 \leq \left(\frac{d^2}{k}\right) \leq 1 \end{aligned} \quad (4.17)$$

Tomando la anterior desigualdad y definiendo una función de mapeo $\widehat{d^2}$, se logró colocar la distancia en un intervalo válido en módulo M:

$$\begin{aligned} \text{Data}_{\max} &= \frac{M}{2} - 1 \\ \widehat{d^2} &= \frac{\text{Data}_{\max}}{k} \cdot d^2 \\ \Rightarrow 0 \leq \widehat{d^2} \leq \text{Data}_{\max} \end{aligned} \quad (4.18)$$

La función de mapeo fue útil para el tránsito hacia una representación en números enteros, y se pudo definir en términos del producto interno de los vectores $\vec{x}_i$ y $\vec{x}_j$:

$$\widehat{d^2} = \frac{\text{Data}_{\max}}{k} \cdot d^2 = \sqrt{\frac{\text{Data}_{\max}}{k}} \cdot (\vec{x}_i - \vec{x}_j)^T \cdot \sqrt{\frac{\text{Data}_{\max}}{k}} \cdot (\vec{x}_i - \vec{x}_j) \quad (4.19)$$

Posteriormente se pudo realizar una aproximación utilizando la función piso:

$$\sqrt{\frac{\text{Data}_{\max}}{k}} \cdot (\vec{x}_i - \vec{x}_j) \approx \left\lfloor \sqrt{\frac{\text{Data}_{\max}}{k}} \cdot \vec{x}_i \right\rfloor - \left\lfloor \sqrt{\frac{\text{Data}_{\max}}{k}} \cdot \vec{x}_j \right\rfloor \quad (4.20)$$

Esta última aproximación permitió obtener un método para mapear un vector de características normalizado en su equivalente de números enteros:

$$\widehat{\vec{x}} = \left\lfloor \sqrt{\frac{\text{Data}_{\max}}{k}} \cdot \vec{x} \right\rfloor \quad (4.21)$$

Al obtener una representación válida para los vectores de entrada y la función distancia, los siguientes pasos consistieron en convertir el parámetro γ y la función $z(d)$ a equivalentes en números enteros.

Para convertir γ a su equivalente entero, se compensó en el exponente el efecto de $\frac{\text{Data}_{\max}}{k}$ que se obtuvo a partir del mapeo de la distancia, con lo cual resultó

conveniente usar la siguiente función de mapeo:

$$\widehat{\gamma} = \left\lfloor \frac{k}{\text{Data}_{\max}} \gamma \right\rfloor \quad (4.22)$$

Estas aproximaciones conllevaron al siguiente modelo para la función exponencial:

$$z(d) \approx e^{-\widehat{\gamma} d^2} \quad (4.23)$$

Finalmente, para ajustar el Kernel RBF a un rango válido en enteros módulo M, se propuso utilizar el siguiente equivalente:

$$\begin{aligned} \widehat{z}(\widehat{d}) &= \left\lfloor \left(\frac{\text{Data}_{\max}}{k \cdot G_e} \right) \cdot e^{-\widehat{\gamma} \widehat{d}^2} \right\rfloor \\ G_e &\geq 1 \\ \widehat{z}(\widehat{d}) &\approx \left(\frac{\text{Data}_{\max}}{k \cdot G_e} \right) \cdot z(d) \end{aligned} \quad (4.24)$$

En este caso, $\widehat{z}(\widehat{d})$ corresponde a una función con un rango contenido en el intervalo $[0, (\frac{\text{Data}_{\max}}{k \cdot G_e})]$, el cual es un subconjunto válido de P. G_e es una ganancia constante positiva que permitió ajustar la amplitud de la función exponencial acorde al mapeo posterior del hiperplano de la SVM.

Para ilustrar lo anterior, asumiendo $M = 510$, $G_e = 5$, la función exponencial equivalente en enteros tendría los siguientes parámetros:

$$\begin{aligned} \text{Amp}_e &= \left(\frac{\text{Data}_{\max}}{k \cdot G_e} \right) = \frac{254}{2.5} = 25,4 \\ \lfloor \text{Amp}_e \rfloor &= 25 \\ \widehat{d_{\text{MAX}}^2} &= \lfloor \text{Data}_{\max} \rfloor = \lfloor \frac{510}{2} - 1 \rfloor = \lfloor 254 \rfloor \\ \widehat{d_{\text{MAX}}^2} &= 254 \end{aligned} \quad (4.25)$$

Esto se puede ver representado en la figura 4.2.

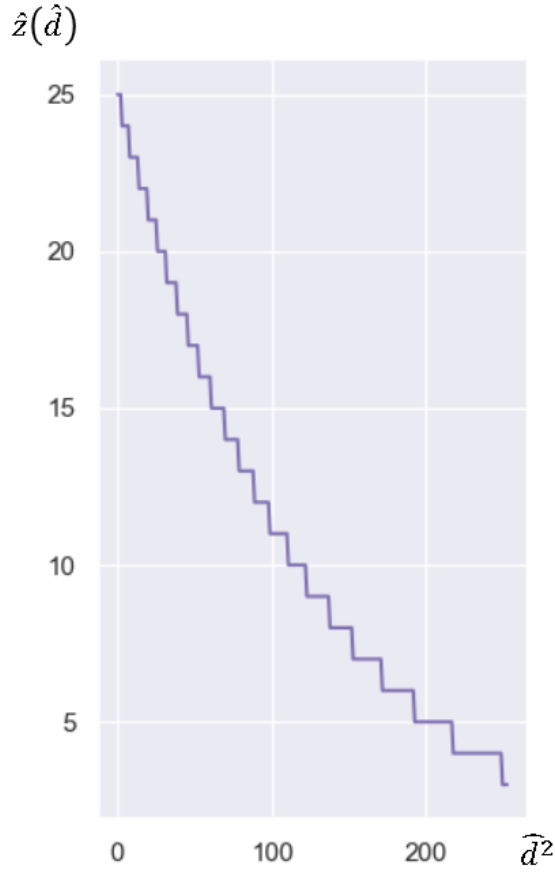


Figura 4.2: Equivalente en enteros de la función RBF del Kernel, obtenida a partir de los *scripts* descritos en Python.

La función Kernel debió ser almacenada como una *Look-Up-Table* (LUT), porque en otro tipo de aproximaciones, con las Series de Taylor para campos finitos, se obtuvieron dificultades en su realización. El problema principal usando el último método mencionado yace en encontrar un inverso multiplicativo para la función factorial, inherente de las Series de Taylor. No se puede garantizar la existencia de un inverso multiplicativo modular si el módulo no es un número primo, y consecuentemente no coprimo con todos los coeficientes de Taylor. En este caso, M es un número compuesto, y no se pudo garantizar la existencia de todos los coeficientes de la aproximación.

Finalmente, un diagrama de bloques que representa la aproximación realizada se muestra en la figura 4.3.

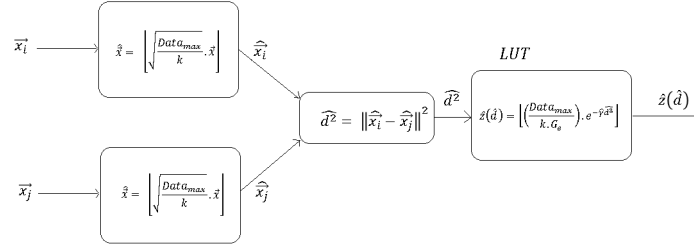


Figura 4.3: Equivalente en enteros de la función RBF del Kernel, obtenida a partir de los *scripts* descritos en Python.

4.1.2.2 Propuesta de función para el mapeo de una SVM sobre enteros en módulo M

De acuerdo a las secciones previas, el hiperplano de la SVM se mapeó al realizar el equivalente en módulo M de cada una de sus partes. Teniendo la función Kernel representada en un campo finito módulo M , fue posible hallar las demás conversiones de parámetros utilizando pasos similares. Estas conversiones se resumen en la tabla 4.1.

Parámetro	Rango	Función de conversión	Rango de la función de conversión
Vector de características: $\vec{x}$	$0 \leq \vec{x} \leq 1$	$\hat{\vec{x}} = \left\lfloor \sqrt{\frac{\text{Data}_{\max}}{k}} \vec{x} \right\rfloor$	$0 \leq \hat{\vec{x}} \leq \left\lfloor \sqrt{\frac{\text{Data}_{\max}}{k}} \right\rfloor$
Multiplicadores de Lagrange: λ_i	$0 \leq \lambda_i \leq \lambda_{\max}$	$\hat{\lambda}_i = \left\lfloor G_{\text{lambd}} \frac{\lambda_i}{\lambda_{\max}} \right\rfloor$	$0 \leq \hat{\lambda}_i \leq \lfloor G_{\text{lambd}} \rfloor$
Gamma: γ	$\gamma > 0$	$\hat{\gamma} = \left\lfloor \frac{k}{\text{Data}_{\max}} \gamma \right\rfloor$	$\hat{\gamma} > 0$

Sigue en la próxima página.

Parámetro	Rango	Función de conversión	Rango de la función de conversión
Etiquetas de clase: l_i	$l_i \in \{-1, 1\}$	$\hat{l}_i = l_i$	$\hat{l}_i \in \{-1, 1\}$
Bias: b	$b > 0$	$\hat{b} = G_{\text{bias}} \cdot b $	$\hat{b} > 0$

Cuadro 4.1: Resumen de las funciones de conversión de parámetros

Usando estas funciones de conversión, el límite aproximado del hiperplano $g(\vec{x})$, g_{lim} pudo ser obtenido. Este parámetro corresponde al máximo valor de g restringido a un área de interés, en este caso fue el espacio vectorial que contiene todos los posibles vectores de características. Para el análisis se tomaron características normalizadas de dos dimensiones, formando una región cuadrada definida en $[0, 1] \times [0, 1]$, como se indica en la figura 4.4.

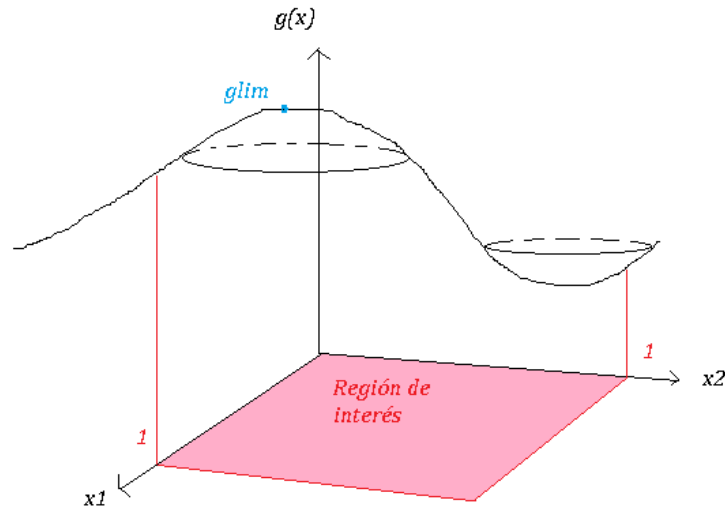


Figura 4.4: Esquema ilustrativo del hiperplano de la SVM con su región de interés y su límite

Para aproximar el valor g_{lim} , se tomó el valor absoluto del hiperplano en q puntos pseudoaleatorios sobre toda el área de interés y se compararon las evaluaciones para

obtener g_{lim} :

$$g_{lim} = \max\{|g_i|\} \quad (4.26)$$

$$0 \leq i \leq q$$

Para apreciar de manera esquemática la búsqueda de g_{lim} se puede observar la figura 4.5.

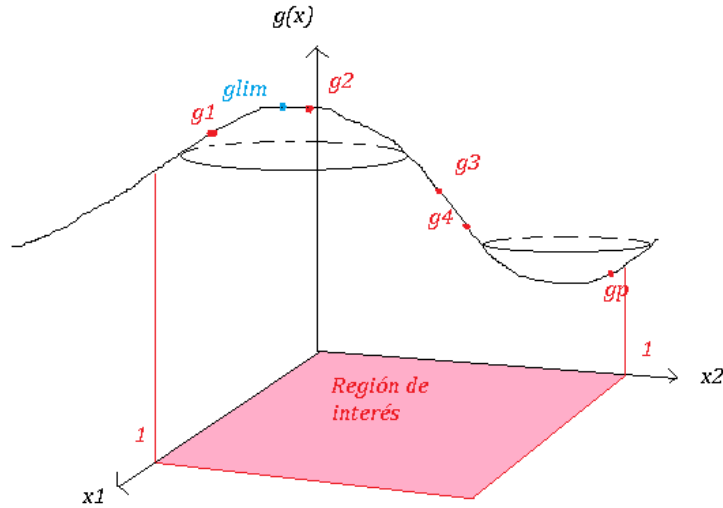


Figura 4.5: Esquema de la búsqueda de g_{lim}

Esta búsqueda de g_{lim} podría ser más exacta si se explora el uso de análisis numéricos para la optimización (SMO, algoritmos genéticos, etc.), pero por simplicidad, el método con los q puntos se utilizó en este trabajo. Esto se justifica en que el método de los q puntos se basa en el algoritmo de optimización aleatoria *Random Search* [Bergstra and Bengio, 2012] que sirve para buscar mínimos en funciones de alta complejidad. En este caso, el hiperplano de la SVM consiste de una superposición de funciones kernel no lineales y con el método basado en *Random Search* se puede conseguir una respuesta rápida, aunque preferiblemente considerando holguras y márgenes de error en las ganancias utilizadas en los parámetros de conversión

Recordando que el hiperplano puede ser multiplicado por una ganancia positiva G_1 sin

cambiar su signo, y utilizando la desigualdad:

$$|g(\vec{x}_j)| \leq g_{lim} \quad (4.27)$$

Se consiguió:

$$\begin{aligned} g(\hat{x}_j) &= G_{plane} \cdot \frac{g(\vec{x}_j)}{g_{lim}} \\ G_1 &= \frac{G_{plane}}{g_{lim}} \\ -G_{plane} &\leq g(\hat{x}_j) \leq G_{plane} \end{aligned} \quad (4.28)$$

En principio, G_{plane} pudo ser utilizado para mapear el hiperplano en el intervalo $\left[\frac{-Data_{max}}{k}, \frac{Data_{max}}{k} \right]$, pero debido a la aproximación en g_{lim} , se usó una holgura de un 20 % en G_{plane} , lo cual es el 80 % del rango total:

$$G_{plane} = 0,8 \cdot \frac{Data_{textmax}}{k} \quad (4.29)$$

Por otro lado, G_{lambda} es un parámetro que permitió ponderar los Multiplicadores de Lagrange en la función del hiperplano. Como los Multiplicadores de Lagrange resultaron pequeños comparados con el rango $\left[\frac{-Data_{max}}{k}, \frac{Data_{max}}{k} \right]$, se pudo definir G_{lambda} como una proporción de G_{plane} :

$$G_{lambda} = 0,001 \cdot G_{plane} \quad (4.30)$$

Finalmente, al utilizar todas las partes de la SVM en conjunto, se pudo obtener una SVM entera en módulo M:

$$g(\hat{x}) = \sum_{i \in S.V}^T \hat{\lambda}_i \cdot \hat{l}_i \cdot \hat{z}_{ij} + \hat{b} \quad (4.31)$$

Para obtener las ganancias faltantes, se utilizaron las ecuaciones deducidas previamente:

$$\begin{aligned}
\frac{G_{\text{plane}}}{g_{\text{lim}}}.g(\vec{x}) &= \sum_{i \in S.V}^T \lambda_i.l_i.K\langle \vec{x}_i \vec{x}_j \rangle .G_{\text{lambda}}.\frac{\text{Data}_{\text{max}}}{\lambda_{\text{MAX}}.k.G_e} + G_b.b \\
\Rightarrow G_b &= \frac{G_{\text{plane}}}{g_{\text{lim}}} \\
G_{\text{lambda}}.\frac{\text{Data}_{\text{max}}}{\lambda_{\text{MAX}}.k.G_e} &= \frac{G_{\text{plane}}}{g_{\text{lim}}}
\end{aligned} \tag{4.32}$$

4.1.2.3 Propuesta de modelo de SVM utilizando RNS

De acuerdo con lo planteado previamente, dada una SVM modelada en un anillo módulo M, fue posible plantear un clasificador SVM utilizando RNS. El primer paso consistió en seleccionar un conjunto adecuado de módulos. Por simplicidad en la implementación hardware, se escogieron módulos que utilizan potencias de dos.

$$\begin{aligned}
\text{modset} &= \{m_3, m_2, m_1\} = \{2^\beta - 1, 2^\alpha, 2^\beta + 1\} \\
M &= m_1.m_2.m_3 = 2^{2\beta+\alpha} - 2^\alpha
\end{aligned} \tag{4.33}$$

Los residuos en RNS para los parámetros de la SVM descritos en módulo M se pueden obtener utilizando las conversiones de la tabla 4.2.

Parámetro	Representación entera	Representación RNS
Multiplicadores de Lagrange en módulo M	$\widehat{\lambda}_i$	$\langle \widehat{\lambda}_i \rangle_{m_k}$
Etiquetas de clase en módulo M	$\widehat{l}_i$	$\langle \widehat{l}_i \rangle_{m_k}$
Bias en módulo M	$\widehat{b}$	$\langle \widehat{b} \rangle_{m_k}$
Exponencial entera en módulo M	$\widehat{z}_{ij}$	$\langle \widehat{z}_{ij} \rangle_{m_k}$

Cuadro 4.2: Conversión de parámetros enteros a RNS

En lo descrito por la tabla 4.2, por cada parámetro p, existe una tupla de tres números donde cada componente se define como:

$$\begin{aligned}\langle p_i \rangle_{m_k} &= p_i \bmod m_k \\ k &\in \{1, 2, 3\}\end{aligned}\tag{4.34}$$

Debido a que en el Kernel exponencial, los dos vectores de entrada $\vec{x}_i$ y $\vec{x}_j$ corresponden a un número positivo en módulo M, es decir, sus valores están contenidos en $[0, \frac{M}{2}]$, implicó que la *Look-Up Table* de z_{ij} tuviera $\lfloor \frac{M}{2} \rfloor \times \lfloor \frac{M}{2} \rfloor$ dimensiones para su representación en módulo M. Por otro lado, la equivalencia final de la función exponencial en el conjunto de tres módulos tuvo una dimensionalidad de $\lfloor \frac{M}{2} \rfloor \times \lfloor \frac{M}{2} \rfloor \times 3$.

Con todas las conversiones previas, fue posible crear una representación del hiperplano SVM en RNS, pero fue necesario definir una función nueva en RNS que permitiera sustituir a la función signo:

$$\text{sign}(g(\widehat{\vec{x}})) \rightarrow \text{sign}_{\text{RNS}}(((g(\widehat{\vec{x}})))_{m_k})\tag{4.35}$$

La función signo es equivalente a comparar un número con cero, y determinar si es menor, mayor o igual a zero. La comparación no es una operación trivial en RNS, pero diseñando una variante del algoritmo LPN [Salamat et al., 2018] en el conjunto de módulos se pudo lograr este objetivo.

Para deducir el diseño de la función prevista, se partió de un número entero representado en módulo M, denominado X, y su equivalente en RNS:

$$\begin{aligned}X &\rightarrow \langle R_1, R_2, R_3 \rangle \\ \langle X \rangle_{2^{\beta-1}} &= X \bmod (2^{\beta} - 1) = R_1 \\ \langle X \rangle_{2^{\alpha}} &= X \bmod (2^{\alpha}) = R_2 \\ \langle X \rangle_{2^{\beta+1}} &= X \bmod (2^{\beta} + 1) = R_3\end{aligned}\tag{4.36}$$

De lo anterior, se pudo observar una periodicidad de los residuos respecto al módulo m_1 y m_3 , definiendo:

$$\begin{aligned}
X' &= X + m_1.m_3 = X + (2^\beta - 1).(2^\beta + 1) \\
\langle X' \rangle_{2^\beta - 1} &= X' \bmod (2^\beta - 1) = R'_1.R_1 \\
\langle X' \rangle_{2^\beta + 1} &= X' \bmod (2^\beta + 1) = R'_3.R_3 \\
\langle X' \rangle_{2^\alpha} &= X' \bmod (2^\alpha) = R'_2
\end{aligned} \tag{4.37}$$

El último residuo pudo ser definido como:

$$\begin{aligned}
\langle X' \rangle_{2^\alpha} &= \langle X + (2^\beta - 1).(2^\beta + 1) \rangle_{2^\alpha} \\
R'_2 &= \langle R_2 - 1 \rangle_{2^\alpha}
\end{aligned} \tag{4.38}$$

A manera de generalización, se tomó un periodo T , tal que $T = m_1.m_3$. Al sumar o restar una cantidad múltiplo del periodo $k.T$ al número X , entonces su residuo R_2 se afecta en k unidades:

$$\begin{aligned}
X' &= X \pm k.T \begin{cases} R'_1 &= R_1 \\ R'_2 &= \langle R_2 \mp k \rangle_{2^\alpha} R'_3 = R_3 \end{cases} \\
T &= m_1.m_3 \\
k &\in \mathbb{N}
\end{aligned} \tag{4.39}$$

La periodicidad en RNS se pudo utilizar para comparar números, definiendo:

- Menor Número posible (*Least Possible Number*- LPN): es el menor número entero con residuos R_1 y R_3 iguales al número que será representado.
- Residuo de Referencia (*Reference Residue* - RR): Es el residuo R_2 del LPN.

Por otro lado, fue posible representar cualquier número en RNS como una combinación lineal del LPN (LPN_X) y el RR (RR_X). Esto puede ser demostrado asumiendo dos

números enteros X y X' y sus residuos:

$$\begin{aligned} X &\rightarrow \langle R_1, R_2, R_3 \rangle \\ X' &\rightarrow \langle R'_1, R'_2, R'_3 \rangle \end{aligned} \quad (4.40)$$

Definiendo X' como una combinación lineal de LPN_X , R_2 y RR_X :

$$X' = m_1.m_3.(RR_X - R_2) + LPN_x = (2^\beta - 1).(2^\beta + 1).(RR_X - R_2) + LPN_X \quad (4.41)$$

Usando aritmética modular fue posible concluir que X es equivalente a X' en representación RNS:

$$\begin{aligned} \langle R'_1 \rangle &= \langle X' \rangle_{2^\beta - 1} \\ \langle R'_2 \rangle &= \langle X' \rangle_{2^\alpha} = \langle R_2 - RR_X + LPN_X \rangle_{2^\alpha} = \langle R_2 - RR_X + RR_X \rangle_{2^\alpha} = R_2 \\ \langle R'_3 \rangle &= \langle X' \rangle_{2^\beta + 1} = \langle LPN_X \rangle_{2^\beta + 1} = R_3 \\ &\rightarrow X' = X \end{aligned} \quad (4.42)$$

$$X = k.m_1.m_3 + r = m_1.m_3.(RR_X - R_2) + LPN_X \rightarrow \langle LPN_X \rangle_{m_1.m_3}$$

Como el LPN_X se encuentra en la primera periodicidad de $T = m_1.m_3$ entonces:

$$\langle X \rangle_{m_1.m_3} = LPN_X \quad (4.43)$$

Por lo anterior, se pudo realizar un algoritmo para comparar dos números enteros, X y Y , representados en RNS:

$$\begin{aligned} X &= k_x.m_1.m_3 + r_x \rightarrow X \equiv \langle R_{x1}, R_{x2}, R_{x3} \rangle \\ Y &= k_y.m_1.m_3 + r_y \rightarrow Y \equiv \langle R_{y1}, R_{y2}, R_{y3} \rangle \end{aligned} \quad (4.44)$$

Según esto, se concluyó que:

$$\begin{aligned}
& \text{Si } \langle k_x \rangle_{m_2} > \langle k_y \rangle_{m_2} \rightarrow X > Y \\
& \text{Si } \langle k_x \rangle_{m_2} < \langle k_y \rangle_{m_2} \rightarrow X < Y \\
& \text{Si } \langle k_x \rangle_{m_2} = \langle k_y \rangle_{m_2} \begin{cases} r_x > r_y \rightarrow X > Y \\ r_x < r_y \rightarrow X < Y \\ r_x = r_y \rightarrow X = Y \end{cases}
\end{aligned} \tag{4.45}$$

En términos de LPN y RR:

$$\begin{aligned}
& \text{Si } \langle RR_x - R_{x_2} \rangle_{m_2} > \langle RR_y - R_{y_2} \rangle_{m_2} \rightarrow X > Y \\
& \text{Si } \langle RR_x - R_{x_2} \rangle_{m_2} < \langle RR_y - R_{y_2} \rangle_{m_2} \rightarrow X < Y \\
& \text{Si } \langle RR_x - R_{x_2} \rangle_{m_2} = \langle RR_y - R_{y_2} \rangle_{m_2} \begin{cases} LPN_x > LPN_y \rightarrow X > Y \\ LPN_x < LPN_y \rightarrow X < Y \\ LPN_x = LPN_y \rightarrow X = Y \end{cases}
\end{aligned} \tag{4.46}$$

Las sustracciones de los RR con los correspondientes R_2 fueron obtenidos en módulo m_2 para garantizar valores positivos. De estas últimas expresiones, pensando en términos de la implementación en hardware, si los LPN y los RR son obtenidos de LUT almacenados en memoria, entonces la función de comparación puede ser calculada utilizando solamente valores de la LUT y los residuos de X y Y. Esto evade el problema principal en RNS: utilizar la conversión RNS a binario basado en procedimientos como el Teorema Chino del Resto (CRT). Finalmente, la función de comparación pudo definirse como:

$$comp_{RNS}\{X, Y\} \begin{cases} \text{Si } \langle RR_x - R_{x_2} \rangle_{m_2} > \langle RR_y - R_{y_2} \rangle_{m_2} & \rightarrow X > Y \\ \text{Si } \langle RR_x - R_{x_2} \rangle_{m_2} < \langle RR_y - R_{y_2} \rangle_{m_2} & \rightarrow X < Y \\ \text{Si } \langle RR_x - R_{x_2} \rangle_{m_2} = \langle RR_y - R_{y_2} \rangle_{m_2} & \begin{cases} LPN_x > LPN_y \rightarrow X > Y \\ LPN_x < LPN_y \rightarrow X < Y \\ LPN_x = LPN_y \rightarrow X = Y \end{cases} \end{cases} \tag{4.47}$$

Esta última función pudo ser implementada fácilmente utilizando sumadores y

comparadores, teniendo en cuenta que para números RNS con signo, el rango de los números positivos yace en $[0, \frac{M}{2} - 1]$ y los números negativos en $[-\frac{M}{2}, -1]$, o de forma equivalente en el rango nominal $[\frac{M}{2}, M - 1]$. Usando el rango nominal completo $[0, M - 1]$, la frontera de decisión entre números positivos y negativos es $\frac{M}{2}$. Entonces, la función signo en RNS pudo verse como:

$$\text{sign}\{X\} \equiv \text{comp}_{\text{RNS}}\{X, \frac{M}{2}\} \quad (4.48)$$

Esto permitió concebir todas las partes necesarias para representar en RNS un clasificador SVM de base radial (RBF).

4.2 Parte 2: Requerimientos de la arquitectura en hardware

Con respecto a la implementación hardware, se decidió utilizar una metodología codiseño, donde se utilizó un SoC para evaluar el funcionamiento de los modelos concebidos. Respecto al uso del SoC, se escogió la plataforma Zynqberry basada en el chip *Zynq 7010 (xc7c010clg255 - 1)*, dado que se tenía disponible para su uso en el laboratorio. Además el uso de un SoC permitió utilizar las metodologías codiseño para realizar tareas algorítmicas en el bloque PS y pruebas de arquitecturas hardware en el bloque PL en una sola plataforma.

En la plataforma seleccionada, en su sección de Lógica Programable (PL basado en FPGA) se utilizó el lenguaje VHDL para describir los bloques hardware para el clasificador, mientras que en el Subsistema de Procesamiento (PS basado en ARM) se decidió realizar la extracción, selección, reducción y normalización de características.

La decisión de realizar el bloque de clasificación SVM en la sección PL reside en su naturaleza de ser en principio un hiperplano con ciertas modificaciones en su funcionamiento. Según lo anterior, una SVM está comprendida por operaciones de Multiplicación y Acumulación (MAC) las cuales son viables en su implementación para el Sistema Numérico de los Residuos (RNS). Dada la independencia de las operaciones entre módulos, la sección del proyecto dedicada al RNS se realizó en Lógica Programable para aprovechar la propiedad de concurrencia de las FPGA.

La extracción, selección, reducción y normalización de características se realizó dentro del PS para aprovechar la flexibilidad del lenguaje C y el desarrollo de algoritmos

secuenciales propios de estas etapas.

4.2.1 Revisión de la arquitectura

Basándose en los métodos revisados en la primera parte de la solución, a nivel de implementación hardware se requirieron los siguientes bloques:

- Unidades aritméticas para la adición, sustracción y multiplicación en RNS.
- Unidad para la función de comparación en RNS.
- Unidad de almacenamiento para los vectores de soporte, Multiplicadores de Lagrange, y etiquetas de clase en representación RNS.
- Unidad para la obtención de la función Kernel, utilizando LUT.
- Una Máquina de Estado Finito (FSM) para controlar las operaciones requeridas y los pasos en el cálculo del hiperplano en RNS y la operación de comparación.

El esquema general puede verse representado en forma de diagrama de bloques en la figura 4.6. Para realizar la clasificación de características utilizando la SVM-RNS, el bloque *controller_FSM* debió manejar el datapath de forma que la arquitectura en conjunto pudiera realizar la siguiente secuencia de pasos:

1. Tomar un vector que debe ser clasificado en dos categorías, y se utilizan como entrada a un bloque que internamente utiliza un Kernel no lineal, basado en LUT. Este Kernel necesita calcular la distancia Euclidiana entre dos vectores RNS, donde la distancia es medida entre los vectores de soporte almacenados en memoria y el vector de entrada.
2. El resultado del Kernel se utiliza para multiplicar con los respectivos Multiplicadores de Lagrange y etiquetas de clase.
3. El resultado de la iteración actual es acumulada con resultados previos utilizando un sumador en RNS y un registro de realimentación.

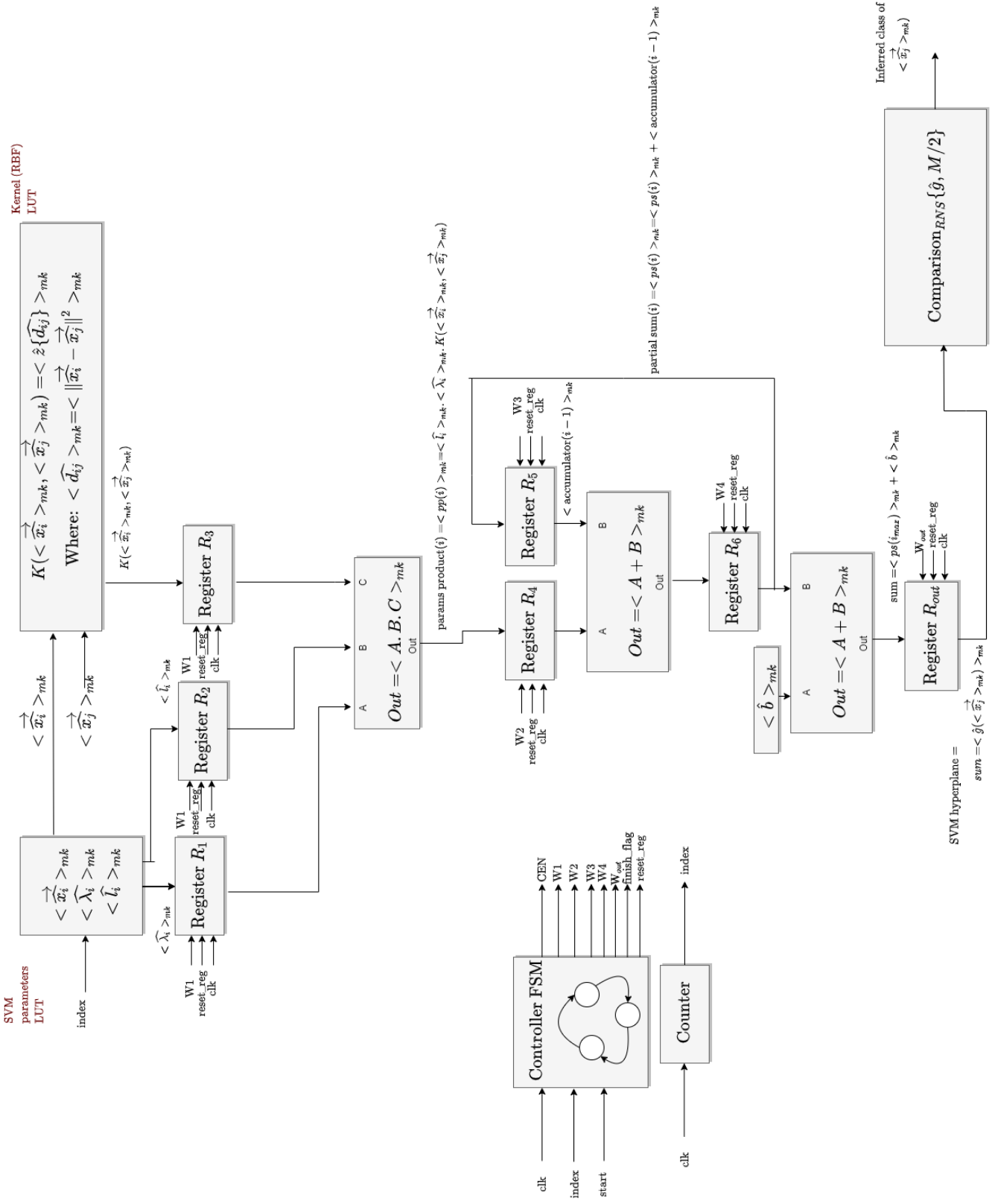


Figura 4.6: Diagrama de bloques de la arquitectura propuesta para realizar el clasificador SVM-RNS

4. Cuando las iteraciones terminan sobre todos los vectores de soporte, se suma el *bias*.
5. Finalmente, utilizando un bloque especial para la comparación en RNS, la clase del vector de entrada es inferida.

4.2.1.1 Bloque del Kernel

Para el cálculo del Kernel, se utilizó una aproximación con una LUT debido al comportamiento no lineal y a su complejidad de implementación utilizando aproximaciones numéricas. Las direcciones de indexación para la memoria LUT provinieron de la concatenación de las tres distancias Euclidianas para la representación en tres módulos. La distancia fue utilizada como una medida de cercanía entre un vector de entrada y un vector de soporte, ambos representados en RNS. El resultado del Kernel también se expresó en términos de los módulos RNS. Un esquema representativo de este bloque se aprecia en la figura 4.7.

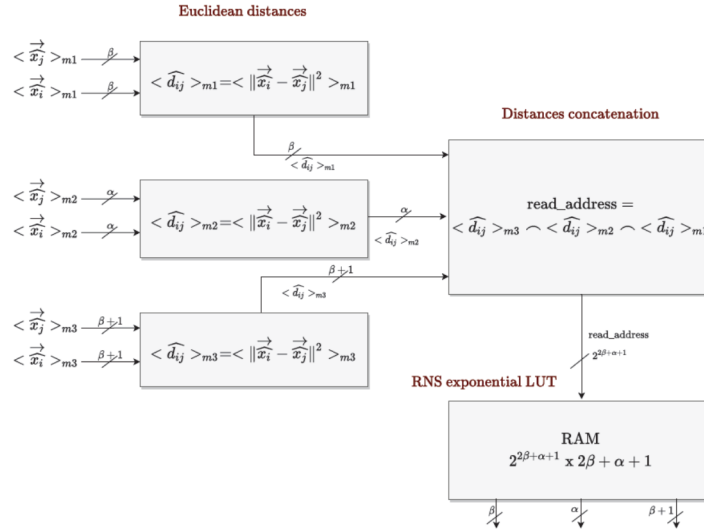


Figura 4.7: Diagrama de bloques de la función Kernel

4.2.1.2 Detección de signo

La detección de signo es una función lineal que se calcula mediante un método basado en los LPN, como fue presentado en la sección 4.1.2.3 del capítulo 4. De manera general, el diagrama de bloques que representa este sistema se muestra en la figura 4.8.

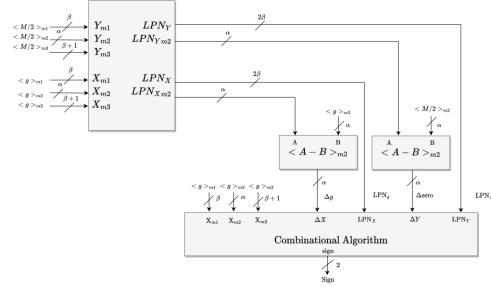


Figura 4.8: Diagrama de bloques de la función signo

Usando el método LPN descrito previamente, es posible obtener el signo de $\langle X \rangle_{m_i}$ a través de un algoritmo implementado de manera combinacional definido de la siguiente manera:

- Si $X_{m_1} = 0$, $X_{m_2} = 0$ y $X_{m_3} = 0$, entonces $\text{signo} = 00_2 = 0_{10}$
- Si $\Delta X > \Delta Y$, entonces $\text{signo} = 11_2 = -1_{10}$
- Si $\Delta X < \Delta Y$, entonces $\text{signo} = 01_2 = 1_{10}$
- Si $\Delta X = \Delta Y$:

$$\begin{cases} \text{Si } LPN_x > LPN_y \rightarrow \text{signo} = 01_2 = 1_{10} \\ \text{Si } LPN_x < LPN_y \rightarrow \text{signo} = 11_2 = -1_{10} \\ \text{Si } LPN_x = LPN_y \rightarrow \text{signo} = 11_2 = -1_{10} \end{cases} \quad (4.49)$$

Un diagrama de flujo que permite representar el algoritmo combinacional se muestra en el esquema 4.9.

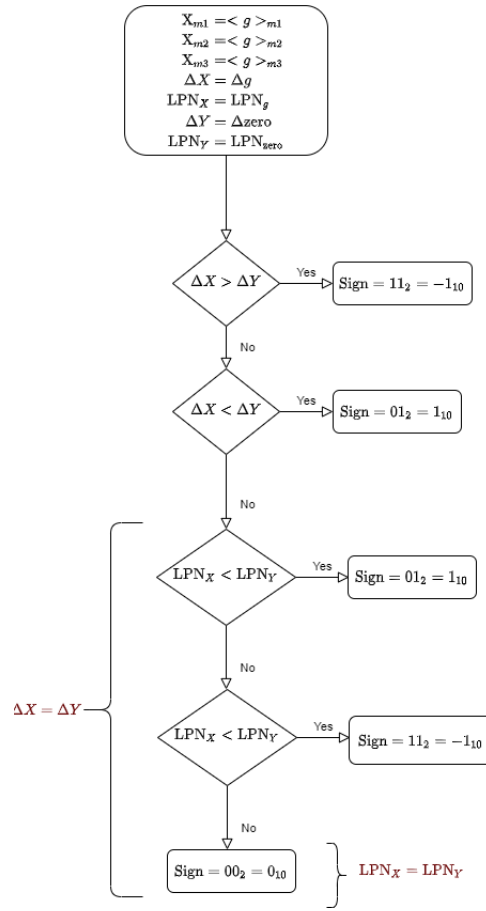


Figura 4.9: Diagrama de flujo de la función signo

Graficamente, la función signo tomó el valor f del hiperplano SVM y determinó si su valor correspondía a la primera mitad del rango dinámico (números positivos) o a la segunda mitad (números negativos), lo cual se aprecia en la gráfica 4.10.

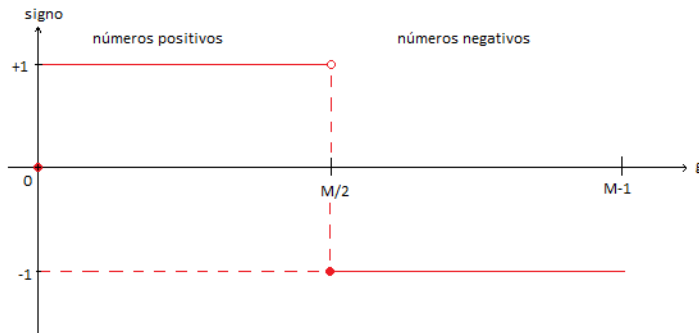


Figura 4.10: Gráfico de la función signo

4.2.1.3 Controlador principal

El controlador principal fue modelado como una Máquina de Estados Finita tipo Moore que permitió ejecutar en una secuencia ordenada todas las operaciones involucradas en el sistema: búsqueda de parámetros, operaciones de multiplicación-acumulación, adición del *bias* y la detección de signo.

En conjunto con la FSM existe un contador ascendente que permitió detectar si la suma ponderada con todos los vectores de soporte fue terminada. La manera en que se conectó el contador y la FSM se puede observar en la figura 4.11.



- **W1:** Permite la escritura de los registros conectados a las entradas del multiplicador RNS para obtener la multiplicación $\langle \langle \widehat{\lambda}_i \rangle_{m_k} \cdot \langle \widehat{l}_i \rangle_{m_k} \cdot \langle \widehat{z}_{ij} \rangle_{m_k} \rangle_{m_k}$
- **W2:** Permite la escritura de los registros conectados a la salida del multiplicador RNS,
- **W3:** Permite la escritura de los registros de realimentación conectados al sumador RNS.
- **W4:** Permite la escritura de los registros conectados a la salida del sumador RNS, acumulando progresivamente la suma ponderada.
- **Wout:** Permite la escritura de los registros que almacenaron la suma ponderada, incluyendo el bias.
- **Index:** La salida del contador es utilizada para direccionar la memoria que contiene los parámetros $\langle \vec{x}_i \rangle_{m_k}, \langle \widehat{\lambda}_i \rangle_{m_k}, \langle \widehat{l}_i \rangle_{m_k}$.

- **CEN:** Permite la habilitación del incremento en el contador.
- **reset_registers:** Coloca en cero todos los registros del sistema.
- **finish_flag:** Indica cuando el vector de entrada era clasificado por el sistema.
- **reset:** Reinicia toda la FSM.
- **start:** Indica a la FSM que debía empezar un proceso de clasificación.
- **clk:** Reloj del sistema.

Para la máquina de estados del controlador, el diagrama de estados se definió como el mostrado en la figura 4.12.

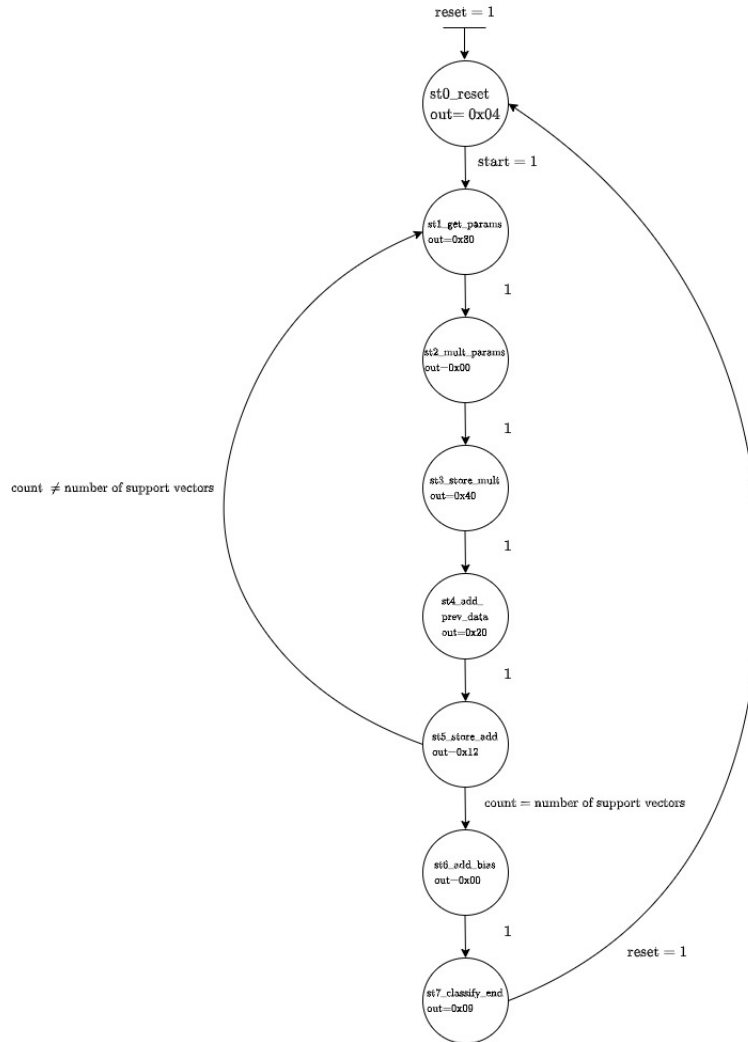


Figura 4.12: Máquina de estados del controlador del sistema

La dependencia que se obtuvo de las salidas respecto a los estados se aprecia en la tabla 4.3.

	out_7 (W1)	out_6 (W2)	out_5 (W3)	out_4 (W4)	out_3 (Wout)	out_2 (reset_reg)	out_1 (CEN)	out_0 (finish_flag)	$out[7..0]$
st0	0	0	0	0	0	1	0	0	0x04
st1	1	0	0	0	0	0	0	0	0x80
st2	0	0	0	0	0	0	0	0	0x00

Sigue en la próxima página.

	out_7 (W1)	out_6 (W2)	out_5 (W3)	out_4 (W4)	out_3 (Wout)	out_2 (reset_reg)	out_1 (CEN)	out_0 (finish_flag)	$out[7..0]$
st3	0	1	0	0	0	0	0	0	0x40
st4	0	0	1	0	0	0	0	0	0x20
st5	0	0	0	1	0	0	1	0	0x80
st6	0	0	0	0	0	0	0	0	0x00
st7	0	0	0	0	1	0	0	1	0x09

Cuadro 4.3: Salidas de la máquina de estados del controlador

4.2.1.4 Registros

Dada la necesidad de trabajar con datos representados en tres módulos, se utilizaron tres registros independientes para almacenar las componentes RNS, como se muestra en la figura 4.13.

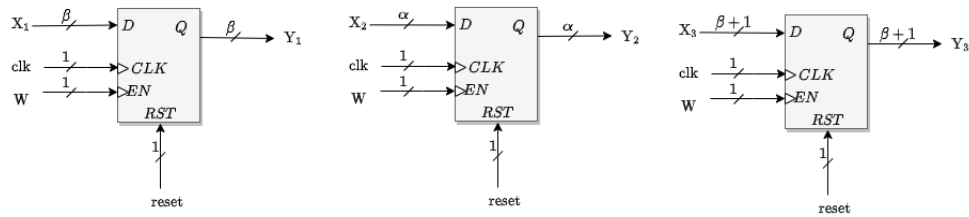


Figura 4.13: Registros para el almacenamiento de datos en RNS

4.2.1.5 Sumadores RNS

Para realizar la suma o resta en RNS fue requerido el uso de bloques específicos en hardware para lograr las operaciones en los términos de la aritmética modular. Inicialmente, para cada módulo m , y los residuos de los operadores de entrada A y B , y restringiendo la longitud de los operadores a n bits y el módulo m siendo menor que

2^n , se establecieron las siguientes condiciones:

$$\langle A + B \rangle_m = \begin{cases} A + B - m, & \text{Si } A + B \geq m \\ A + B, & \text{En otro caso.} \end{cases} \quad (4.50)$$

Esta operación puede ser vista desde otra perspectiva para su implementación en lógica similar al complemento a dos (TCS):

$$\begin{aligned} \langle A + B \rangle_m &= \begin{cases} A + B - m + 2^n - 2^n, & \text{Si } A + B + 2^n - m \geq 2^n \\ A + B, & \text{En otro caso} \end{cases} \\ \Rightarrow \langle A + B \rangle_m &= \begin{cases} \langle A + B - m + 2^n \rangle_{2^n}, & \text{Si } A + B + 2^n - m \geq 2^n \\ A + B, & \text{En otro caso} \end{cases} \\ \Rightarrow \langle A + B \rangle_m &= \begin{cases} \langle A + B + \neg(m) + 1 \rangle_{2^n}, & \text{Si } A + B + 2^n + \neg(m) + 1 \geq 2^n \\ A + B, & \text{En otro caso} \end{cases} \end{aligned} \quad (4.51)$$

Para incluir a la operación resta, se cambió al residuo B por su inverso multiplicativo modular:

$$\langle -B \rangle_m = \langle m - B \rangle_m \quad (4.52)$$

Debido a que $m < 2^n$, la última ecuación pudo simplificarse ya que los posibles residuos respecto al módulo m estaban contenidos en el conjunto de residuos respecto a 2^n :

$$\begin{aligned} \langle -B \rangle_m &= \langle m - B \rangle_m = \langle m - B \rangle_{2^n} \\ \langle -B \rangle_m &= \langle -2^n + m + 2^n - B \rangle_{2^n} \\ \langle -B \rangle_m &= \langle \langle -2^n \rangle_{2^n} + \langle m \rangle_{2^n} + \langle 2^n - B \rangle_{2^n} \rangle_{2^n} \\ \langle -B \rangle_m &= \langle \langle 2^n - 2^n \rangle_{2^n} + m + \neg(B) + 1 \rangle_{2^n} \\ \langle -B \rangle_m &= \langle m + \neg(B) + 1 \rangle_{2^n} \end{aligned} \quad (4.53)$$

Finalmente, el conjunto de las expresiones anteriores permitió implementar un sumador/restador en módulo m como se aprecia en la figura 4.14.

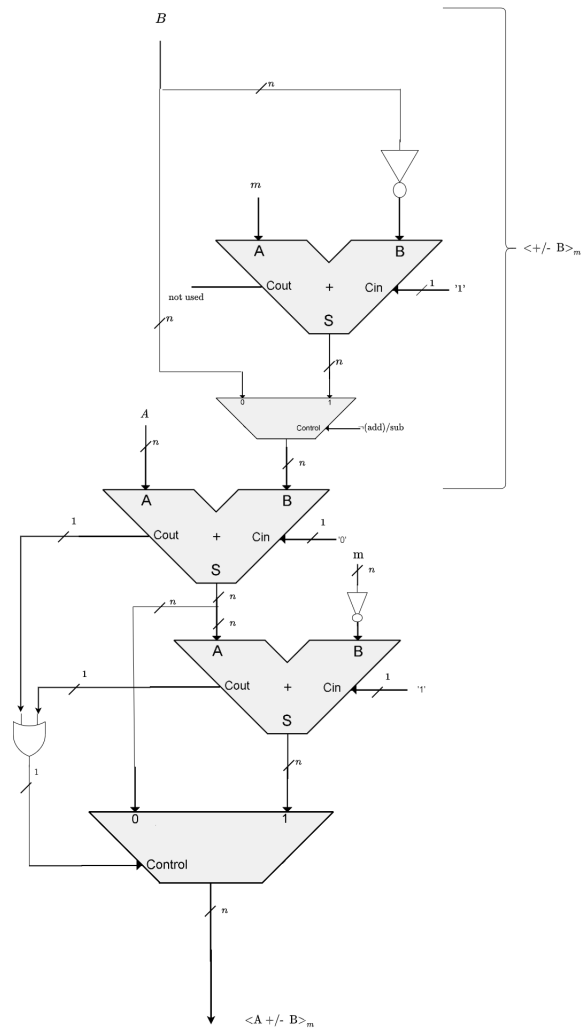


Figura 4.14: Esquema de sumador/restador en módulo m

De una forma más compacta, el sumador/restador resultante tuvo como parámetros de entrada a A , B , m , una entrada de control para escoger la operación, y una salida con el resultado, como se observa en la figura 4.15.

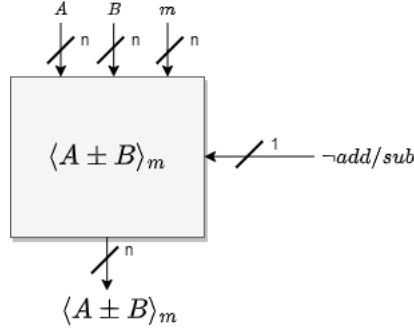


Figura 4.15: Esquema compacto de sumador/restador en módulo m

Finalmente, se utilizaron tres bloques de suma/resta en módulo m para conseguir las tres operaciones requeridas por el conjunto de módulos RNS, como se aprecia en la figura 4.16.

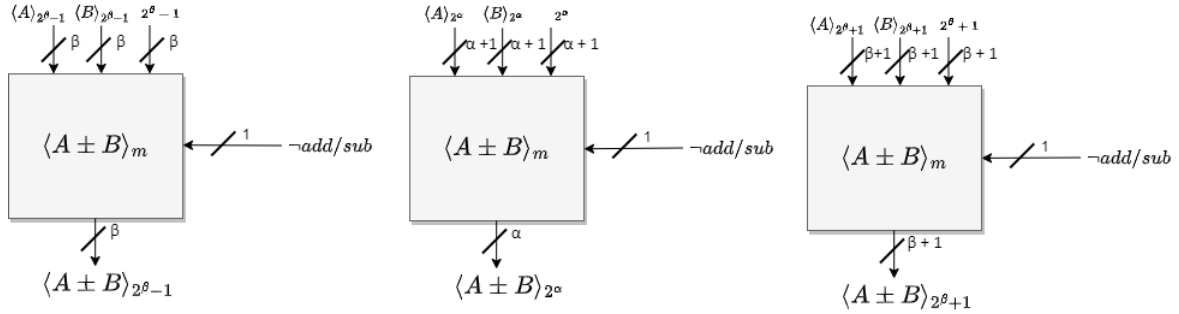


Figura 4.16: Esquema de sumador/restador para un conjunto de módulos

4.2.1.6 Multiplicador RNS

Para realizar la operación de multiplicación en cada uno de los tres módulos, se partió de un multiplicador combinacional de números sin signo y en su salida se le añadieron bloques de reducción al módulo. Debido a que los operadores de entrada eran los residuos obtenidos en el módulo de conversión binario a RNS, los bloques hardware se simplificaron un poco. De manera general, el multiplicador modular mencionado tiene la estructura que se muestra en la figura 4.18.

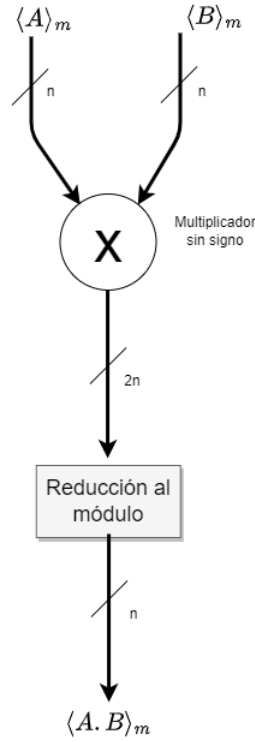


Figura 4.17: Esquema de multiplicador en módulo m

La arquitectura interna del bloque de reducción al módulo cambió dependiendo del módulo implicado, debido a las diferencias de las operaciones implicadas.

4.2.1.7 Reducción al módulo $2^\beta - 1$

En este caso, la obtención de un residuo partió de suponer un número entero A de n bits y su correspondiente representación ponderada:

$$\begin{aligned}
 A &= \sum_{i=0}^{n-1} a_i \cdot 2^i \\
 \Rightarrow \langle A \rangle_{2^\beta-1} &= \langle a_{n-1} \cdot 2^{n-1} + a_{n-2} \cdot 2^{n-2} + \dots + a_1 \cdot 2^1 + a_0 \cdot 2^0 \rangle_{2^\beta-1}
 \end{aligned} \tag{4.54}$$

Posteriormente, se tomó el número A y se fragmentó en segmentos T_i de β bits, significando que la cantidad de fragmentos L pudo definirse por la fórmula:

$$L = \frac{n + \beta - 1}{\beta} = \lceil \frac{n}{\beta} \rceil \quad (4.55)$$

De lo anterior, el número A pudo ser reescrito como:

$$\langle A \rangle_{2^{\beta-1}} = \langle \langle T_{L-1} \cdot 2^{(L-1) \cdot \beta} \rangle_{2^{\beta-1}} + \langle T_{L-2} \cdot 2^{(L-2) \cdot \beta} \rangle_{2^{\beta-1}} + \dots + \langle T_1 \cdot 2^{\beta} \rangle_{2^{\beta-1}} + \langle T_0 \cdot 2^0 \rangle_{2^{\beta-1}} \rangle_{2^{\beta-1}} \quad (4.56)$$

De manera compacta:

$$\langle A \rangle_{2^{\beta-1}} = \langle \sum_{i=0}^{L-1} \langle T_i \cdot 2^{i\beta} \rangle_{2^{\beta-1}} \rangle_{2^{\beta-1}} \quad (4.57)$$

Como se puede observar del último resultado, cada término T_i tiene una longitud de β bits, implicando que sus valores posibles serían los del conjunto:

$$T_i \in R = \{0, 1, 2 \dots 2^{\beta} - 1\} \quad (4.58)$$

Por otro lado, recordando que el módulo es $m = 2^{\beta} - 1$, entonces el conjunto posible de residuos sería:

$$\begin{aligned} x &= \{0, 1, 2 \dots 2^{\beta} - 2\} \\ \Rightarrow R &\not\subseteq x \\ \Rightarrow T_i &\rightarrow \text{Requiere reducción al módulo} \end{aligned} \quad (4.59)$$

Para realizar la reducción del número A , se podría empezar por la reducción de las potencias de dos:

$$\langle 2^{i\beta} \rangle_{2^{\beta-1}} = \langle 2^{i\beta-1+1} \rangle_{2^{\beta-1}} = \langle 1 \rangle_{2^{\beta-1}} \quad (4.60)$$

Con lo anterior, la reducción de un número entero A al módulo m_1 pudo verse de la siguiente manera:

$$\begin{aligned} \langle A \rangle_{2^{\beta-1}} &= \langle \langle T_{L-1} \rangle_{2^{\beta-1}} + \langle T_{L-2} \rangle_{2^{\beta-1}} + \dots + \langle T_1 \rangle_{2^{\beta-1}} + \langle T_0 \rangle_{2^{\beta-1}} \rangle_{2^{\beta-1}} \\ \rightarrow \langle A \rangle_{2^{\beta-1}} &= \langle \sum_{i=0}^{L-1} \langle T_i \rangle_{2^{\beta-1}} \rangle_{2^{\beta-1}} \end{aligned} \quad (4.61)$$

4.2.1.8 Reducción al módulo 2^α

Para obtener un residuo en este módulo, se separó el número entero en dos fragmentos donde la parte menos significativa es de α bits de longitud:

$$\begin{aligned}\langle A \rangle_{2^\alpha} &= \langle a_{n-1}.2^{n-1} + a_{n-2}.2^{n-2} + \dots + a_1.2^1 + a_0.2^0 \rangle_{2^\alpha} \\ \langle A \rangle_{2^\alpha} &= \langle (a_{n-1}.2^{n-\alpha-1} + a_{n-2}.2^{n-\alpha-2} + \dots + a_\alpha.2^0).2^\alpha + (a_{\alpha-1}.2^{\alpha-1} + a_{\alpha-2}.2^{\alpha-2} + \dots + a_0.2^0) \rangle_{2^\alpha}\end{aligned}\quad (4.62)$$

Posteriormente, se definieron las siguientes igualdades:

$$\begin{aligned}k &= a_{n-1}.2^{n-\alpha-1} + a_{n-2}.2^{n-\alpha-2} + \dots + a_\alpha.2^0 \\ r &= a_{\alpha-1}.2^{\alpha-1} + a_{\alpha-2}.2^{\alpha-2} + \dots + a_0.2^0 \\ \rightarrow \langle A \rangle_{2^\alpha} &= \langle k.2^\alpha + r \rangle_{2^\alpha} = \langle r \rangle_{2^\alpha}\end{aligned}\quad (4.63)$$

Según lo anterior, se obtuvo que el residuo de A era el conjunto de sus α bits menos significativos:

$$\rightarrow \langle A \rangle_{2^\alpha} = \sum_{i=0}^{\alpha-1} a_i.2^i \quad (4.64)$$

Por otro, dado que r variaba entre 0 y $2^\alpha - 1$ no fue necesario realizar operaciones de resta con el módulo.

4.2.1.9 Reducción al módulo $2^\beta + 1$

En el último caso, la reducción al módulo se realizó fragmentando al vector de entrada A en segmentos de β bits:

$$\begin{aligned}\langle A \rangle_{2^\beta+1} &= \langle a_{n-1}.2^{n-1} + a_{n-2}.2^{n-2} + \dots + a_1.2^1 + a_0.2^0 \rangle_{2^\beta+1} \\ \langle A \rangle_{2^\beta+1} &= \langle \langle T_{L-1}.2^{(L-1).\beta} \rangle_{2^\beta+1} + \langle T_{L-2}.2^{(L-2).\beta} \rangle_{2^\beta+1} + \dots + \langle T_1.2^\beta \rangle_{2^\beta+1} + \langle T_0.2^0 \rangle_{2^\beta+1} \rangle_{2^\beta+1}\end{aligned}\quad (4.65)$$

La cantidad de fragmentos L quedó definida por:

$$L = \frac{n + \beta - 1}{\beta} = \lceil \frac{n}{\beta} \rceil \quad (4.66)$$

En conjunto, A en forma compacta pudo definirse como:

$$\langle A \rangle_{2^{\beta+1}} = \langle \sum_{i=0}^{L-1} \langle T_i \cdot 2^{i\beta} \rangle_{2^{\beta+1}} \rangle_{2^{\beta+1}} \quad (4.67)$$

Cada segmento T_i tomó un valor posible dentro del siguiente rango:

$$T_i \in R = 0, 1, 2, \dots, 2^\beta - 1 \quad (4.68)$$

Para el módulo mencionado, el conjunto posible de residuos estuvo definido por:

$$\begin{aligned} x &= 0, 1, 2, \dots, 2^\beta \\ \rightarrow R &\subseteq x \end{aligned} \quad (4.69)$$

Como R estaba contenido en x , no fue necesario realizar reducciones en T_i :

$$\langle T_i \rangle_{2^{\beta+1}} = T_i \quad (4.70)$$

Para las expresiones en potencias de 2 contenidas dentro de la expresión de A , fue requerido observar las siguientes propiedades:

$$\begin{aligned} \langle 2^{k\beta} \rangle_{2^{\beta+1}} &= \langle \prod_{j=1}^k 2^\beta \rangle_{2^{\beta+1}} \\ \langle 2^{k\beta} \rangle_{2^{\beta+1}} &= \langle \prod_{j=1}^k \langle 2^\beta \rangle_{2^{\beta+1}} \rangle_{2^{\beta+1}} \\ \text{Como: } \langle 2^\beta \rangle_{2^{\beta+1}} &= \langle 2^\beta + 1 - 1 \rangle = \langle -1 \rangle_{2^{\beta+1}} \\ \rightarrow \langle 2^{k\beta} \rangle_{2^{\beta+1}} &= \langle \prod_{j=1}^k \langle -1 \rangle_{2^{\beta+1}} \rangle_{2^{\beta+1}} = \langle (-1)^k \rangle_{2^{\beta+1}} \end{aligned} \quad (4.71)$$

Por lo anterior, la reducción quedó expresada como:

$$\begin{aligned} \langle A \rangle_{2^{\beta+1}} &= \langle \sum_{i=0}^{L-1} T_i (-1)^k \rangle_{2^{\beta+1}} \\ \langle A \rangle_{2^{\beta+1}} &= \langle \langle T_{L-1} \cdot (-1)^{L-1} \rangle_{2^{\beta+1}} + \langle T_{L-2} \cdot (-1)^{L-2} \rangle_{2^{\beta+1}} + \dots - \langle T_1 \rangle_{2^{\beta+1}} + \langle T_0 \rangle_{2^{\beta+1}} \rangle_{2^{\beta+1}} \end{aligned} \quad (4.72)$$

4.2.1.10 Arquitectura interna del multiplicador RNS

Dadas las condiciones anteriores, se pudo plantear en detalle la arquitectura del multiplicador utilizado, como se aprecia en la figura 4.18.

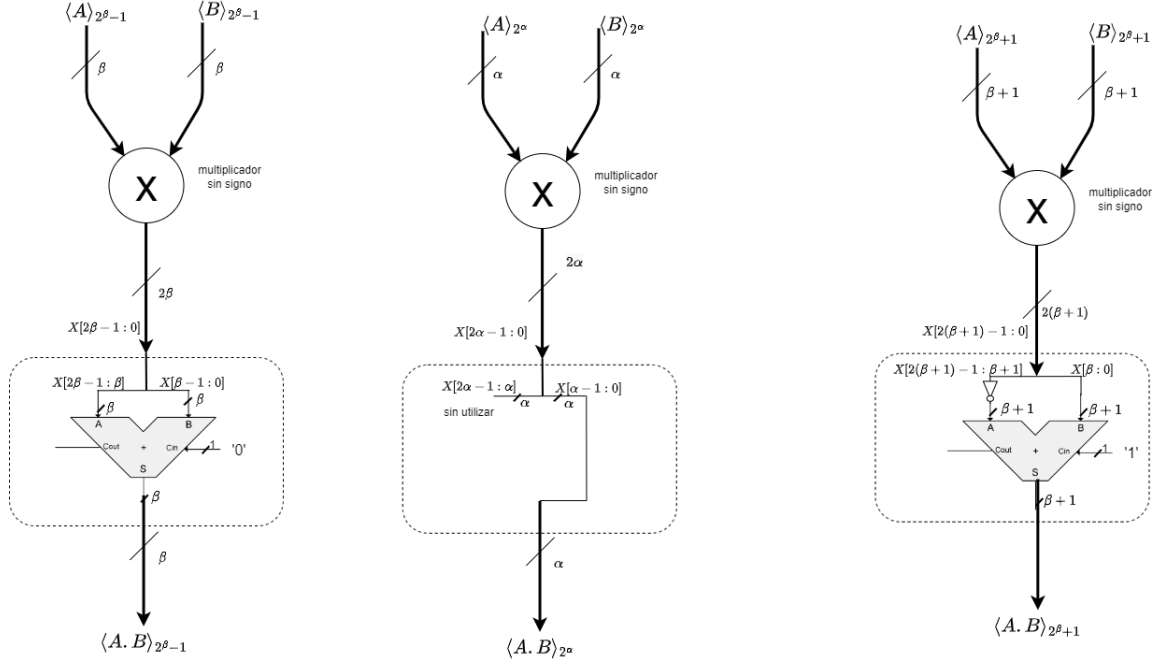


Figura 4.18: Esquema del multiplicador RNS considerando los bloques de reducción al módulo

4.3 Parte 3: Especificaciones para el análisis de señales EMG de miembros superiores

A partir de los modelos obtenidos en software y hardware, se hizo una validación de todas las etapas del sistema según lo requerido para el análisis de las señales EMG de miembros superiores.

Para la extracción de características se utilizó un conjunto recomendado por diferentes fuentes [Caesarendra et al., 2017] [Oskoei and Hu, 2008] [Hudgins et al., 1991] [Atzori and Müller, 2019] [Englehart and Hudgins, 2003] [Negi et al., 2016], posteriormente se realizó una selección de características a través de un método de filtros y finalmente se

utilizó una reducción de características basado en el Análisis de Componentes Principales.

4.3.1 Señales utilizadas

Las señales utilizadas para la validación del modelo propuesto fueron las incluidas en la base de datos NinaPro[Atzori et al., 2014a], cuyas principales propiedades fueron las siguientes:

- La base de datos utilizada fue la número 3, denominada como *DB3* en la literatura. Esto debido a que contaba con varios sujetos con diferentes porcentajes de amputación de brazos, permitiendo inyectar datos ricos en información en el entrenamiento del clasificador.
- La cantidad de sujetos de prueba dentro de *DB3* fueron 11, donde solo se escogieron a los sujetos que utilizaron todos los sensores EMG disponibles y que realizaran los todos los ejercicios analizados. Estos sujetos contaron con los siguientes niveles de amputación, medidos porcentualmente en la cantidad remanente de la extremidad superior[Atzori et al., 2016]:
 - Sujeto 7 (S7): 0 %.
 - Sujeto 3 (S3): 30 %.
 - Sujeto 4 (S4) y sujeto 6 (S6): 40 %.
 - Sujeto 1 (S1), sujeto 8 (S8) y sujeto 10 (S10): 50 %.
 - Sujeto 2 (S2): 70 %.
 - Sujeto 5 (S5), sujeto 9 (S9) y sujeto 11 (S11): 90 %.

De los sujetos nombrados, para el entrenamiento y prueba se utilizaron a S1, S2, S3 y S5, dado que contenían todos los sensores y movimientos disponibles, además de brindar distintos porcentajes en la cantidad remanente de la extremidad superior. Gráficamente, los niveles de amputación se pueden observar en la figura 4.19.

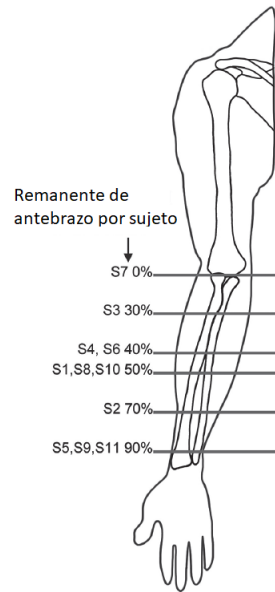


Figura 4.19: Ilustración de los remanentes de brazo por sujeto en *DB3* (basado en [Atzori et al., 2016])

- A nivel de sensores, se contaron con 12 electrodos diferenciales del sistema Delsys Trigno Wireless EMG. Los electrodos se ubicaron de la siguiente forma:
 - 8 electrodos ubicados de manera equidistante alrededor del antebrazo en correspondencia a la unión radio-humeral (codo).
 - 2 electrodos ubicados en los puntos de mayor actividad del *flexor digitorum* y del *extensor digitorum (superficialis)*.
 - 2 electrodos ubicados en los puntos de mayor actividad del biceps y triceps (*biceps brachii* y *triceps brachii*)
- La frecuencia de muestreo en la adquisición de señales fue de 2 kHz.
- Los movimientos realizados en *DB3* se dividieron en tres tipos de ejercicios: movimientos de los dedos, movimientos de muñeca, movimientos de agarre y funcionales. Adicionalmente, se hizo sensado de las señales EMG del brazo en reposo.
- Para la realización de los ejercicios, se realizaba el movimiento durante 5 segundos seguido de un reposo de 3 segundos, repitiendo el patrón 5 veces.

- En total se vieron 50 tipos de movimientos en *DB3*, etiquetados numéricamente de la siguiente forma:
 - Reposo: etiqueta 0.
 - Ejercicio 1 (movimientos de dedos): etiquetas 1 al 17.
 - Ejercicio 2 (movimientos de muñeca): etiquetas 18 al 40.
 - Ejercicio 3 (movimientos de agarre y funcionales): etiquetas 41 al 49.
- Cada repetición de 5 segundos de cada movimiento, se subdividió en ventanas de 200 ms para ser analizadas. Dada la frecuencia de muestreo de 2 kHz, cada ventana tuvo una longitud de 400 muestras.
- El número de movimientos se vio reducido en los sujetos 1,3 y 10 por situaciones experimentales como estrés.

4.3.2 Extracción de características

Para realizar la extracción de características se buscaron aquellas que fueran computacionalmente sencillas de calcular dadas las limitantes de utilizar un sistema embebido, y que mostraran un buen desempeño en la clasificación. Luego de la revisión de antecedentes, se tomaron en cuenta principalmente las recomendaciones de autores que trabajaron con la base de datos Ninapro [Atzori and Müller, 2019] [Atzori et al., 2014a] en adición con trabajos relacionados con búsqueda de conjuntos de características en señales EMG [Phinyomark et al., 2012][Chen et al., 2018][Hakonen et al., 2015][Oskoei and Hu, 2008] para seleccionar el siguiente conjunto:

- Valor Eficaz (*Root Mean Square - RMS*): El valor eficaz de la señal EMG se calculó como:

$$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (4.73)$$

Donde N era la longitud de la señal y x_i era el valor de la señal en el segmento i -ésimo.

- Cruces por Cero (*Zero Crossing - ZC*): La característica *ZC* se utilizó para contar el número de veces que la señal EMG pasó por cero en el eje de amplitud. Para dos muestras consecutivas x_i y x_{i+1} , el conteo de *ZC* incrementó si $x_i > 0$ y $x_{i+1} < 0$, o si $x_i < 0$ y $x_{i+1} > 0$. Por otro lado, debía cumplirse que se superase un umbral $|x_i - x_{i+1}| > threshold$, donde el umbral *threshold* fue de 10^{-5} por recomendaciones de trabajos previos [Atzori and Müller, 2019].
- Longitud de la Forma de Onda(*Waveform Lenght - WL*): La característica *WL* se empleó al utilizar la ecuación:

$$WL = \sum_{i=1}^{N-1} |x_{i+1} - x_i| \quad (4.74)$$

Donde N correspondía a la longitud de la señal, y x_i y x_{i+1} correspondían a dos muestras consecutivas de la señal EMG.

- Cambio de Signo de la Pendiente(*Slope Sign Change - SSC*): La característica *SSC* determinó el número de veces que la señal EMG cambió de signo en su pendiente. Para ello, se tomaron tres muestras consecutivas x_{i-1} , x_i y x_{i+1} y se incrementó un contador si se cumplía: $x_i > x_{i-1}$ y $x_i > x_{i+1}$ o si $x_i < x_{i-1}$ y $x_i < x_{i+1}$. Adicionalmente, debían cumplirse las condiciones de umbral $|x_i - x_{i+1}| \leq threshold$ ó $|x_i - x_{i-1}| \leq threshold$, donde el umbral *threshold* fue de 10^{-5} por recomendaciones de trabajos previos [Atzori and Müller, 2019].
- Valor Promedio Absoluto (*Mean Absolute Value - MAV*): La característica *MAV* se calculó mediante la fórmula:

$$MAV = \frac{1}{N} \cdot \sum_{i=1}^N |x_i| \quad (4.75)$$

Donde N era la longitud de la señal y x_i el valor de la señal.

- Pendiente del Valor Promedio Absoluto(*Mean Absolute Value Slope - MAVS*): La característica *MAVS* representó la diferencia entre las características *MAV* de dos segmentos de señal EMG consecutivos k y $k + 1$:

$$MAVS = MAV_{k+1} - MAV_k \quad (4.76)$$

4.3.3 Selección de características

Dado que por cada segmento de señal EMG se extrajeron 6 características por cada uno de los 12 electrodos (72 características en total), se planteó utilizar un método de selección de características para evitar el uso de información redundante provista por la etapa de extracción. El método utilizado fue el de la Eliminación Recursiva de Características (*Recursive Feature Elimination - RFE*) [Guyon et al., 2002] [Granitto et al., 2006].

De manera general, el algoritmo *RFE* realizó un posicionamiento para una medida de importancia en el conjunto de características. El algoritmo 1 muestra a manera de pseudocódigo la manera en que se seleccionaron las características: Por cada iteración se retiraba la característica que tuviera la menor posición según el método de posicionamiento, que típicamente parte de una validación cruzada o *cross-validation*. En la validación cruzada se busca obtener los subconjuntos de características que tengan el menor error promedio de clasificación, para lo cual se hacen varias iteraciones entrenando y probando diferentes grupos de características ante un modelo de clasificador.

En el sistema planteado, se tomaron dos tipos de movimiento y se planteó la selección de las 6 características más significativas utilizando un modelo de regresión lineal provisto por la librería *sklearn* en Python. Con esto se buscó el conjunto de características que presentaran una mejor relación lineal en las iteraciones del modelo de regresión.

Algorithm 1 Algoritmo *RFE*

input : $T \rightarrow$ conjunto de entrenamiento;
 $F = [f_1, f_2, \dots, f_p] \rightarrow$ conjunto de p características;
 $M(T, F) \rightarrow$ método de posicionamiento.

output: $R \rightarrow$ posicionamiento final

for $i \leftarrow 1$ **to** p **do**
 Posicionar el conjunto F utilizando $M(T, F)$
 $f^* \leftarrow$ característica con la última posición de F
 $R(p - i + 1) \leftarrow f^*$
 $F \leftarrow F - f^*$
end

4.3.4 Reducción de características

Posterior a la selección de características, se buscó un método que permitiera reducir la dimensionalidad de las características. Este paso resultó de vital importancia, ya que por limitantes de hardware no resultaba conveniente el uso de un vector de características mayor a dos dimensiones. Por otro lado, se tuvo que tener en cuenta que la reducción de características debía seguir algún criterio para perder la menor cantidad posible de información y que fuera fácilmente implementable en hardware. Por lo anterior, se decidió escoger el Análisis de Componentes Principales (*Principal Component Analysis* - *PCA*) como método de reducción de características, porque permitió la reducción de características utilizando unas matrices de transformación sencillas que siguen un criterio de optimalidad. Además, como se vio en trabajos previos, el uso de método *PCA* en conjunto con clasificadores *SVM* producen buenos resultados en el ámbito de la clasificación de señales EMG [Fontana, 2010].

Con fines de optimalidad, el método *PCA* cumplió los siguientes criterios [Abdi and Williams, 2010][Zhang et al., 2012]:

- Dado un vector de características $\vec{x} \in \mathbb{R}^d$, se quiere hallar un vector $\vec{y} \in \mathbb{R}^P$ de menor dimensionalidad ($P \ll d$) que logre mantener una alta varianza en sus componentes.
- Por lo anterior, si $P = 2$, deben encontrarse unos ejes $\vec{u}_1$ y $\vec{u}_2$ ortonormales que maximicen la varianza de las proyecciones de $\vec{x}$ (denominado como $\vec{y}$). De manera general, para P dimensiones deben hallarse $\vec{u}_1, \vec{u}_2, \dots, \vec{u}_P$ ejes ortonormales.
- Estos ejes se llaman componentes principales (*PC*).
- De manera general, suponiendo n puntos de datos:

$$\begin{aligned} X &= [\vec{x}_1, \vec{x}_2, \dots, \vec{x}_n]_{d \times n} \\ \vec{x}_i &\in \mathbb{R}^d \end{aligned} \tag{4.77}$$

- La proyección de cada vector $\vec{x}_i$ sobre un eje $\vec{u}$ puede verse como:

$$\vec{u}^T . X$$

Donde:

$$\vec{u}^T \rightarrow 1 \times d \text{ dimensiones} \quad (4.78)$$

$$X \rightarrow d \times n \text{ dimensiones}$$

$$\Rightarrow \vec{u}^T . X \rightarrow 1 \times n \text{ dimensiones}$$

- Estas proyecciones deben cumplir con un criterio de máxima varianza, es decir, hallar las componentes $\vec{u}$ tales que maximicen la función de costo:

$$J = \text{Var}(\vec{u}^T . X) \quad (4.79)$$

Las propiedades de la covarianza permiten expresar la función de costo como:

$$\text{Var}(\vec{u}^T . X) = \vec{u}^T . S . \vec{u}$$

Donde: $S = \text{Var}(X) \rightarrow$ Matriz de $d \times d$ dimensiones

$$S_{ij} = \text{Cov}(\vec{x}_i, \vec{x}_j) = E[(\vec{x}_i - E[\vec{x}_i]) . (\vec{x}_j - E[\vec{x}_j])] \quad (4.80)$$

$$\text{Con: } E[\vec{x}] = \frac{1}{d} . \sum_{k=1}^d x_d \rightarrow \text{Valor promedio de } \vec{x}$$

- Recordando que los ejes deben ser unitarios, el problema de optimización se reescribe como:

$$\begin{aligned} \max \quad & \vec{u}^T . S . \vec{u} \\ \text{s.t: } & \vec{u}^T . \vec{u} = 1 \end{aligned} \quad (4.81)$$

- Este problema puede abordarse utilizando Multiplicadores de Lagrange:

$$\begin{aligned} L(\vec{u}, \lambda) &= \vec{u}^T . S . \vec{u} - \lambda . (\vec{u}^T . \vec{u} - 1) \\ \frac{\partial L}{\partial \vec{u}} &= 2 . S . \vec{u} - 2 . \lambda . \vec{u} = 0 \\ \Rightarrow S . \vec{u} &= \lambda . \vec{u} \end{aligned} \quad (4.82)$$

Por lo cual, λ y $\vec{u}$ resultan ser los valores y vectores propios de la matriz de covarianza S . Lo anterior implica que existen d pares $(\lambda_i, \vec{u}_i)$ que solucionan el

problema de optimización.

- Teniendo de manera ordenada los valores propios $\lambda_1 > \lambda_2 > \lambda_3 > \dots \lambda_d$ y sus correspondientes vectores propios $\vec{u}_1, \vec{u}_2, \vec{u}_3, \dots, \vec{u}_d$, se puede encontrar que la función objetivo tendrá los siguientes valores:

$$\begin{aligned} J_i &= \vec{u}_i^T \cdot S \cdot \vec{u}_i = \vec{u}_i^T \cdot \lambda_i \cdot \vec{u}_i = \vec{u}_i^T \cdot \vec{u}_i \cdot \lambda_i \\ \Rightarrow J_i &= \lambda_i \end{aligned} \quad (4.83)$$

- Esto significa que los valores ordenados $\lambda_1 > \lambda_2 > \lambda_3 > \dots \lambda_d$ entregan en orden de importancia a las componentes $\vec{u}_1, \vec{u}_2, \vec{u}_3, \dots, \vec{u}_d$.
- En el caso del sistema de análisis de señales EMG se tomaron las primeras dos componentes $\vec{u}_1$ y $\vec{u}_2$, que se pueden hallar a través de métodos como la Descomposición por Valores Singulares (*Singular Value Decomposition - SVD*) que entrega de manera ordenada las componentes principales.
- El método *SVD* permite expresar una matriz X de $d \times n$ dimensiones como:

$$X_{d \times n} = U \cdot \Sigma \cdot V^T$$

$U \rightarrow$ matriz de vectores propios de $X \cdot X^T$ de $d \times d$ dimensiones

$V^T \rightarrow$ vectores propios de $X^T \cdot X$ de $n \times n$ dimensiones

$\Sigma \rightarrow$ matriz diagonal con los valores propios de $X \cdot X^T$ ó $X^T \cdot X$, de $d \times n$ dimensiones

(4.84)

En mayor detalle, la matriz Σ provee los valores propios λ_i de manera ordenada:

$$\begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_d \end{bmatrix}_{d \times n} \quad (4.85)$$

Se puede utilizar el método *SVD* para calcular las componentes principales de

PCA, centrando los datos en una matriz $\tilde{X}$ de $d \times n$ dimensiones:

$$\begin{aligned}\tilde{X} &= (X - M)_{d \times n} \\ X &= [\vec{x}_1 \dots \vec{x}_n] \rightarrow \text{matriz de entrada de } d \times n \text{ dimensiones} \\ M &= [\bar{\vec{x}} \dots \bar{\vec{x}}] \rightarrow \text{matriz de promedios de } d \times n \text{ dimensiones} \\ \bar{\vec{x}} &= \frac{1}{n} \sum_{i=1}^n \vec{x}_i \rightarrow \text{vector promedio de las entradas } \vec{x}_i\end{aligned}\tag{4.86}$$

De aquí se puede ver que la matriz de covarianza de X queda implícita en:

$$S = \tilde{X} \cdot \tilde{X}^T \tag{4.87}$$

Por lo cual, utilizando el método *SVD* para $\tilde{X}$:

$$\begin{aligned}\tilde{X} &= U \cdot \Sigma \cdot V^T \\ U &\rightarrow \text{matriz con los vectores que representan los ejes de } PCA \\ \Sigma &\rightarrow \text{matriz diagonal con los valores propios de la matriz de covarianza } S\end{aligned}\tag{4.88}$$

- Finalmente, al utilizar las componentes $\vec{u}_1$ y $\vec{u}_2$ de $1 \times n$ dimensiones, se pudo crear la matriz de transformación PCA_{mat} de $d \times n$ dimensiones, en este caso de 2×6 dimensiones:

$$PCA_{mat} = \begin{bmatrix} \vec{u}_1 \\ \vec{u}_2 \end{bmatrix}_{2 \times 6} \tag{4.89}$$

Con lo anterior, la proyección de un vector $\vec{x}$ de características con $P = 6$ dimensiones se definió por la transformación:

$$\begin{aligned}\vec{x}' &= (\vec{x} - \bar{\vec{x}}) \cdot (PCA_{mat})^T \\ \bar{\vec{x}} &= \frac{1}{n} \sum_{i=1}^n \vec{x}_i \rightarrow \text{matriz con los vectores promedio de } X\end{aligned}\tag{4.90}$$

Los datos de la matriz PCA_{mat} y el promedio $\bar{\vec{x}}$ se obtuvieron a partir de las características seleccionadas de las señales EMG. Con lo cual la reducción de características se simplificó a una operación matricial fácilmente implementable

en sistemas embebidos.

4.3.5 Normalización de características

Luego de obtener un vector $\vec{x}$, el cual proviene de una reducción de características de dos dimensiones del método *PCA*, se utilizó una normalización de sus componentes para limitar sus valores al rango $[0, 1]$ como se requería en la *SVM* de la etapa de clasificación.

Para lograr esto, se utilizó un escalamiento min-max [Pandey and Jain, 2017] dado por la fórmula:

$$x'_i = \frac{x_i - x_{i \min}}{x_{i \max} - x_{i \min}} \quad (4.91)$$
$$i \in \{1, 2\}$$

Donde x_i representa el valor de la componente i-ésima del vector $\vec{x}$, $x_{i \min}$ el mínimo valor alcanzado por la componente i-ésima en la base de datos, $x_{i \max}$ el máximo valor alcanzado por la componente i-ésima en la base de datos, y x'_i representa el valor normalizado de la componente i-ésima.

Con lo anterior, se obtuvo un vector normalizado $\vec{x}_n$ que fue convertido a un rango válido para el clasificador en *RNS*.

4.3.6 Conversión de características al dominio RNS

Finalmente, para cumplir con las condiciones descritas en la sección previa acerca del clasificador *SVM*, se hizo una conversión del vector de características normalizado

$\vec{x}_n$ a su equivalente en enteros módulo M $\widehat{\vec{x}}$ y luego a su equivalente en RNS $\widehat{\vec{x}}_{RNS}$:

$$\begin{aligned}
\widehat{\vec{x}}_i &= \left\lfloor \sqrt{\frac{\text{Data}_{\max}}{k}} \cdot (\vec{x}_n)_i \right\rfloor \rightarrow \text{componentes del vector en módulo } M \\
i &\in \{1, 2\} \\
k &= 2 \rightarrow \text{dimensiones del vector} \\
\text{Data}_{\max} &= \frac{M}{2} - 1 \\
M &= m_1 \cdot m_2 \cdot m_3 \\
\{m_1, m_2, m_3\} &= \{2^\beta - 1, 2^\alpha, 2^\beta + 1\} \\
(\widehat{\vec{x}}_{RNS})_i &= [\langle \widehat{\vec{x}}_i \rangle_{m_1}, \langle \widehat{\vec{x}}_i \rangle_{m_2}, \langle \widehat{\vec{x}}_i \rangle_{m_3}] \rightarrow \text{componentes del vector en RNS} \\
\text{Donde: } \langle \widehat{\vec{x}}_i \rangle_{m_p} &= \widehat{\vec{x}}_i \bmod m_p \\
p &\in \{1, 2, 3\}
\end{aligned} \tag{4.92}$$

Los valores de α y β fueron 4 y 5 respectivamente, logrando un rango dinámico $M = 16368$. Estos valores fueron escogidos conforme a las capacidades de la tarjeta de desarrollo.

4.3.7 Selección del clasificador

Para el clasificador del sistema se decidió utilizar una SVM ya que mostraba un desempeño similar a algoritmos más sofisticados como las Redes Neuronales Convolucionales (*Convolutional Neural Network - CNN*) [Park and Lee, 2016a]. Específicamente, en [Park and Lee, 2016b] se referencia una variante de clasificador SVM con un 80 % de precisión que solo está por debajo de una CNN en un 10 %, lo que implica poder realizar un clasificador con SVM que contiene operaciones menos complejas que una CNN con un desempeño útil para el problema. Respecto a los clasificadores tipo LDA, aunque resultan sencillos de implementar, en el estado del arte mostrado en [Hakonen et al., 2015] se recomienda el uso de clasificadores de mayor complejidad para lograr una buena clasificación de movimientos a largo plazo, por lo cual el clasificador LDA fue descartado. Además, en [Gu et al., 2018] se muestra que una SVM con kernel de base radial (*Radial Basis Kernel - RBF*) tiene un mejor desempeño para la clasificación de señales EMG respecto al LDA.

Otro factor decisivo fue el hecho de encontrar que para las bases de datos NinaPro la

SVM con RBF tuvo un mejor desempeño en clasificación que el perceptrón multicapa (MLP), la SVM lineal, el clasificador k-NN, y el LDA [Atzori et al., 2014b].

Por otro lado, el kernel utilizado fue de base radial, siguiendo las recomendaciones de varios autores [Fontana, 2010] [Park and Lee, 2016a] [Oskoei and Hu, 2008], se utilizaron parámetros $\gamma = 1$ y $C = 10$ establecidos por los intervalos recomendados por los mismos autores.

5. Pruebas y resultados

Dentro de las pruebas planteadas para validar el sistema, se hizo lo siguiente:

- Prueba 1: Se realizaron pruebas con un conjunto de puntos en software para determinar que los algoritmos de clasificación basados en SVM con RNS funcionaran correctamente.
- Prueba 2: Se realizaron pruebas con un conjunto de puntos en hardware para determinar que los algoritmos de clasificación basados en SVM con RNS funcionaran correctamente.
- Prueba 3: Se realizaron pruebas con un conjunto de puntos de una base de datos de señales EMG de miembros superiores en software y hardware para determinar que los algoritmos de clasificación basados en SVM con RNS y las etapas previas de procesamiento de señales funcionaran correctamente.

5.1 Prueba 1: Pruebas de concepto de la *SVM-RNS* en software

Utilizando los métodos propuestos en las secciones anteriores, las *SVM* con base radial aplicadas a enteros en módulo-M y RNS fueron implementados en software como una prueba de concepto. El lenguaje de programación utilizado para este propósito fue Python, en conjunto con las librerías Numpy, sklearn, matplotlib y un método de entrenamiento basado en un algoritmo simplificado SMO (*SMO-lite*) [Tymchyshyn and Khlevniuk, 2020]. El problema de ejemplo analizado consistió de 25 muestras de características bidimensionales etiquetadas en dos grupos, donde el grupo principal se centró en un *cluster*. Todas las muestras fueron normalizadas y se les añadió un ruido Gaussiano, cuya desviación estándar fue de 0.1. La Función Radial en el kernel utilizó un parámetro γ de valor 1.

La idea de utilizar este tipo de muestras fue obtener una visión general de la capacidad de obtener una SVM basada en RNS a partir de los planteamientos teóricos descritos con anterioridad, con la de clasificar un *cluster* que no tuviese una

separación lineal para visualizar la capacidad del Kernel tipo RBF.

En las figuras 5.1, 5.2, 5.3 se muestran las superficies de decisión de la *SVM* de base radial aproximadas a módulo M , y se compara con el algoritmo *SMO-lite*, la *SVM* de la librería *sklearn* y una aproximación por Series de Maclaurin con un polinomio de grado nueve. También se muestra el efecto de cambiar el rango dinámico M con los parámetros α y β en las formas de las superficies de decisión. Para contrastar mejor los resultados, en la tabla 5.1 se muestra un resumen de los resultados obtenidos para cada conjunto de módulos.

Coefficientes del conjunto de módulos	Rango dinámico	Parámetros calculados
$\alpha = 2, \beta = 6$	$M = 16380$	▪ $g_{\text{lim}} = 3.584$ ▪ $G_{\text{plane}} = 3275$ ▪ $g_{\text{bias}} = 913.554$ ▪ $g_{\text{lambda}} = 32$ ▪ $g_{\text{exp}} = 14.342$
$\alpha = 4, \beta = 5$	$M = 16368$	▪ $g_{\text{lim}} = 3.707$ ▪ $G_{\text{plane}} = 3273$ ▪ $g_{\text{bias}} = 882.807$ ▪ $g_{\text{lambda}} = 32$ ▪ $g_{\text{exp}} = 14.830$

Sigue en la próxima página.

Coeficientes del conjunto de Rangos dinámicos		Parámetros calculados
$\alpha = 7, \beta = 5$		$M = 524256$
		▪ $g_{lim} = 3.810$ ▪ $G_{plane} = 104851$ ▪ $g_{bias} = 27516.964$ ▪ $g_{lambda} = 1048$ ▪ $g_{exp} = 499.163$

Cuadro 5.1: Resumen de comparativas de la SVM en módulo M

Por otro lado, el comportamiento que tuvo la función signo utilizando el método *LPN* con un $\alpha = 4, \beta = 5, M = 16368$ se muestra en la figura 5.4: La primera mitad se etiquetó con 1 para denotar que se encontraba dentro de los números enteros positivos, y la segunda mitad se etiquetó con -1 para los negativos. Para una entrada de valor 0, su signo se representa como 0.

Finalmente, se probó el sistema *SVM-RNS* de Kernel *RBF*, donde su superficie de decisión resultó similar a las demás, como se aprecia en la figura 5.5

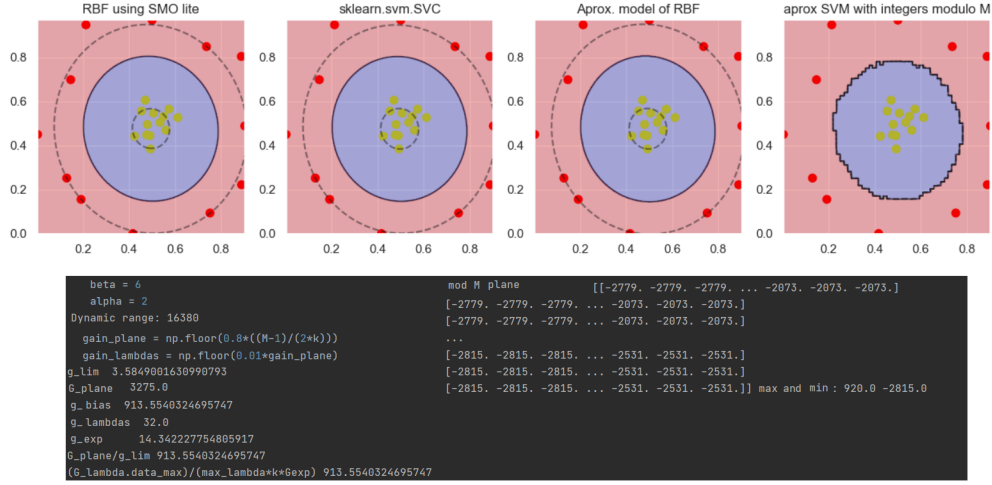


Figura 5.1: *SVM* en enteros módulo M con $\alpha = 2, \beta = 6, M = 16380$.

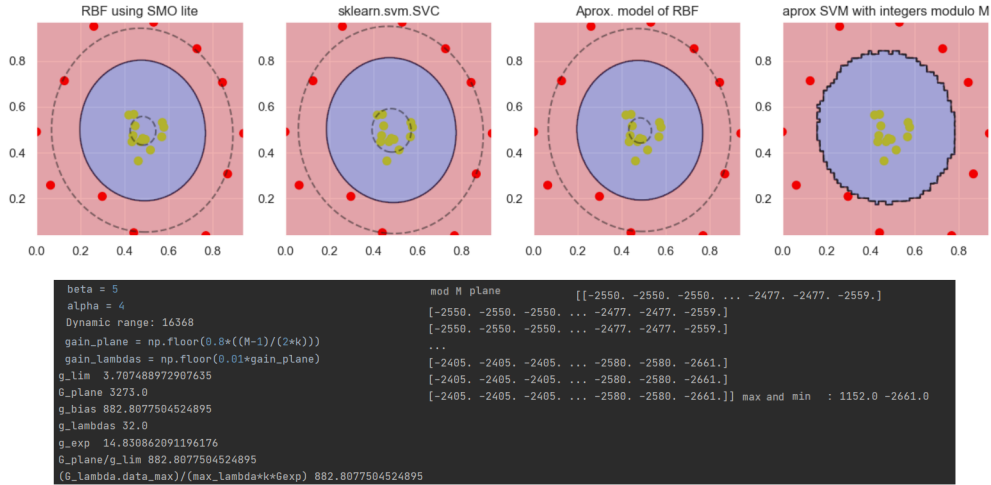


Figura 5.2: *SVM* en enteros módulo M con $\alpha = 4, \beta = 5, M = 16368$.

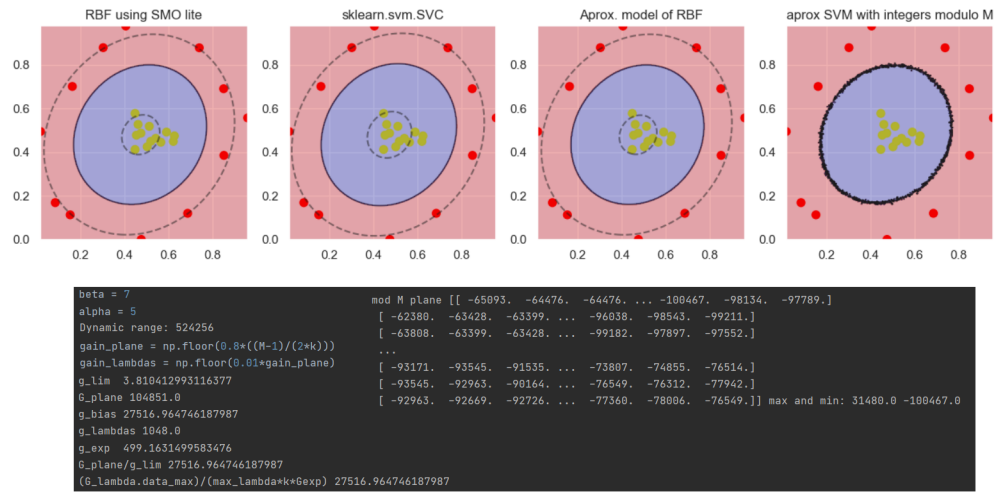


Figura 5.3: *SVM* en enteros módulo M con $\alpha = 5, \beta = 7, M = 524256$.

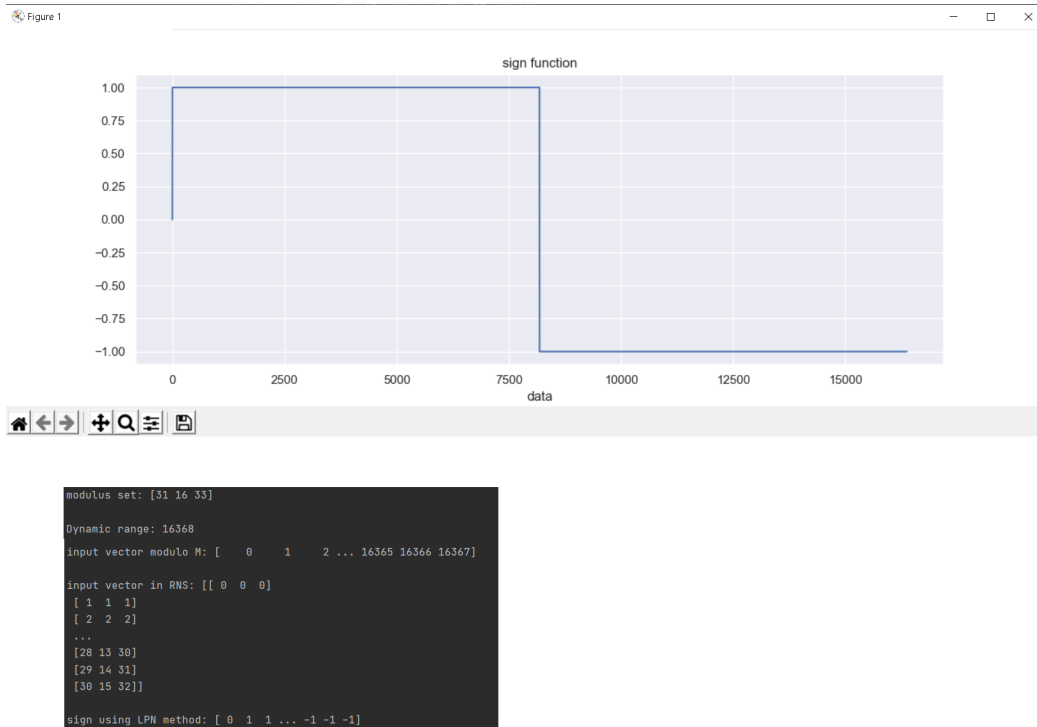


Figura 5.4: Función signo en RNS $\alpha = 4, \beta = 5, M = 16368$.

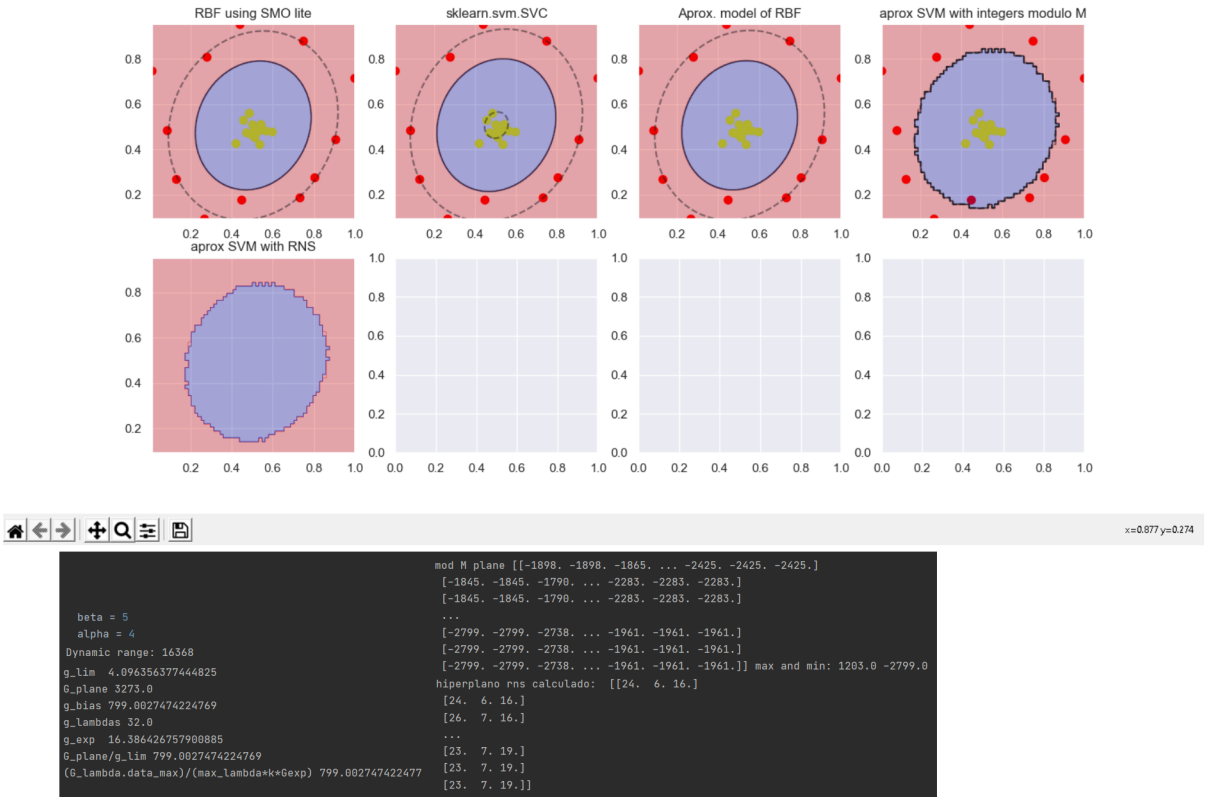


Figura 5.5: Ejemplo funcionamiento en RNS $\alpha = 4, \beta = 5, M = 16368$.

Debe resaltarse que los hiperplanos de decisión en la figura 5.5 fueron similares entre los diferentes tipos de SVM, incluyendo el obtenido con la librería *sklearn* y su equivalente con SVM-RNS basado en SMO-lite, permitiendo mostrar la aplicabilidad de la última. Por otro lado, gracias a los planteamientos teóricos se pudieron obtener los parámetros de ganancias en el hiperplano, en el *bias* en los Multiplicadores de Lagrange (*lambdas*) y el uso práctico de la obtención de los puntos límites de la superficie de decisión (*Random Search*), mostrados como las salidas de ejecución (pantallas de fondo negro en las figuras 5.5, 5.1, 5.2 y 5.3) Los parámetros obtenidos del entrenamiento de la SVM se almacenaron en archivos de extensión **.data* para inicializar las memorias del bloque *PL* de la plataforma. Para la reducción *PCA* se almacenaron los parámetros

en archivos compatibles para el proyecto del bloque *PS* en extensiones **.h* y **.c*.

5.2 Prueba 2: Pruebas de concepto de la *SVM-RNS* en hardware

Para la realización en hardware de la prueba de concepto, se utilizó una metodología codiseño en la plataforma *Zynqberry* aprovechando la rapidez de diseño en el bloque *PS* y la versatilidad de diseñar en el bloque *PL*.

Dentro de la prueba de concepto se implementó el clasificador *SVM-RNS* con dos categorías de ejemplo que no pertenecían a la base de datos de señales EMG *NinaPro*. Para lograr tal fin, se desarrolló un código en Python que describió dos grupos con categorías bidimensionales y normalizadas al rango $[0,1]$. Posteriormente los parámetros de funcionamiento se exportaron para ser utilizados en el proyecto diseñado en el software *Vivado*, cuyos bloques generados para el sistema principal se muestran en la figura 5.6

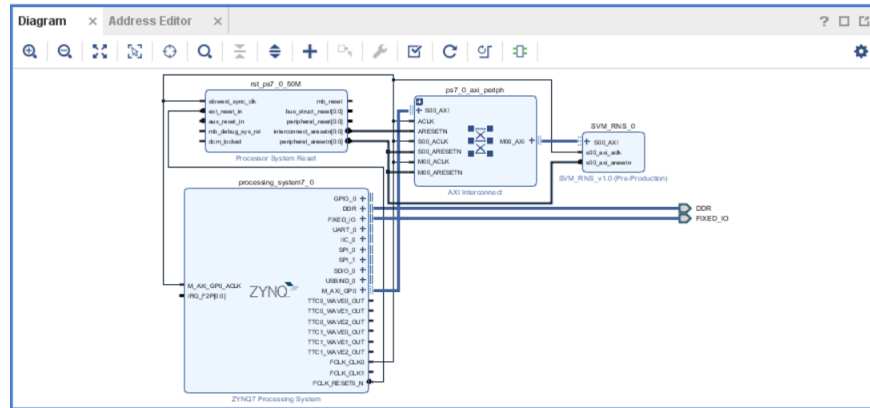


Figura 5.6: Esquema de los bloques hardware realizados en *Vivado*

Debe tenerse en cuenta que los conjuntos de datos tuvieron una distribución similar a la que se muestra en el esquema de la figura 5.7

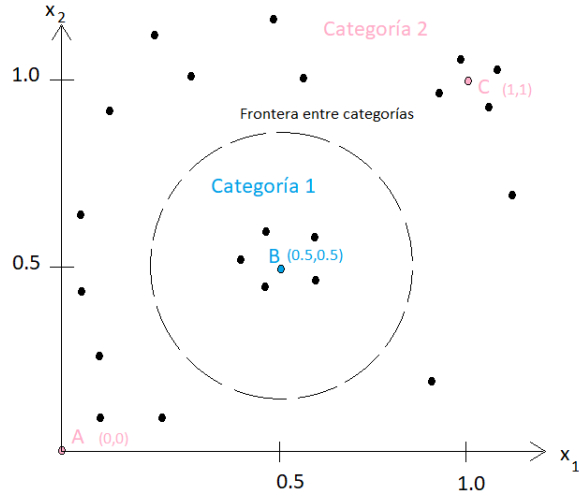


Figura 5.7: Esquema de la distribución de características según las categorías

En el esquema se muestran tres puntos de prueba bidimensionales A, B, C que recorren las nubes de puntos en las dos categorías. Según los métodos de conversión a RNS, a cada punto A, B, C se le asoció un punto $\hat{A}, \hat{B}, \hat{C}$ como se describe a continuación.

Sea un punto $x = \{x_1; x_2\}$, con $x_1, x_2 \in [0, 1]$, cada característica x_i se convierte a su equivalente RNS $\hat{x}_i$ como $\hat{x}_i = \left\lfloor \sqrt{\frac{D_M}{k}} \cdot x_i \right\rfloor$, donde $D_M = \frac{M}{2} - 1$, $k = 2$ es el número de características y $M = m_1 \cdot m_2 \cdot m_3$ es el rango dinámico. Con $\alpha = 4$ y $\beta = 5$ el conjunto de módulo, rango dinámico y los restantes parámetros de la conversión estarían dados

por:

$$\begin{aligned}
m_1 &= 2^\beta - 1 = 2^5 - 1 = 31 \\
m_2 &= 2^\alpha = 2^4 = 16 \\
m_3 &= 2^\beta + 1 = 2^5 + 1 = 33 \\
M &= m_1.m_2.m_3 = 31.16.33 = 16368 \\
D_M &= \frac{16368}{2} - 1 = 8183 \\
\frac{D_M}{k} &= 4091,5 \\
\sqrt{\frac{D_M}{k}} &= 63,9648 \\
x_{MAX} &= \left\lfloor \sqrt{\frac{D_M}{k}} \right\rfloor = 63
\end{aligned} \tag{5.1}$$

Utilizando los parámetros obtenidos, la representación de cada punto en módulo M fue la siguiente:

$$\begin{aligned}
A = \{0; 0\} &\rightarrow \widehat{A} = \{\widehat{A}_0; \widehat{A}_1\} = \{0; 0\} \\
B = \{0, 5; 0, 5\} &\rightarrow \widehat{B} = \{\widehat{B}_0; \widehat{B}_1\} = \{31; 31\} \\
C = \{1; 1\} &\rightarrow \widehat{C} = \{\widehat{C}_0; \widehat{C}_1\} = \{63; 63\}
\end{aligned} \tag{5.2}$$

Dada la representación en módulo M, se pudo hallar la representación en el espacio de los residuos:

$$\begin{aligned}
\widehat{A}_{RNS} &= \{\{\widehat{A}_0 \bmod m_1, \widehat{A}_0 \bmod m_2, \widehat{A}_0 \bmod m_3, \}; \{\widehat{A}_1 \bmod m_1, \widehat{A}_1 \bmod m_2, \widehat{A}_1 \bmod m_3\}\} \\
\rightarrow \widehat{A}_{RNS} &= \{\{0, 0, 0\}; \{0, 0, 0\}\} \\
\widehat{B}_{RNS} &= \{\{\widehat{B}_0 \bmod m_1, \widehat{B}_0 \bmod m_2, \widehat{B}_0 \bmod m_3, \}; \{\widehat{B}_1 \bmod m_1, \widehat{B}_1 \bmod m_2, \widehat{B}_1 \bmod m_3\}\} \\
\rightarrow \widehat{B}_{RNS} &= \{\{0, 15, 31\}; \{0, 15, 31\}\} \\
\widehat{C}_{RNS} &= \{\{\widehat{C}_0 \bmod m_1, \widehat{C}_0 \bmod m_2, \widehat{C}_0 \bmod m_3, \}; \{\widehat{C}_1 \bmod m_1, \widehat{C}_1 \bmod m_2, \widehat{C}_1 \bmod m_3\}\} \\
\rightarrow \widehat{C}_{RNS} &= \{\{1, 15, 30\}; \{1, 15, 30\}\}
\end{aligned} \tag{5.3}$$

La prueba funcional consistió en verificar la clasificación de los puntos con la SVM-RNS,

de manera que se obtuvo:

$$\begin{aligned} \text{SVM-RNS}\{\hat{A}_{RNS}\} &= -1 \\ \text{SVM-RNS}\{\hat{B}_{RNS}\} &= 1 \\ \text{SVM-RNS}\{\hat{C}_{RNS}\} &= -1 \end{aligned} \quad (5.4)$$

Utilizando las herramientas de simulación funcional de Vivado, se validaron los resultados para los tres puntos como se muestra en las figuras 5.8, 5.9 y 5.10. Debe tenerse en cuenta que en las simulaciones se obtuvo un resultado de simulación cada vez que la señal *finish_flag_signal* tomaba un valor de 1 lógico.

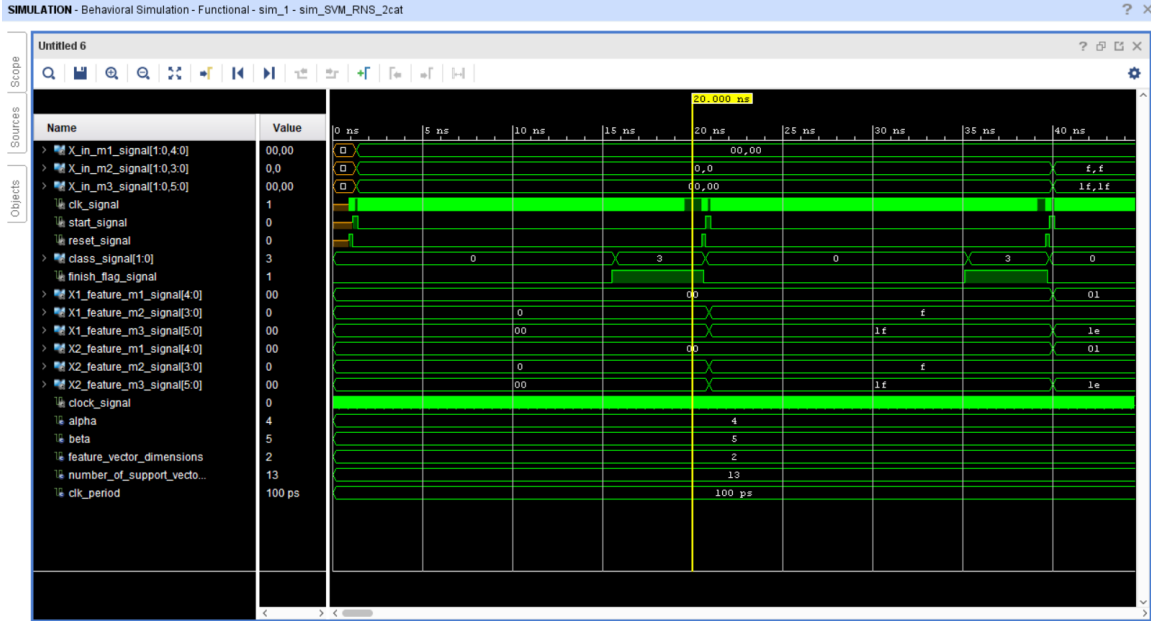


Figura 5.8: Simulación del punto $\hat{A}_{RNS} = \{\{0, 0, 0\}; \{0, 0, 0\}\}$

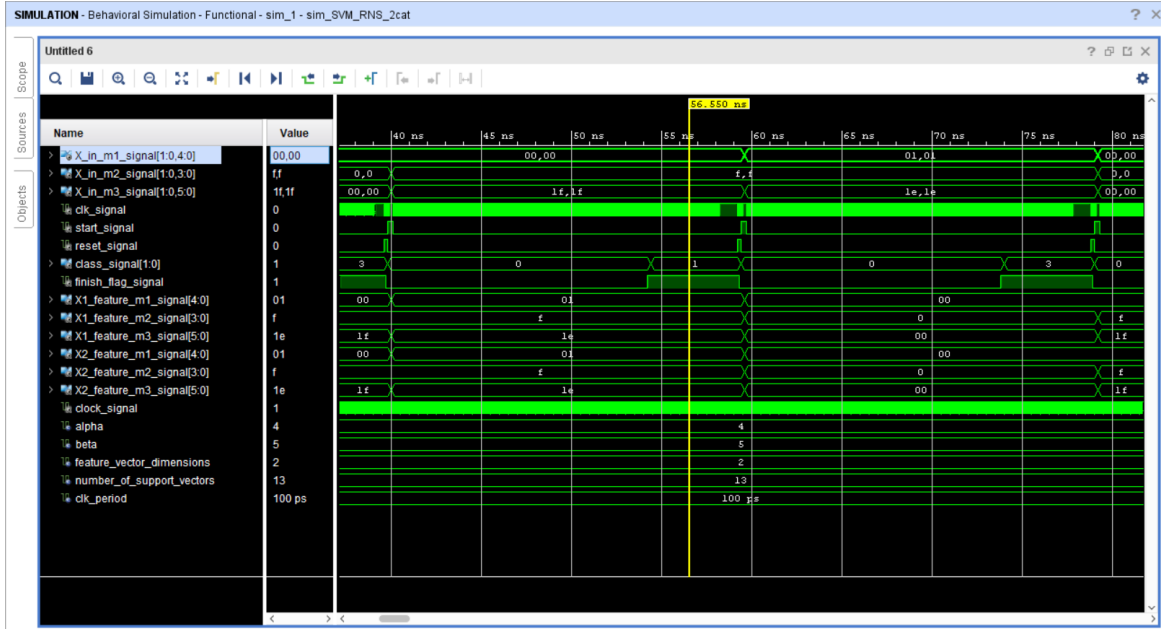


Figura 5.9: Simulación del punto $\hat{B}_{RNS} = \{\{0, 15, 31\}; \{0, 15, 31\}\}$

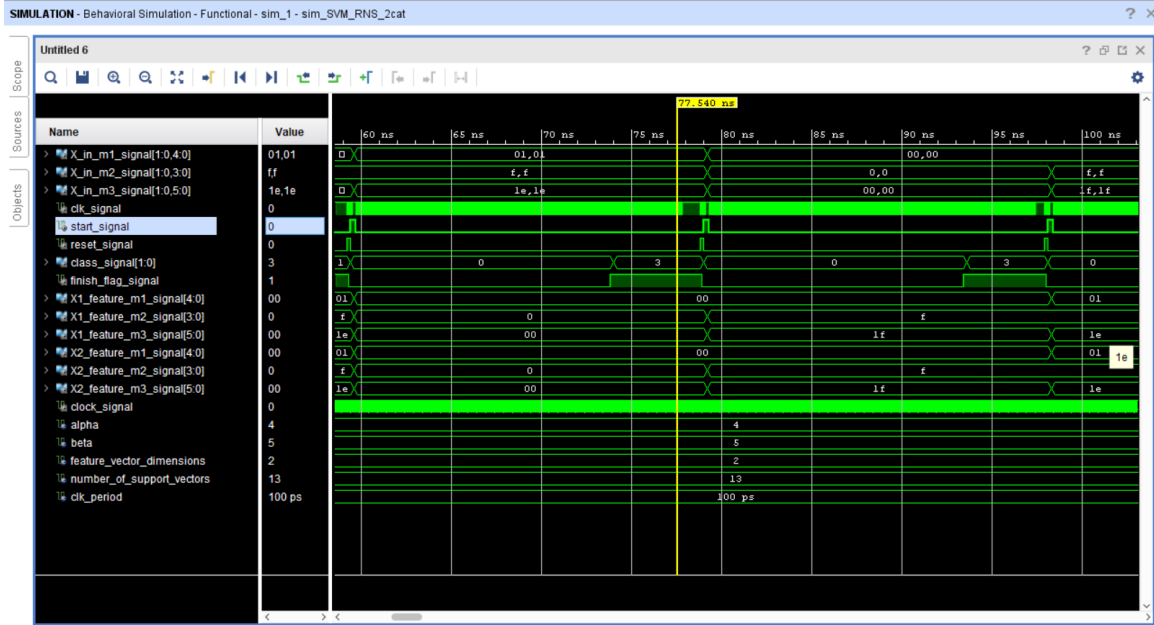
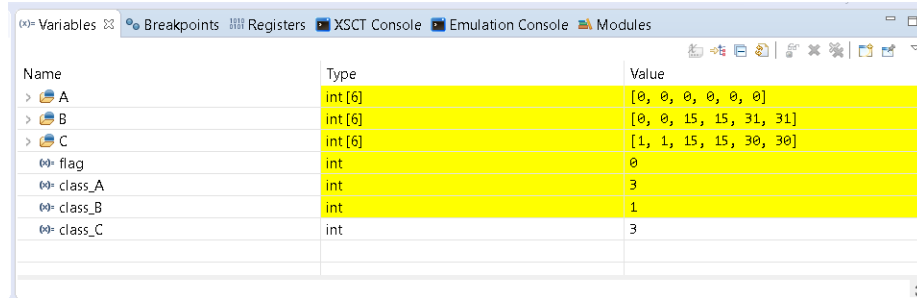


Figura 5.10: Simulación del punto $\hat{C}_{RNS} = \{\{1, 15, 30\}; \{1, 15, 30\}\}$

Posterior a la validación en las simulaciones, se realizó una prueba del sistema

completo utilizando las herramientas de depuración (*debug*) del software *Xilinx SDK*, permitiendo verificar la implementación funcional del sistema como se aprecia en la figura 5.11.



Name	Type	Value
> A	int [6]	[0, 0, 0, 0, 0, 0]
> B	int [6]	[0, 0, 15, 15, 31, 31]
> C	int [6]	[1, 1, 15, 15, 30, 30]
flag	int	0
class_A	int	3
class_B	int	1
class_C	int	3

Figura 5.11: Resultados de la prueba codiseño con las herramientas de *Xilinx SDK*

5.3 Prueba 3: Prueba de concepto de la SVM-RNS en hardware con señales EMG

Luego de haber realizado las pruebas concernientes a los modelos en hardware y software de la SVM-RNS, se planteó el uso del sistema para explorar análisis de pares de movimientos representados por señales EMG contenidas dentro de la base de datos NinaPro. Inicialmente se realizó todo el código en lenguaje C para lograr obtener la segmentación de datos, extracción, selección, reducción y normalización de características. Para lo anterior, se partió de realizar el código en un computador portátil y luego de que pasara pruebas funcionales se adaptó al lenguaje C con las restricciones y particularidades del microcontrolador disponible en el SoC de la *Zynqberry*.

Al tener disponibles las funcionalidades previas requeridas para la clasificación, se implementó junto con la SVM-RNS un sistema que permitió la clasificación de dos movimientos, utilizando la base de datos NinaPro. En general, las características de entrada hacia el clasificador provenían de los resultados del microcontrolador, mientras que la SVM fue desarrollada en su totalidad en lenguaje VHDL. La interconexión de los bloques *PS* y *PL* se realizó con las herramientas provistas por *Vivado*, a través del protocolo de comunicación AXI. Esto permitió llegar al prototipo requerido por el objetivo general del proyecto.

En mayor detalle, al evaluar la clasificación de movimientos se escogieron de la base

de datos número 3 (*DB3*) a los sujetos *S1* (50 % de antebrazo remanente), *S2* (70 % de antebrazo remanente), *S3* (30 % de antebrazo remanente), *S5* (90 % de antebrazo remanente) los ejercicios de tipo 2 (movimientos de muñeca y de agarre) etiquetados en el campo *restimulus* de la siguiente manera:

- Movimientos tipo 0: los que poseían como etiqueta la número 22 (*Medium wrap* [Atzori et al., 2014a]).
- Movimientos tipo 1: aquellos con etiqueta número 27(*Power sphere grasp* [Atzori et al., 2014a]).
- Movimientos tipo 2: con número de etiqueta 28(*Three finger sphere grasp* [Atzori et al., 2014a]).

Debe tenerse en cuenta que para cada par de movimientos se realizó un procedimiento con los scripts *feature_extraction_test.py* y *feature_extraction_nina.py* para obtener los parámetros requeridos en el entrenamiento del clasificador:

1. **Importación de datos:** Dado que las bases de datos se encuentran en un formato *.mat compatible con MATLAB y los scripts del proyecto se realizaron en python, se utilizó la función *loadmat* de *scipy* para obtener los datos en forma de diccionario (compatible con Python).
2. **Creación de *dataframes*:** De los diccionarios obtenidos en el paso anterior, se tomaron los campos más importantes para el caso de estudio: señales emg de los sensores, etiqueta del tipo de movimiento y repetición de movimiento. Esta información es colocada en objetos tipo *DataFrame* de la utilidad *Pandas*, dado que facilitaba la manipulación de información dentro de Python.
3. **Segmentación:** Para cada una de las señales provenientes de los 12 electrodos se realizó una segmentación de 200 ms (400 muestras) sin solapamiento entre ventanas. Para ello se añadió una campo al Dataframe, cuyo contenido indicó con una enumeración la pertenencia a un segmento.
4. **Extracción de características:** De los segmentos obtenidos, se realizó una extracción de características: MAV, WL, ZC, RMS, SSC y MAVS.

5. **Selección de características:** Se utilizó el método RFE provisto por la utilidad *sklearn* para obtener a partir del paso anterior las características más importantes para cada par de movimientos evaluado. Para ello, se fijó una reducción con las primeras 6 características más representativas según un modelo de regresión lineal provisto por la función *LinearRegression* de la utilidad *sklearn*.
6. **Reducción de características con PCA:** Luego de obtener las características más representativas, se utilizó el método PCA para conseguir una reducción a dos componentes.
7. **Normalización de los datos PCA:** Los datos del paso anterior fueron normalizados al rango $[0, 1]$ para luego utilizarse como insumo dentro de la etapa de entrenamiento de la SVM-RNS realizada en software.
8. **Creación de archivos para el bloque PS:** En este paso se crean dos archivos *features_for_movements_A_vs_B.h* y *features_for_movements_A_vs_B.c* que serán utilizados en la programación del bloque PS de la tarjeta de desarrollo, donde A y B correspondieron a los números dados al par de tipos de movimiento. El contenido de estos archivos fueron: los parámetros de la conversión PCA (matriz de ejes con las componentes principales y promedio de las características), los nombres de las características que resultaron del procedimiento RFE y señales EMG que fueron utilizadas para realizar las pruebas en la implementación hardware.
9. **Creación de archivos con datos para el entrenamiento de la SVM-RNS:** Los datos de entrenamiento y prueba obtenidos de la normalización en PCA y sus etiquetas se almacenaron como *DataFrames* en archivos tipo *pickle* que luego fue utilizado para el entrenamiento de la SVM-RNS. Estos fueron: *Xtrain.pkl*, *ytrain.pkl*, *Xtest.pkl* y *ytest.pkl*.

Aunque los scripts utilizados en el diseño final no mostraron gráficamente los pasos de lectura de las bases de datos NinaPro hasta la obtención de componentes con PCA, si se pudo hacer una inspección visual de los procedimientos realizados gracias al script *nina_feature_extraction_clean.py*, el cual fue utilizado como la base para la extracción de características en el desarrollo final. Por ejemplo, respecto a la segmentación con

etiquetas que se realizó a los movimientos, en la figura 5.12 se muestra en orden la información que se obtuvo a partir de las bases de datos:

- Las muestras obtenidas de las señales EMG para el movimiento con la etiqueta 22, del sujeto 0 y del canal 3.
- La enumeración de los segmentos por cada repetición del movimiento. Esta segmentación fue realizada a partir de los *DataFrames*, y no venía provista por la base de datos.
- La etiqueta de repetición de movimiento, para usos posteriores solo se tomaron 5 repeticiones por cada movimiento, dado que la sexta tenía truncamiento.
- La etiqueta del movimiento analizado, que en este caso correspondió al movimiento 22.

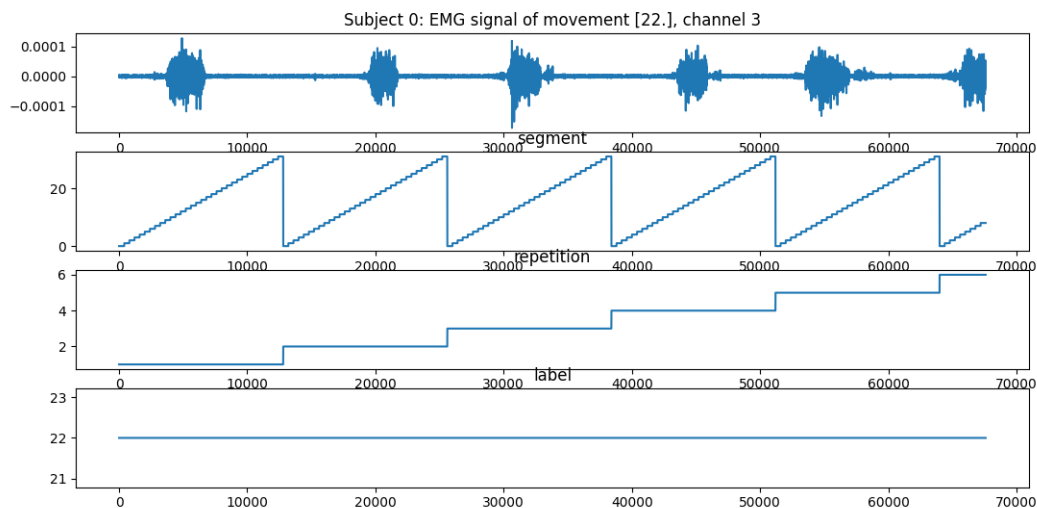


Figura 5.12: Segmentación de un movimiento completo

Este tipo de etiquetado facilitó los filtrados en los *DataFrames* para realizar la extracción de características en cada segmento según su repetición, porque permitía tener un campo con etiqueta específica por segmento. Por ejemplo, en la figura 5.13 se muestran las etiquetas de segmentación para una repetición de movimiento.

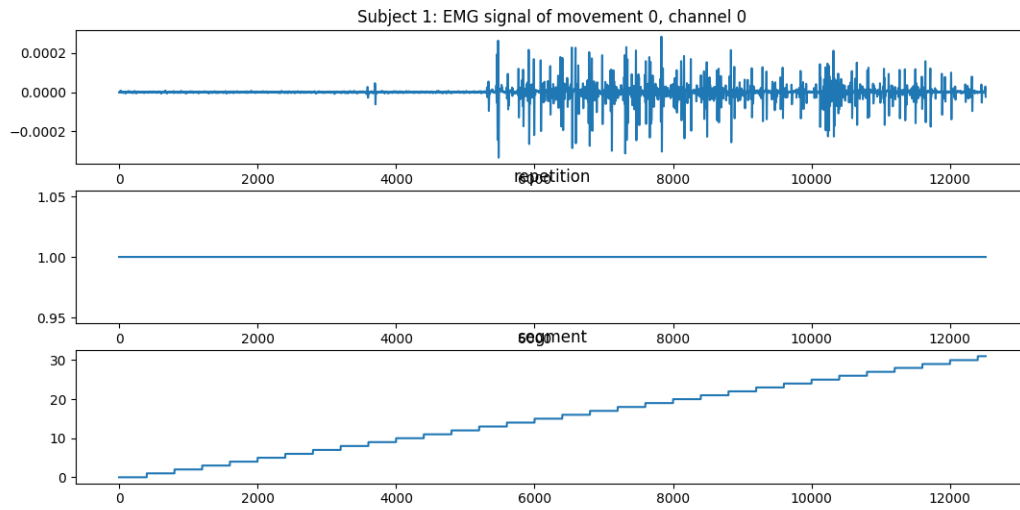


Figura 5.13: Segmentación de una repetición en un movimiento

Para la extracción de características, se observó un comportamiento como el de la figura 5.14, donde se muestran las características *MAV*, *ZC* y *MAV* para un movimiento completo del canal 4.

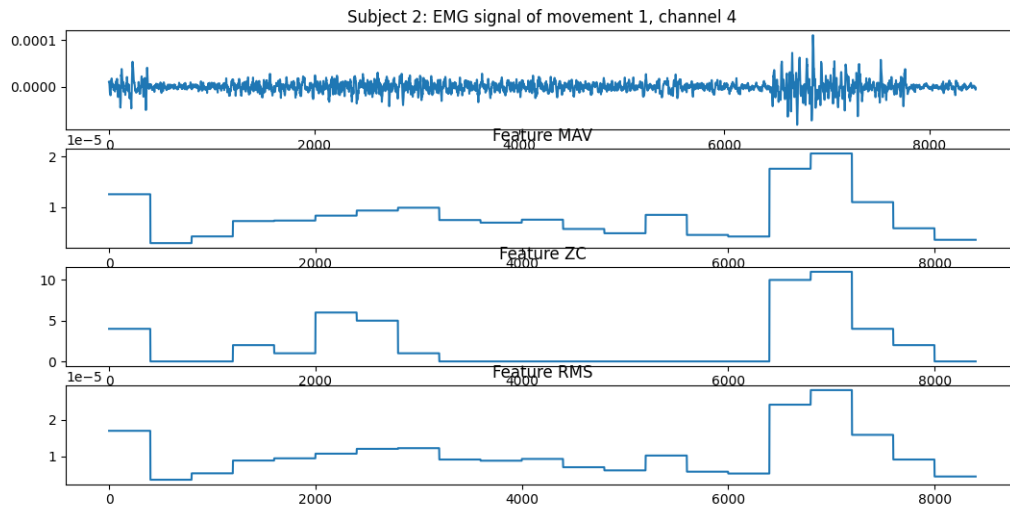


Figura 5.14: Características extraídas para un solo canal (electrodo 4 - antebrazo)

Las componentes PCA obtenidas a partir de la selección de características con el método RFE también se pudieron observar como grupos de color rojo y azul dependiendo del tipo de movimiento, como se muestra en la figura 5.15

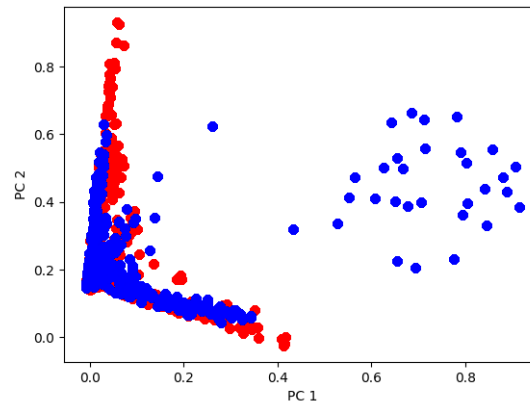


Figura 5.15: Componentes PCA obtenidas de la selección con RFE para un par de movimientos)

Adicionalmente, al finalizar el proceso relacionado con las etapas desde el preprocesamiento hasta normalización de características, con los scripts *feature_extraction_test.py* y *feature_extraction_nina.py* se realizó una visualización previa de una SVM de base radial de prueba con el módulo *SVC* de la utilidad *sklearn* que utilizara las características normalizadas de PCA en su entrada. Para ello, la superficie de decisión se mostraba en regiones rojas y azules denotando una Por ejemplo en la figura 5.16 se muestra un clasificador de prueba proveniente de las herramientas disponibles en *sklearn* entrenado para los movimientos tipo 1 y tipo 2. En la figura 5.17 se observan algunas métricas estadísticas donde se obtuvo una precisión de clasificación del 67% para un conjunto de datos de prueba y del 68% para datos del conjunto de entrenamiento. Debe resaltarse que a pesar de disminuir la dimensionalidad de las características con PCA fue posible obtener un porcentaje de clasificación importante en los datos de prueba, además de que los datos de entrenamiento provenían de sujetos con porcentajes de amputación variados.

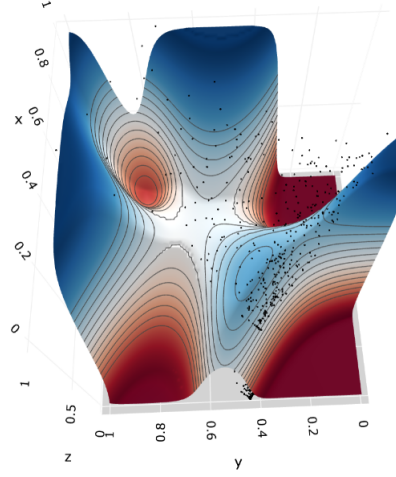


Figura 5.16: Clasificador de prueba para los movimientos tipo 1 (etiqueta 27) y tipo 2 (etiqueta 28) en la etapa de procesamiento de los datos de la base de datos NinaPro

```

----- Evaluation on Test Data -----
Accuracy Score: 0.6738056013179572

```

	precision	recall	f1-score	support
0	0.70	0.72	0.71	1012
1	0.64	0.62	0.63	809
accuracy			0.67	1821
macro avg	0.67	0.67	0.67	1821
weighted avg	0.67	0.67	0.67	1821

```

-----
----- Evaluation on Training Data -----
Accuracy Score: 0.6782799748901444

```

	precision	recall	f1-score	support
0	0.70	0.74	0.72	1775
1	0.65	0.60	0.62	1411
accuracy			0.68	3186
macro avg	0.67	0.67	0.67	3186
weighted avg	0.68	0.68	0.68	3186

Figura 5.17: Estadísticas obtenidas para el clasificador de prueba con los movimientos tipo 1 y tipo 2

Posteriormente, se entrenó la SVM-RNS en los scripts de Python para la clasificación de los siguientes pares de movimientos:

- Tipo 0 vs tipo 1: utilizando un $\beta = 5$ y un $\alpha = 4$, su superficie de decisión se observa en la figura 5.18.
- Tipo 1 vs tipo 2: utilizando un $\beta = 5$ y un $\alpha = 4$, su superficie de decisión se observa en la figura 5.19.
- Tipo 2 vs tipo 0: utilizando un $\beta = 5$ y un $\alpha = 4$, su superficie de decisión se observa en la figura 5.20.

Para ello, se utilizaron los procedimientos descritos en los scripts *SVM_test_v2.py* y *SVM_RNS.py* que permitieron realizar comparativas entre SVM de base radial modeladas de las siguientes formas:

- Con un hiperplano RBF Gaussiano y el método de entrenamiento SMO-lite[Tymchyshyn and Khlevniuk, 2020].
- Utilizando el módulo *SVC* de la utilidad *sklearn*.
- Con una aproximación del hiperplano RBF Gaussiano con una serie de Taylor de grado 8 y el método de entrenamiento SMO-lite[Tymchyshyn and Khlevniuk, 2020].
- Con un hiperplano RBF Gaussiano y el método de entrenamiento SMO-lite[Tymchyshyn and Khlevniuk, 2020], aplicados a un conjunto de números enteros en módulo M.
- Con un hiperplano RBF Gaussiano y el método de entrenamiento SMO-lite[Tymchyshyn and Khlevniuk, 2020], aplicados a un conjunto RNS con rango dinámico M (SVM-RNS).

De manera general, como fue descrito en la secuencia de pasos realizados con los scripts *feature_extraction_test.py* y *feature_extraction_nina.py* en las diferentes pruebas puede observarse que el uso del algoritmo de entrenamiento SMO-lite en conjunto con el RNS estableció unos resultados de clasificación similares a lo que se provee en paqueterías populares como *sklearn*, lo cual implica que el método propuesto es útil para los fines del proyecto.

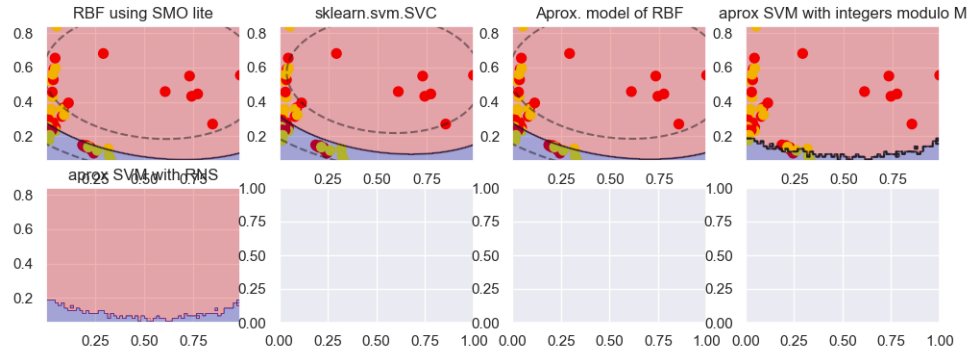


Figura 5.18: Clasificador RNS para el movimiento tipo 0 vs 1 con un $\beta = 5$ y un $\alpha = 4$.

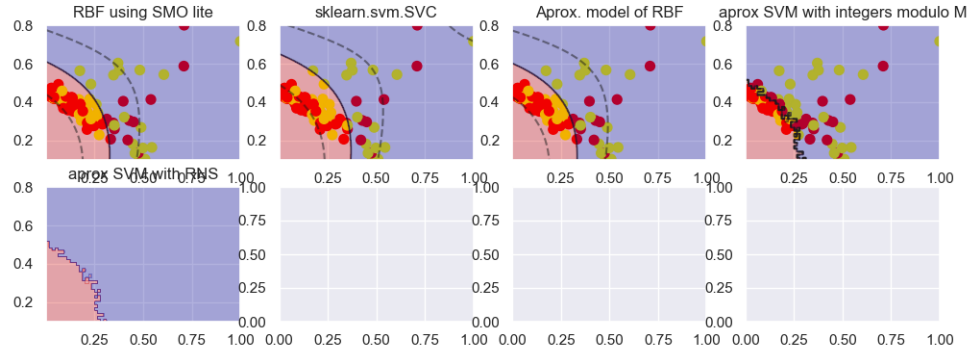


Figura 5.19: Clasificador RNS para el movimiento tipo 1 vs 2 con un $\beta = 5$ y un $\alpha = 4$.

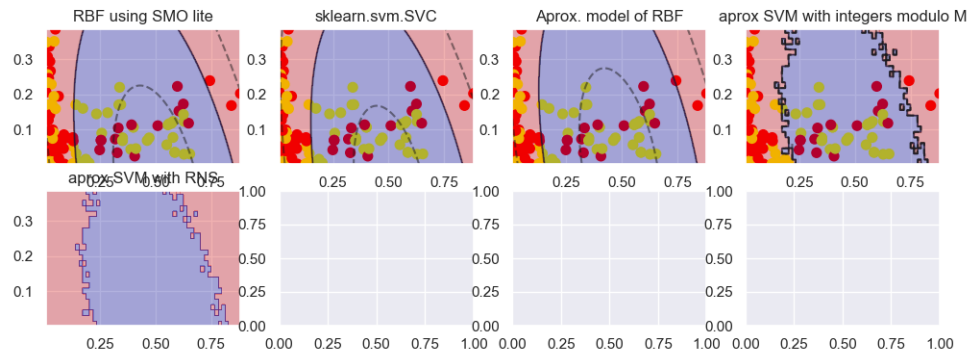


Figura 5.20: Clasificador RNS para el movimiento tipo 2 vs 0 con un $\beta = 5$ y un $\alpha = 4$.

Un detalle específico del script *SVM_test_v2.py* es que luego de realizar el entrenamiento de la SVM-RNS se hicieron llamados a funciones del script *export_data_files.py* para crear los archivos de inicialización de memoria que fueron utilizados en el bloque *PL* del sistema de desarrollo:

- La función *create_data_kernel* creó los archivos *kernel_lut.data* que contenía la LUT con los datos de la función Kernel.
- La función *create_data_file_params* guardaba los archivos *params_lut.data* con los parámetros de entrenamiento de la SVM: vectores de soporte, multiplicadores de Lagrange, etiquetas de clase de los vectores de soporte.
- La función *create_data_file_bias* guardaba los archivos *bias_lut.data* con el parámetro *bias* obtenido del entrenamiento.

Al utilizar los archivos de inicialización de las memorias del bloque *PL* (FPGA) se pudo observar que los datos almacenados en el *bias*, la memoria de parámetros de vectores de soporte y la memoria de kernel coincidían con lo arrojado por los scripts de Python. Esto se puede constatar para el entrenamiento de los movimientos tipo 0 y tipo 1 en las figuras 5.21, 5.22 y 5.23.

En mayor detalle, gracias a los parámetros provistos por el archivo de inicialización en *bias_lut.data* se pudo utilizar dentro del bloque de la SVM-RNS el *bias* necesario en las operaciones del hiperplano, como se indica en el esquemático generado por Vivado representado en la figura 5.21.

De manera similar, el bloque de memoria que contiene los vectores de soporte, etiquetas de los vectores de soporte y sus Multiplicadores de Lagrange coinciden en todas sus posiciones de memoria con lo provisto por el archivo *params_lut.data*, como se indica en la figura 5.22.

Finalmente, el archivo *kernel_lut.data* coincide en contenidos con todas las posiciones de memoria del bloque de almacenamiento del Kernel de base radial.

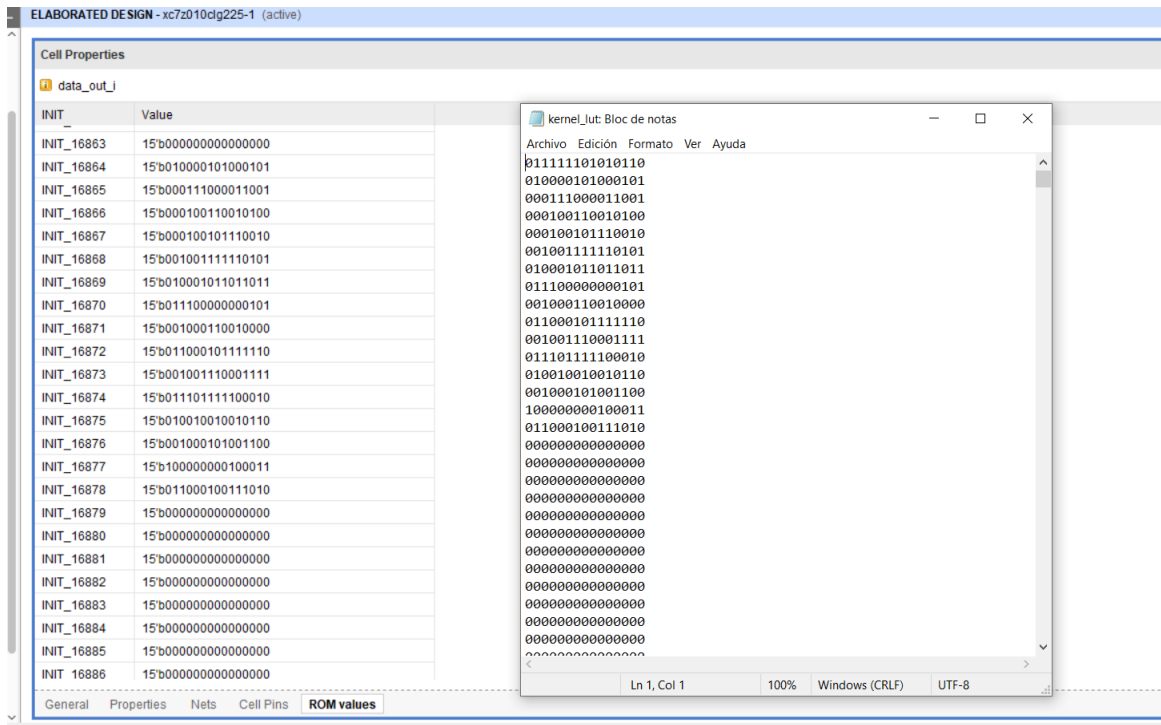


Figura 5.23: Verificación de los datos de inicialización de la memoria dedicada a la función Kernel

Por otro lado, al trasladar los datos a la aplicación diseñada en el bloque *PS* (microcontrolador) del *SoC*, se pudo constatar que cada uno de los pasos realizados en el lenguaje C estándar y los scripts de Python tuvieron un equivalente satisfactorio en la implementación en el sistema embebido. Por ejemplo, en el caso de las matrices de transformación utilizadas en PCA no hubo cambio entre las matrices provenientes de Python, C estándar y la implementación en el *SoC* como se aprecia en la figura 5.24, donde se muestra la matriz PCA para el par de movimientos tipo 0 y 1.

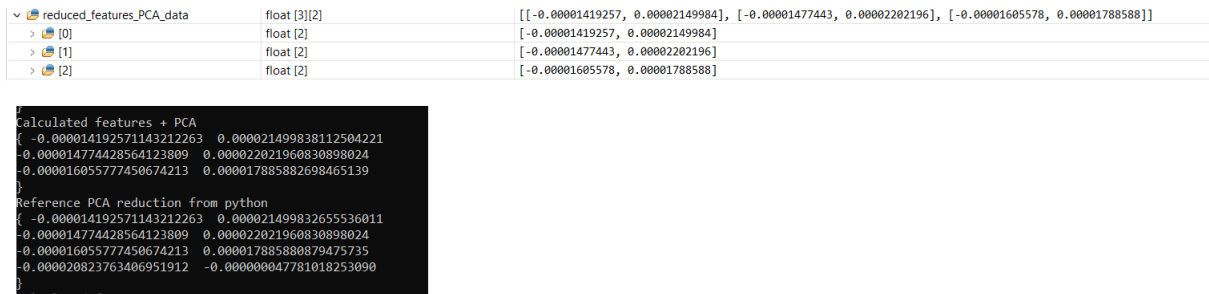


Figura 5.24: Comparativa de las matrices para la reducción PCA de las características obtenidas de NinaPro para un conjunto de datos en Python, C y su ejecución en el SoC

En el paso de la normalización de datos, se hizo una verificación satisfactoria de que no cambiaron los datos al calcularse en un computador portátil y el sistema embebido. En el caso de las características para los movimientos tipo 0 y tipo 1, la normalización se aprecia en la figura 5.25

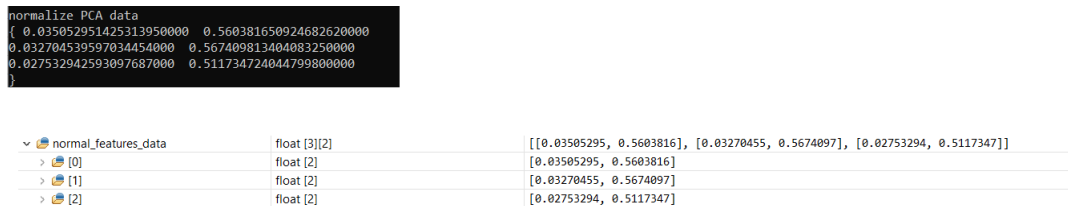


Figura 5.25: Comparativa de la normalización en C y su ejecución en el SoC

Al ejecutar la aplicación para clasificar un punto, se observó que para una muestra de la categoría etiquetada como -1, efectivamente el sistema colocó su correspondiente etiqueta de clase como -1 (3 en complemento a dos de dos bits) en la variable *class_m1_first_point*, como se aprecia en el entorno de depuración mostrado en la figura 5.26

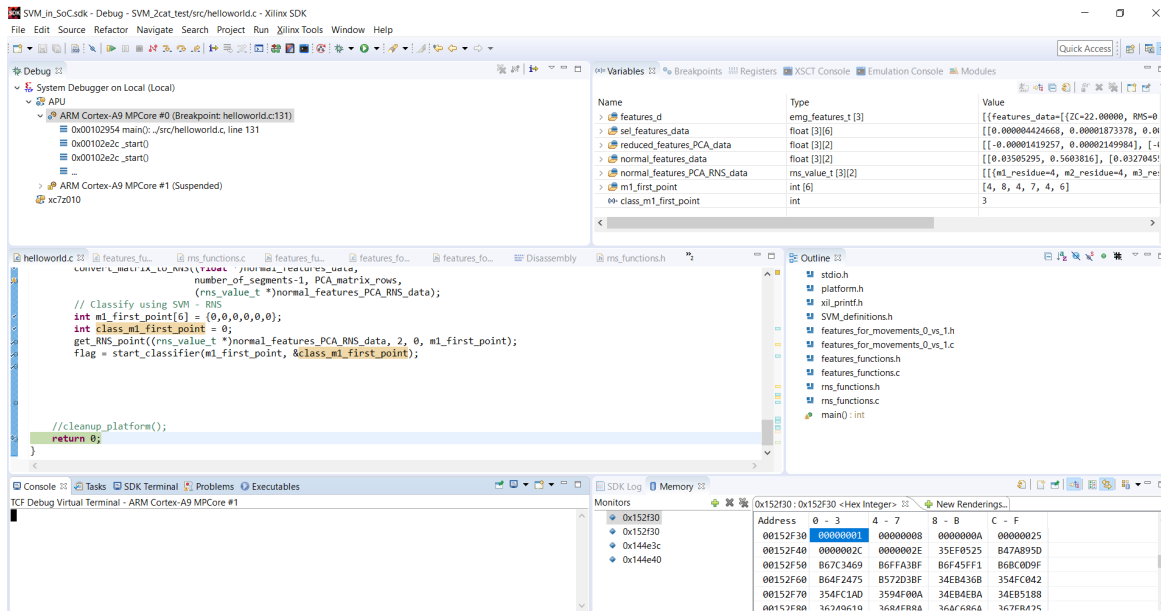


Figura 5.26: Clasificación de un punto en el sistema SVM-RNS

Posteriormente se colocaron 3 puntos de la categoría 1 (movimientos tipo 0) y 3 puntos de la categoría -1 (movimientos tipo 1) que fueron clasificados correctamente por la SVM-RNS como se aprecia en las variables *class_m1_first_point* a *class_m6_first_point* de la figura 5.27

Name	Type	Value
A	int [6]	[0, 0, 0, 0, 0, 0]
B	int [6]	[0, 0, 15, 15, 31, 31]
C	int [6]	[1, 1, 15, 15, 30, 30]
D	int [6]	[0, 0, 0, 15, 0, 31]
flag	int	0
class_A	int	3
class_B	int	1
class_C	int	3
class_D	int	3
emg_data_input	emg_data_segment_t [4]	
emg_data	emg_features_t [3]	[[features_data=[{ZC=77.00000, RMS=0.00005730228, WL=0.01211414, SSC=103.0000, MAV=0.00004133322, MAVS=-0.00...
sel_features_data	float [3][6]	[[[0.00003867537, 0.000004385597, 0.000003121985, 0.00005188457, 0.000005733845, 0.000004533391], [0.00003629...
reduced_features_PCA_data	float [3][2]	[[[0.00003934557, -0.00001057945], [0.00003362555, -0.00001035083], [0.00002263629, -0.000002027977]]
normal_features_data	float [3][2]	[[[0.2511360, 0.1285673], [0.2280496, 0.1316447], [0.1836964, 0.1605590]]
normal_features_PCA_RNS_data	rms_value_t [3][2]	[[[m1_residue=4, m2_residue=4, m3_residue=4], [m1_residue=8, m2_residue=7, m3_residue=6]], [[m1_residue=4, m...
emg_data_input_2	emg_data_segment_t [4]	
features_d_2	emg_features_t [3]	[[features_data=[{ZC=-0.00004849744, RMS=-0.00007172086, WL=-0.00005742389, SSC=-0.00002499312, MAV=-0.00001...
sel_features_data_2	float [3][6]	[[[0.00004754110, 0.00006546522, 0.00001132383, -0.00006291362, -0.0000434445, 0.00001911458], [0.0000396311...
reduced_features_PCA_data_2	float [3][2]	[[[0.00001719245, 0.00002061177], [0.00002257225, 0.00002380551], [0.00002669715, 0.00003045916]]
normal_features_data_2	float [3][2]	[[[-0.00001971927, -0.00002284930], [-0.00002093368, -0.00001410891], [-0.00001849371, 0.00001015031]]
normal_features_PCA_RNS_data_2	rms_value_t [3][2]	[[[m1_residue=32, m2_residue=0, m3_residue=32], [m1_residue=16, m2_residue=0, m3_residue=16]], [[m1_residue=...
m1_first_point	int [6]	[4, 8, 4, 7, 4, 6]
class_m1_first_point	int	3
m2_first_point	int [6]	[4, 9, 4, 8, 4, 7]
class_m2_first_point	int	3
m3_first_point	int [6]	[3, 2, 3, 1, 3, 0]
class_m3_first_point	int	3
m4_first_point	int [6]	[32, 16, 0, 0, 32, 16]
class_m4_first_point	int	1
m5_first_point	int [6]	[29, 16, 13, 0, 29, 16]
class_m5_first_point	int	1
m6_first_point	int [6]	[23, 20, 7, 4, 23, 20]
class_m6_first_point	int	1

Figura 5.27: Clasificación de dos categorías en el sistema SVM-RNS

Posterior a la clasificación se analizó el desempeño del sistema según las métricas brindadas por Vivado, como se muestran en las figuras 5.30, 5.31, 5.32. Según lo anterior, el desempeño del sistema se midió con las siguientes figuras de mérito:

- Potencia total en el chip: 1.879 W.
- Potencia dinámica: 1.747 W.
- Potencia estática: 0.131 W.
- Temperatura de unión: 46,7 °C
- LUT: Se utilizaron 6981 de 17600, representando un 39.66 % de uso.
- LUTRAM: Se utilizaron 62 de 6000, representando un 1.03 % de uso.
- FF: Se utilizaron 1072 de 35200, representando 3.05 % de uso.
- BUFG: Se utilizó 1 de 32, representando el 3.13 % de uso.

Respecto a la máxima frecuencia de operación se puede obtener a partir del parámetro *Worst Negative Slack* asociado al reloj de 50MHz del bloque *clk_fpga_0*:

$$f_{MAX} = \frac{1}{T_{clk} - WNS} = \frac{1}{20ns - 0.164ns} = 50.4MHz \quad (5.5)$$

La resta $T_{clk} - WNS$ corresponde al tiempo de ejecución para el camino de datos más largo, por lo cual su inverso representa la frecuencia máxima del sistema (f_{MAX}). La asociación entre el WNS y el reloj de 50MHz se puede observar en la figura 5.28, y el valor de los relojes en 5.29

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
Path 1	0.164	20	165	uP_and_SVM_JS_out_reg[0]C	uP_and_SVM_JS_var_reg[0]D	19.427	3.735	15.692	20.0	clk_fpga_0	clk_fpga_0
Path 2	0.199	20	75	uP_and_SVM_JS_out_reg[0]C	uP_and_SVM_JS_var_reg[0]D	19.318	3.969	15.349	20.0	clk_fpga_0	clk_fpga_0
Path 3	0.219	20	171	uP_and_SVM_JS_out_reg[0]C	uP_and_SVM_JS_var_reg[0]D	19.404	3.772	15.632	20.0	clk_fpga_0	clk_fpga_0
Path 4	0.240	19	122	uP_and_SVM_JS_out_reg[0]C	uP_and_SVM_JS_var_reg[0]D	19.273	3.372	15.901	20.0	clk_fpga_0	clk_fpga_0
Path 5	0.357	20	109	uP_and_SVM_JS_out_reg[0]C	uP_and_SVM_JS_var_reg[0]D	19.196	3.804	15.392	20.0	clk_fpga_0	clk_fpga_0
Path 6	0.425	19	165	uP_and_SVM_JS_out_reg[0]C	uP_and_SVM_JS_var_reg[0]D	19.204	3.362	15.842	20.0	clk_fpga_0	clk_fpga_0
Path 7	0.470	19	185	uP_and_SVM_JS_out_reg[0]C	uP_and_SVM_JS_var_reg[0]D	19.180	3.602	15.578	20.0	clk_fpga_0	clk_fpga_0
Path 8	0.492	19	100	uP_and_SVM_JS_out_reg[0]C	uP_and_SVM_JS_var_reg[0]D	19.117	3.362	15.755	20.0	clk_fpga_0	clk_fpga_0
Path 9	0.499	20	97	uP_and_SVM_JS_out_reg[0]C	uP_and_SVM_JS_var_reg[0]D	19.106	3.777	15.329	20.0	clk_fpga_0	clk_fpga_0
Path 10	0.512	20	181	uP_and_SVM_JS_out_reg[0]C	uP_and_SVM_JS_var_reg[0]D	19.019	3.744	15.275	20.0	clk_fpga_0	clk_fpga_0

Figura 5.28: WNS reportado y su asociación al reloj de 50 MHz *clk_fpga_0*

Name	Waveform	Period (ns)	Frequency (MHz)
clk_fpga_0	(0.000 10.000)	20.000	50.000
clk_fpga_1	(0.000 5.000)	10.000	100.000

General Information
Timer Settings
Design Timing Summary
Clock Summary (2)
Check Timing (0)
Intra-Clock Paths
clk_fpga_0
Setup 0.164 ns (10)
Hold 0.014 ns (10)

Figura 5.29: relojes del sistema

Todo lo anterior correspondió al diseño implementado que se muestra en la figura 5.33. Se aprecia que para un clasificador de dos categorías se utiliza aproximadamente un 50 % de las celdas en hardware disponibles en el chip *Zynq 7010 (xc7c010clg255 - 1)*.

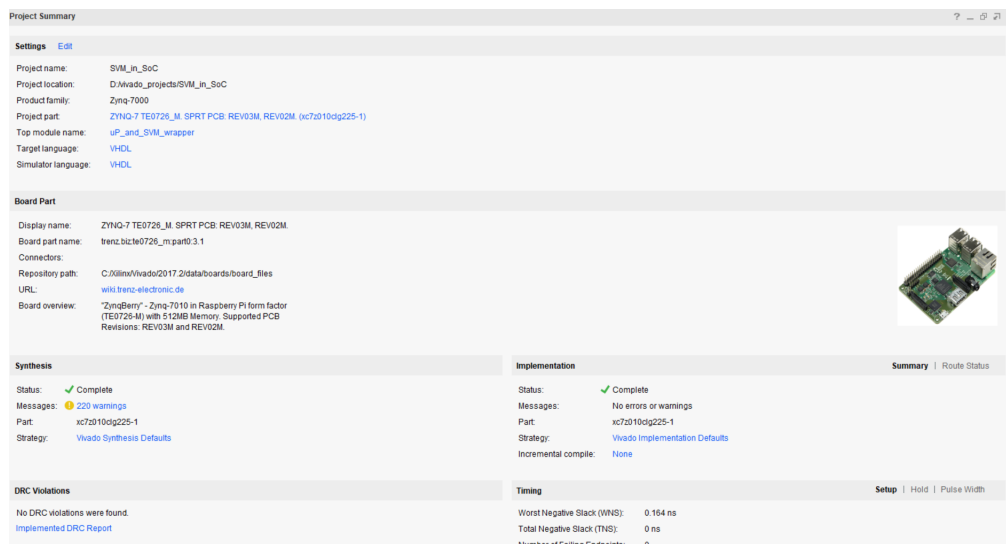


Figura 5.30: Información general de la plataforma y del proyecto diseñado

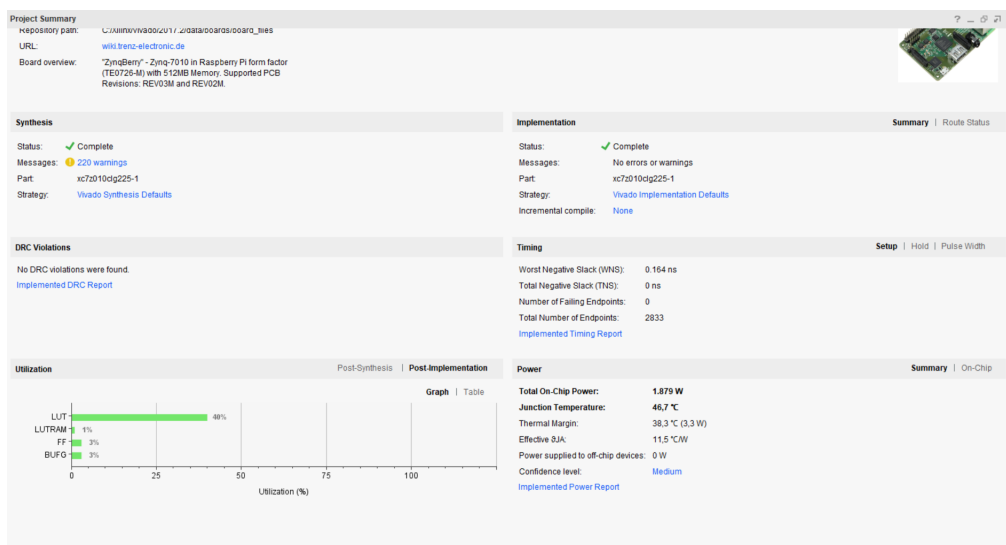


Figura 5.31: Desempeño porcentual medido en el sistema por *Vivado*

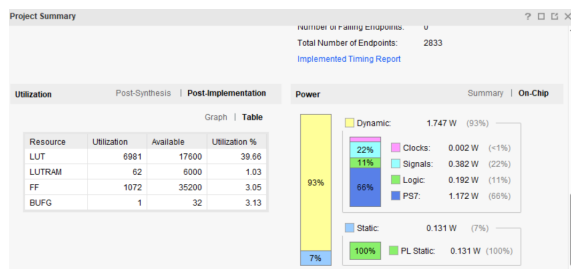


Figura 5.32: Desempeño medido en el sistema por *Vivado*

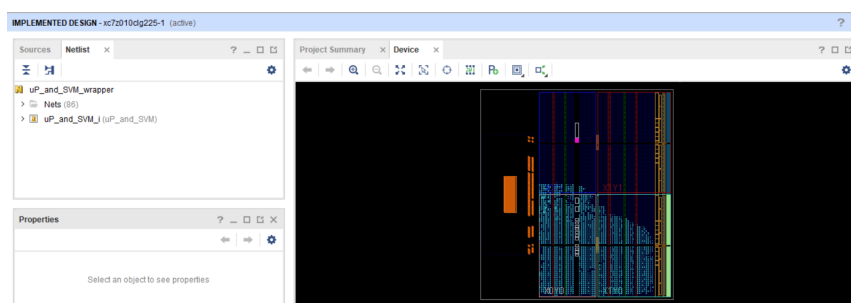


Figura 5.33: Esquema del sistema implementado con las herramientas de *Vivado*

5.4 Discusión

Esta investigación tuvo como propósito principal la exploración del Sistema Numérico de los Residuos para su uso en sistemas de reconocimiento de patrones y análisis de datos en señales EMG para prótesis de miembros superiores. Cabe resaltar que fue posible llevar a la realidad el objetivo gracias a la implementación del clasificador SVM-RNS, cuyos fundamentos sobre el conjunto específico de módulos seleccionados y sus concernientes técnicas asociadas fueron desarrolladas completamente en el transcurso de la etapa de ejecución del proyecto.

Debe resaltarse que el uso de métodos de comparación entre números representados en el espacio de los residuos y el uso de las Look-Up Tables para la función Kernel son novedosas y partieron de la lectura y adaptación de diversas teorías encontradas en la literatura (véase [Salamat et al., 2018] [Molahosseini et al., 2008] [Kaplun et al., 2016] [Lin et al., 2020] [Huang et al., 2019] [Isupov, 2020]) al problema concreto de investigación.

Como consecuencia de lo anterior, fue desarrollado un método que permitió el uso de las SVM en RNS, con una función de comparación basada en LPN utilizando un conjunto de módulos diferentes a los reportados en la literatura [Salamat et al., 2018], y adicionalmente se realizó un método para lograr la representación de la función exponencial en RNS. Estos métodos para representar funciones no lineales son objetivos de importancia fundamental en las investigaciones relacionadas al uso práctico de sistemas numéricos no convencionales como el RNS en sistemas de reconocimiento de patrones y de procesamiento digital de señales [Jenkins et al., 2018] [Sousa, 2021] [Soloviev et al., 2020]. Por ejemplo, en los reportes del sistema implementado en [Soloviev et al., 2020] se realizó una CNN en hardware con dos tipos de funciones de activación no lineales: ReLU y sigmoide. En el primer caso resultó difícil de implementar en RNS por el uso de las operaciones de comparación con cero y en el segundo caso por el uso de la función exponencial y la división. En el artículo se propuso el uso de una función modificada para evitar la operación de comparación, pero dificultó el entrenamiento porque no se pueden utilizar redes neuronales previamente entrenadas. En resumen, en las consideraciones específicas de los trabajos previos sobre funciones de activación en RNS se consideraba el uso de polinomios de Taylor para aproximar la función exponencial, sin embargo este método se sustituyó por el mapeo de la función exponencial a través de diversos escalamientos y el uso de tablas porque no era factible el uso directo de la función factorial presente en la serie de Taylor. Por otro lado, aunque el enfoque original de los trabajos mencionados estuvo en el contexto de las Redes Neuronales Artificiales, se pudieron extrapolar y adaptar tales fundamentos al contexto de las Máquinas de Vectores de Soporte, demostrando la posibilidad del uso del RNS dentro del análisis de señales EMG.

Previo a la etapa de clasificación, el uso de PCA para reducir características permitió disminuir drásticamente la dimensionalidad de los vectores de entrada al clasificador sintetizado en el bloque *PL* del *SoC*. Esto fue algo crucial dadas las limitantes en la cantidad de elementos lógicos de la plataforma.

Por otro lado, debe tenerse en cuenta que las soluciones encontradas en trabajos previos no son comparables directamente con la solución planteada en el proyecto dado que ninguna de las implementaciones revisadas reunía todos los focos principales que se revisaron en la pregunta de investigación planteada en el estado del arte. Sin embargo, se puede realizar un resumen de las figuras de mérito en implementaciones con focos afines a la solución desarrollada:

1. En el trabajo reportado en [Boschmann et al., 2016] se enlistaron entre sus características una metodología codiseño en un SoC Zynq XC7Z020 con las siguientes medidas de desempeño en el bloque *PL* para la implementación en VHDL:

- LUTs: 1.58 %
- FFs: 0.38 %
- BRAMs: 7.93 %
- DSP: 0.52 %

Debe tenerse muy en cuenta que en esta implementación el clasificador se realizó en el bloque *PS* del SoC Zynq, aligerando la carga del bloque *PS* que solo se encargó de extraer características para 8 movimientos diferentes.

2. Para contrastar el trabajo anterior, en el sistema reportado en [Garai and Dutta, 2015] se logró un DSP reconfigurable basado en RNS descrito en Verilog para la plataforma Virtex 5 XC5VLX30. Para una configuración con complejidad similar en tipo y número de bloques aritméticos a la del trabajo presentado, y bajo un conjunto de módulos (64, 65, 63) se obtuvo un uso de área de 134 slices de 4800 [Xilinx, 2006] ($\frac{134}{4800} = 2.7\%$). Debe tenerse en cuenta que este procesador como tal no contaba con un bloque de funciones almacenadas en LUT, como la función Kernel de la SVM-RNS.
3. En el caso específico de la extracción de características estadísticas de señales EMG en una FPGA [Díaz et al., 2010] para 4 canales de entrada, se obtuvieron las siguientes métricas:
 - Número de Slices utilizados: 4608 de 5888 (78.2 %).
 - Número de LUTs de 4 entradas: 9206 de 11776 (78.1 %).
 - Número de multiplicadores: 18 de 32 (56.2 %).
 - Frecuencia máxima: 44.57 MHz.
 - Consumo de potencia: 100 mW.
4. En el sistema mostrado en [Szecowka et al., 2010], se realiza una extracción de características basadas en la Transformada de Fourier Discreta (DFT), y se

muestra el uso de 5000 registros y 6000 unidades lógicas en una Xilinx Spartan 3 (xc3s1000).

5. En el diseño de [Bence J. Borbély et al., 2013] se muestra un sistema en SoC con la Zynq 7020 para el análisis de señales mioeléctricas. Sin embargo, la clasificación con LDA se realizó en una computadora distinta que fue conectada al SoC. Aún así, las operaciones esenciales de extracción de características y operaciones con vectores si se realizaron en el sistema embebido. Sus métricas para 300 muestras por ventana de procesamiento fueron:
 - LUTs de 6 entradas: 1163 de 53200 (2.1 %).
 - Flip - Flops: 1566 de 106400 (1.4 %).
 - Slices: 376 de 13300 (2.8 %).
 - Slices DSP48: 21 de 220 (9.5 %).
 - BlockRAM: 0 de 40 (0 %).
6. En el trabajo propuesto en [Sakellariou et al., 2022] se realizó una implementación de funciones de activación sigmoidales y de tangente hiperbólica con RNS de cinco módulos que fueron aproximadas con funciones lineales por trozos (*piecewise-linear*) utilizados por un bloque de Red Neuronal Recurrente (RNN) denominado *Long Short-Term memory block (LSTM)*. En general, el uso del RNS en celdas LSTM resultó en una mayor eficiencia en términos de área respecto a la contraparte binaria. En mayor detalle, las métricas resultantes para una tecnología de 65nm fueron de un área de 0.21 mm² y a una frecuencia de 1 GHz, reduciendo en un 48 % el área con respecto al símil binario, además de tener una velocidad mayor al símil binario con frecuencia de 322 MHz

Estos resultados pueden contrastarse con el sistema propuesto a través de una tabla.

Trabajo revisado	Figuras de mérito	Tipo de tecnología
[Boschmann et al., 2016]	■ LUTs: 1.58 %	SoC
	■ FFs: 0.38 %	
	■ BRAMs: 7.93 %	
	■ DSP: 0.52 %	
[Garai and Dutta, 2015]	Uso de Slices $\frac{134}{4800} = 2.7 \%$	FPGA
[Díaz et al., 2010]	■ Número de Slices utilizados: 4608 de 5888 (78.2 %).	FPGA
	■ Número de LUTs de 4 entradas: 9206 de 11776 (78.1 %).	
	■ Número de multiplicadores: 18 de 32 (56.2 %).	
	■ Frecuencia máxima: 44.57 MHz.	
	■ Consumo de potencia: 100 mW.	
Sigue en la próxima página.		

Trabajo revisado	Figuras de mérito	Tipo de tecnología
[Szecowka et al., 2010]	5000 registros y 6000 unidades lógicas	FPGA

- LUTs de 6 entradas: 1163 de 53200 (2.1 %).
- Flip - Flops: 1566 de 106400 (1.4 %).
- Slices: 376 de 13300 SoC (2.8 %).
- Slices DSP48: 21 de 220 (9.5 %).
- BlockRAM: 0 de 40 (0 %).

[Sakellariou et al., 2022]	Área = 0.21mm ² f _{MAX} 1GHz	ASIC de 65nm
----------------------------	--	--------------

Sigue en la próxima página.

Trabajo revisado	Figuras de mérito	Tipo de tecnología
	■ Potencia total en el chip: 1.879 W. ■ Potencia dinámica: 1.747 W. ■ Potencia estática: 0.131 W.	
Diseño propio	■ Temperatura de unión: 46,7 °C ■ LUT: $\frac{6981}{17600} = 39.66 \%$ ■ LUTRAM: $\frac{62}{6000} = 1.03 \%$ ■ FF: $\frac{1072}{35200} = 3.05 \%$ ■ BUFG: $\frac{1}{32} = 3.13 \%$ ■ $f_{MAX} = 50.4\text{MHz}$	SoC

Cuadro 5.2: Resumen de comparativas en implementaciones afines

Observando las métricas y comparándolas con las obtenidas por el sistema desarrollado, se puede contrastar que el mayor consumo de bloques lógicos en el sistema se da en funciones almacenadas en memoria como es el caso del sistema SVM-RNS, ya que las medidas porcentuales de bloques no relacionados con LUTs resultaron similares. Significa esto que para el uso adecuado de funciones no lineales almacenadas en memoria se debe tener una plataforma que pueda almacenar toda la función Kernel con la precisión requerida. La precisión cambia con los valores de α y β del conjunto de módulos RNS, ya que cambia la cantidad de bits involucrados en el

ancho de las señales del sistema. Esto debe tenerse en cuenta, ya que una mayor precisión demanda una mayor cantidad de bits en la representación RNS, incrementando el rango dinámico M . Finalmente, esto quiere decir que el sistema basado en RNS sí es factible para el uso de plataformas embebidas, teniendo en cuenta la precisión del rango dinámico y la cantidad de memoria requerida para almacenar funciones no lineales. Respecto a la velocidad del sistema realizado, aunque se ajustan a las restricciones del reloj en la plataforma, se podrían buscar estrategias donde se pueda mejorar este aspecto, como incrementar la cantidad de módulos para aumentar el rango dinámico sin necesidad de utilizar más bits por canal en el RNS.

6. Conclusiones

- Es posible implementar clasificadores SVM con kernel RBF o kernel no lineales en RNS, siempre y cuando el mapeo a enteros en un conjunto finito sea posible.
- Es viable utilizar clasificadores SVM-RNS con sistemas de análisis de señales mioeléctricas, permitiendo el uso de cálculos en el dominio de los números enteros en múltiples módulos, evitando el uso de unidades de punto flotante, donde se realizan redondeos en los resultados que pueden propagar errores en los cálculos.
- El mayor uso de espacio en hardware correspondió a bloques de memoria RAM, debido al uso intensivo de *LUT*. Este uso constante de bloques RAM podría influir en incrementos en consumo de potencia del sistema.
- La exploración del RNS en sistemas de análisis de señales en sistemas embebidos de bajo consumo de potencia resultó satisfactoria, dado que en conjunto con métodos como el PCA permite la clasificación de puntos que no tienen una distribución uniforme en el espacio de características.
- El uso del método de filtros permitió seleccionar características representativas para su uso en el reconocimiento de patrones. Esto permitió mejorar el uso de recursos, porque se colocaron conjuntos reducidos de características en el método PCA, disminuyendo la dimensionalidad en los cálculos implicados.
- Los bloques hardware diseñados en VHDL utilizados en la clasificación de patrones tuvo como limitante principal la cantidad disponible de celdas en el SoC. Básicamente, la dimensionalidad de las características influye directa en la cantidad de operaciones y señales dentro de los cálculos de la función Kernel.
- Dado que las SVM permiten realizar una clasificación de señales EMG con un porcentaje similar a otros métodos de mayor complejidad de implementación, se eligió como algoritmo de clasificación y resultó en método factible para implementarse en hardware.
- Dado que el uso directo del sistema de numeración no convencional RNS bastó para implementar un sistema que permitiera clasificar señales EMG, no se

abordaron otros sistemas de numeración no convencional. Esto fue debido a que en la revisión de antecedentes se vió un buen uso del RNS en otros sistemas de reconocimiento de patrones en otros problemas de investigación.

Referencias

- [Abdelfattah, 2011] Abdelfattah, O. (2011). Data conversion in residue number system.
- [Abdelfattah et al., 2010] Abdelfattah, O., Swidan, A., and Zilic, Z. (2010). Efficient direct analog-to-residue conversion schemes. In *ICSES 2010 International Conference on Signals and Electronic Circuits*, pages 85–88. IEEE.
- [Abdi and Williams, 2010] Abdi, H. and Williams, L. J. (2010). Principal component analysis. *Wiley interdisciplinary reviews: computational statistics*, 2(4):433–459.
- [Albicocco et al., 2014] Albicocco, P., Cardarilli, G. C., Nannarelli, A., and Re, M. (2014). Twenty years of research on RNS for DSP: Lessons learned and future perspectives. *2014 International Symposium on Integrated Circuits (ISIC)*, (1):436–439.
- [Amor et al., 2019] Amor, A. B. H., El Ghouli, O., and Jemni, M. (2019). Sign language handshape recognition using myo armband. In *2019 7th International conference on ICT & Accessibility (ICTA)*, pages 1–5. IEEE.
- [Apostol, 2013] Apostol, T. M. (2013). *Introduction to analytic number theory*. Springer Science & Business Media.
- [Atzori et al., 2014a] Atzori, M., Gijsberts, A., Castellini, C., Caputo, B., Hager, A.-G. M., Elsig, S., Giatsidis, G., Bassetto, F., and Müller, H. (2014a). Electromyography data for non-invasive naturally-controlled robotic hand prostheses. *Scientific data*, 1(1):1–13.
- [Atzori et al., 2016] Atzori, M., Gijsberts, A., Castellini, C., Caputo, B., Mittaz Hager, A.-G., Elsig, S., Giatsidis, G., Bassetto, F., and Müller, H. (2016). Clinical parameter effect on the capability to control myoelectric robotic prosthetic hands. *Journal of rehabilitation research and development*, 53(3):345–358.
- [Atzori et al., 2014b] Atzori, M., Gijsberts, A., Kuzborskij, I., Elsig, S., Hager, A.-G. M., Deriaz, O., Castellini, C., Müller, H., and Caputo, B. (2014b). Characterization of a benchmark database for myoelectric movement classification. *IEEE transactions on neural systems and rehabilitation engineering*, 23(1):73–83.

- [Atzori and Müller, 2019] Atzori, M. and Müller, H. (2019). Pawfe: Fast signal feature extraction using parallel time windows. *Frontiers in neurorobotics*, 13:74.
- [Benatti et al., 2017] Benatti, S., Milosevic, B., Farella, E., Gruppioni, E., and Benini, L. (2017). A Prosthetic Hand Body Area Controller Based on Efficient Pattern Recognition Control Strategies. *Sensors*, 17(4):869.
- [Bence J. Borbély et al., 2013] Bence J. Borbély, Kincses, Z., Vörösházi, Z., Nagy, Z., and Szolgay, P. (2013). Analysis of myoelectric signals using a Field Programmable SoC. *Circuit Theory and Design (ECCTD), 2013 European Conference on*, pages 1–4.
- [Bergstra and Bengio, 2012] Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of machine learning research*, 13(2).
- [Bhamra et al., 2015] Bhamra, H., Kim, Y. J., Joseph, J., Lynch, J., Gall, O. Z., Mei, H., Meng, C., Tsai, J. W., and Irazoqui, P. (2015). A 24 μ W, Batteryless, Crystal-free, Multinode Synchronized SoC "Bionode" for Wireless Prosthesis Control. *IEEE Journal of Solid-State Circuits*, 50(11):2714–2727.
- [Borbély et al., 2014] Borbély, B. J., Kincses, Z., Vörösházi, Z., Nagy, Z., and Szolgay, P. (2014). A modular test platform for real-time measurement and analysis of EMG signals for improved prosthesis control. *Cellular Nanoscale Networks and their Applications (CNNA), 2014 14th International Workshop on*, pages 1–2.
- [Boschmann et al., 2016] Boschmann, A., Agne, A., Witschen, L., Thombansen, G., Kraus, F., and Platzner, M. (2016). FPGA-based acceleration of high density myoelectric signal processing. *2015 International Conference on ReConFigurable Computing and FPGAs, ReConFig 2015*.
- [Boschmann et al., 2017] Boschmann, A., Thombansen, G., Witschen, L., Wiens, A., and Platzner, M. (2017). A Zynq-based dynamically reconfigurable high density myoelectric prosthesis controller. *Proceedings of the 2017 Design, Automation and Test in Europe, DATE 2017*, pages 1002–1007.
- [Boser et al., 1992] Boser, B. E., Guyon, I. M., and Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. In *Proceedings of the fifth annual workshop on Computational learning theory*, pages 144–152. ACM.

- [Caesarendra et al., 2017] Caesarendra, W., Tjahjowidodo, T., and Pamungkas, D. (2017). Emg based classification of hand gestures using pca and anfis. In *2017 International Conference on Robotics, Biomimetics, and Intelligent Computational Systems (Robionetics)*, pages 18–23. IEEE.
- [Cardarilli et al., 2007] Cardarilli, G. C., Nannarelli, A., and Re, M. (2007). Residue number system for low-power DSP applications. *Conference Record - Asilomar Conference on Signals, Systems and Computers*, (1):1412–1416.
- [Chang et al., 2015] Chang, C., Molahosseini, A., Zarandi, A., and Tay, T. (2015). Residue Number Systems: A New Paradigm to Datapath Optimization for Low-Power and High-Performance Digital Signal Processing Applications. *IEEE Circ. & Syst. Mag.*, 15(4):26–44.
- [Chen et al., 2018] Chen, H., Tong, R., Chen, M., Fang, Y., and Liu, H. (2018). A hybrid cnn-svm classifier for hand gesture recognition with surface emg signals. In *2018 international conference on machine learning and cybernetics (ICMLC)*, volume 2, pages 619–624. IEEE.
- [Chokshi et al., 2009] Chokshi, R., Berezowski, K. S., Shrivastava, A., and Piestrak, S. J. (2009). Exploiting residue number system for power-efficient digital signal processing in embedded processors. *Proceedings of the 2009 international conference on Compilers, architecture, and synthesis for embedded systems - CASES '09*, page 19.
- [Concha, 2012] Concha, F. S. (2012). En colombia, el 61 % de las personas con discapacidad no recibe ningún tipo de ingreso económico para su subsistencia, es necesario que como sociedad les brindemos más oportunidades laborales. *Pandi, Agencia de Comunicación*.
- [Côtéallard et al., 2016] Côtéallard, U., Nougrou, F., Fall, C. L., Giguère, P., Gosselin, C., Laviolette, F., and Gosselin, B. (2016). A Convolutional Neural Network for robotic arm guidance using sEMG based frequency-features. *IEEE International Conference on Intelligent Robots and Systems*, 2016-Novem:2464–2470.
- [Díaz et al., 2010] Díaz, S. A. A., Sanchez, A. D., Sánchez, L. A., and Escudero, A. Z. (2010). 4 channel signal based FPGA architecture for myoelectric features extraction by H.O.S. *2010 2nd Circuits and Systems for Medical and Environmental Applications Workshop, CASME 2010*.

- [Dinero, 2015] Dinero, R. (2015). El 55 % de las empresas no contrata discapacitados.
- [Dong, 2006] Dong, S. (2006). Methods for constrained optimization. *Massachusetts Institute of Technology, Massachusetts*.
- [Duan et al., 2016] Duan, F., Dai, L., Chang, W., Chen, Z., Zhu, C., and Li, W. (2016). SEMG-Based Identification of Hand Motion Commands Using Wavelet Neural Network Combined with Discrete Wavelet Transform. *IEEE Transactions on Industrial Electronics*, 63(3):1923–1934.
- [Englehart and Hudgins, 2003] Englehart, K. and Hudgins, B. (2003). A robust, real-time control scheme for multifunction myoelectric control. *IEEE transactions on biomedical engineering*, 50(7):848–854.
- [Fontana, 2010] Fontana, J. M. (2010). *Classification of EMG signals to control a prosthetic hand using time-frequesncy representations and Support Vector Machines*. Louisiana Tech University.
- [Garai and Dutta, 2015] Garai, P. and Dutta, C. B. (2015). RNS based reconfigurable processor for high speed signal processing. *IEEE Region 10 Annual International Conference, Proceedings/TENCON*, 2015-Janua(February).
- [Granitto et al., 2006] Granitto, P. M., Furlanello, C., Biasioli, F., and Gasperi, F. (2006). Recursive feature elimination with random forest for ptr-ms analysis of agroindustrial products. *Chemometrics and intelligent laboratory systems*, 83(2):83–90.
- [Gu et al., 2018] Gu, Y., Yang, D., Huang, Q., Yang, W., and Liu, H. (2018). Robust emg pattern recognition in the presence of confounding factors: features, classifiers and adaptive learning. *Expert Systems with Applications*, 96:208–217.
- [Guyon et al., 2002] Guyon, I., Weston, J., Barnhill, S., and Vapnik, V. (2002). Gene selection for cancer classification using support vector machines. *Machine learning*, 46(1):389–422.
- [Hakonen et al., 2015] Hakonen, M., Piitulainen, H., and Visala, A. (2015). Biomedical Signal Processing and Control Current state of digital signal processing in myoelectric interfaces and related applications. *Biomedical Signal Processing and Control*, 18:334–359.

- [Hospital, 2015] Hospital, E. (2015). Colombia avanza en el desarrollo de prótesis mioeléctricas.
- [Huang et al., 2019] Huang, T.-C., Chiang, C.-H., and Lin, M.-H. (2019). Low-cost and fast design of precise activation functions in neural network. In *2019 IEEE International Conference on Consumer Electronics-Taiwan (ICCE-TW)*, pages 1–2. IEEE.
- [Hudgins et al., 1991] Hudgins, B., Parker, P., and Scott, R. (1991). The recognition of myoelectric patterns for prosthetic limb control. In *Proceedings of the Annual International Conference of the IEEE Engineering in Medicine and Biology Society Volume 13: 1991*, pages 2040–2041. IEEE.
- [Isupov, 2020] Isupov, K. (2020). Using floating-point intervals for non-modular computations in residue number system. *IEEE Access*, 8:58603–58619.
- [Jenkins et al., 2018] Jenkins, W. K., Soderstrand, M. A., and Radhakrishnan, C. (2018). Historical patterns of emerging residue number system technologies during the evolution of computer engineering and digital signal processing. In *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5. IEEE.
- [Kaplun et al., 2016] Kaplun, D., Gulvanskiy, V., Klionskiy, D., Kupriyanov, M., and Veligosh, A. (2016). Implementation of digital filters in the residue number system. In *2016 IEEE NW Russia Young Researchers in Electrical and Electronic Engineering Conference (EIconRusNW)*, pages 220–224. IEEE.
- [Lin et al., 2020] Lin, L.-Y., Schroff, J., Lin, T.-P., and Huang, T.-C. (2020). Residue number system design automation for neural network acceleration. In *2020 IEEE International Conference on Consumer Electronics-Taiwan (ICCE-Taiwan)*, pages 1–2. IEEE.
- [Madan et al., 2016] Madan, V. K., Balas, M. M., and Radhakrishnan, S. (2016). Soft Computing Applications. 356:403–411.
- [Molahosseini et al., 2008] Molahosseini, A. S., Navi, K., Hashemipour, O., and Jalali, A. (2008). An efficient architecture for designing reverse converters based on a general three-moduli set. *journal of Systems Architecture*, 54(10):929–934.

- [Molahosseini et al., 2017] Molahosseini, A. S., Zarandi, A. A. E., Martins, P., and Sousa, L. (2017). A multifunctional unit for designing efficient rns-based datapaths. *IEEE Access*, 5:25972–25986.
- [Moraga, 2012] Moraga, C. (2012). Pattern Analysis under Number Theoretic Transforms. pages 280–287.
- [Muda et al., 2010] Muda, L., Begam, M., and Elamvazuthi, I. (2010). Voice recognition algorithms using mel frequency cepstral coefficient (mfcc) and dynamic time warping (dtw) techniques. *arXiv preprint arXiv:1003.4083*.
- [Nakahara and Sasao, 2015] Nakahara, H. and Sasao, T. (2015). A deep convolutional neural network based on nested residue number system. *2015 25th International Conference on Field Programmable Logic and Applications (FPL)*, pages 1–6.
- [Navi et al., 2011] Navi, K., Molahosseini, A. S., and Esmaeildoust, M. (2011). How to teach residue number system to computer scientists and engineers. *IEEE Transactions on Education*, 54(1):156–163.
- [Negi et al., 2016] Negi, S., Kumar, Y., and Mishra, V. (2016). Feature extraction and classification for emg signals using linear discriminant analysis. In *2016 2nd International Conference on Advances in Computing, Communication, & Automation (ICACCA)(Fall)*, pages 1–6. IEEE.
- [Nina, 2019] Nina (2019). Ninaweb: The nina project database web interface.
- [Olsen, 2017] Olsen, E. B. (2017). Proposal for a High Precision Tensor Processing Unit. pages 1–16.
- [Olsen, 2018] Olsen, E. B. (2018). Rns hardware matrix multiplier for high precision neural network acceleration:rns tpu". In *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5. IEEE.
- [Oskoei and Hu, 2008] Oskoei, M. A. and Hu, H. (2008). Support vector machine-based classification scheme for myoelectric control applied to upper limb. *IEEE transactions on biomedical engineering*, 55(8):1956–1965.
- [Pais, 2011] Pais, E. (2011). ¿sabe cuántas personas en situación de discapacidad viven en cali?

- [Pandey and Jain, 2017] Pandey, A. and Jain, A. (2017). Comparative analysis of knn algorithm using various normalization techniques. *International Journal of Computer Network and Information Security*, 9(11):36.
- [Park and Park, 2008] Park, C. H. and Park, H. (2008). A comparison of generalized linear discriminant analysis algorithms. *Pattern Recognition*, 41(3):1083–1097.
- [Park and Lee, 2016a] Park, K. H. and Lee, S. W. (2016a). Movement intention decoding based on deep learning for multiuser myoelectric interfaces. *4th International Winter Conference on Brain-Computer Interface, BCI 2016*, pages 7–8.
- [Park and Lee, 2016b] Park, K.-H. and Lee, S.-W. (2016b). Movement intention decoding based on deep learning for multiuser myoelectric interfaces. In *2016 4th international winter conference on brain-computer Interface (BCI)*, pages 1–2. IEEE.
- [Phinyomark et al., 2012] Phinyomark, A., Phukpattaranont, P., and Limsakul, C. (2012). Feature reduction and selection for EMG signal classification. *Expert Systems with Applications*, 39(8):7420–7431.
- [Phinyomark et al., 2013] Phinyomark, A., Quaine, F., Charbonnier, S., Serviere, C., Tarpin-Bernard, F., and Laurillau, Y. (2013). EMG feature evaluation for improving myoelectric pattern recognition robustness. *Expert Systems with Applications*, 40(12):4832–4840.
- [Pizzolato et al., 2017] Pizzolato, S., Tagliapietra, L., Cognolato, M., Reggiani, M., Müller, H., and Atzori, M. (2017). Comparison of six electromyography acquisition setups on hand movement classification tasks. *PloS one*, 12(10):e0186132.
- [Radhakrishnan and Jenkins, 2009] Radhakrishnan, C. and Jenkins, W. K. (2009). Hybrid wht-rns architectures for fault tolerant adaptive filtering. In *2009 IEEE International Symposium on Circuits and Systems*, pages 2569–2572. IEEE.
- [Radhakrishnan and Jenkins, 2010a] Radhakrishnan, C. and Jenkins, W. K. (2010a). Modified discrete fourier transforms for fast convolution and adaptive filtering. In *Proceedings of 2010 IEEE International Symposium on Circuits and Systems*, pages 1611–1614. IEEE.
- [Radhakrishnan and Jenkins, 2010b] Radhakrishnan, C. and Jenkins, W. K. (2010b). Quadratic modified fermat transforms for fast convolution and adaptive filtering.

- In *2010 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 3806–3809. IEEE.
- [Ramírez et al., 2000] Ramírez, J., García, A., Fernández, P. G., and Lloris, A. (2000). An efficient RNS architecture for the computation of discrete wavelet transforms on programmable devices. *European Signal Processing Conference*, 2015-March(March).
- [Ramírez et al., 2003] Ramírez, J., García, A., Meyer-Bäse, U., Taylor, F., and Lloris, A. (2003). Implementation of RNS-based distributed arithmetic discrete wavelet transform architectures using field-programmable logic. *Journal of VLSI Signal Processing Systems for Signal, Image, and Video Technology*, 33(1-2):171–190.
- [Sabbagh et al., 2017] Sabbagh, A., Leonel, M., De Sousa, S., and Chang, C.-H. (2017). Embedded Systems Design with Special Arithmetic and Number Systems.
- [Sakellariou et al., 2022] Sakellariou, V., Paliourasy, V., Kouretasy, I., Saleh, H., and Stouraitis, T. (2022). A high-performance rns lstm block. In *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1264–1268. IEEE.
- [Salamat et al., 2018] Salamat, S., Imani, M., Gupta, S., and Rosing, T. (2018). Rnsnet: In-memory neural network acceleration using residue number system. In *2018 IEEE International Conference on Rebooting Computing (ICRC)*, pages 1–12. IEEE.
- [Salim et al., 2011] Salim, S. I. M., Salim, A. J., and Guan, S. Y. (2011). Hardware implementation of surface electromyogram signal processing: A survey. *Proceedings - 2011 IEEE Control and System Graduate Research Colloquium, ICSGRC 2011*, pages 27–31.
- [Siwiec et al., 2016] Siwiec, K., Marcinek, K., Boguszewicz, P., Borejko, T., Halauko, A., Jarosz, A., Kopanski, J., Kurjata-Pfitzner, E., Narczyk, P., Plasota, M., Wielgus, A., and Pleskacz, W. A. (2016). BioSoC: Highly integrated System-on-Chip for health monitoring. *Formal Proceedings of the 2016 IEEE 19th International Symposium on Design and Diagnostics of Electronic Circuits and Systems, DDECS 2016*.
- [Soloviev et al., 2020] Soloviev, R., Telpukhov, D., Mkrtchan, I., Kustov, A., and Stempkovskiy, A. (2020). Hardware implementation of convolutional neural networks based on residue number system. In *2020 Moscow Workshop on Electronic and Networking Technologies (MWENT)*, pages 1–7. IEEE.

- [Sousa, 2021] Sousa, L. (2021). Nonconventional computer arithmetic circuits, systems and applications. *IEEE Circuits and Systems Magazine*, 21(1):6–40.
- [Sousa et al., 2016] Sousa, L., Antao, S., and Martins, P. (2016). Combining residue arithmetic to design efficient cryptographic circuits and systems. *IEEE Circuits and Systems Magazine*, 16(4):6–32.
- [Szecowka et al., 2010] Szecowka, P. M., Spyra, A., Pedzinska-Rzany, J., and Wolczowski, A. R. (2010). Artificial hand control by myoelectric signals - Reduced DFT approach. *2010 15th International Conference on Methods and Models in Automation and Robotics, MMAR 2010*, pages 170–173.
- [Toivonen and Heikkila, 2006] Toivonen, T. and Heikkila, J. (2006). Video filtering with Fermat number theoretic transforms using residue number system. *IEEE Transactions on Circuits and Systems for Video Technology*, 16(1):92–101.
- [Tymchyshyn and Khlevniuk, 2020] Tymchyshyn, V. and Khlevniuk, A. (2020). Yet more simple smo algorithm.
- [Xilinx, 2006] Xilinx (2006). Virtex-5 family overview.
- [Xu et al., 2017] Xu, J., Liu, X., Huo, Z., Deng, C., Nie, F., and Huang, H. (2017). Multi-class support vector machine via maximizing multi-class margins. In *The 26th International Joint Conference on Artificial Intelligence (IJCAI 2017)*.
- [Xu et al., 2016] Xu, K., Guo, W., Hua, L., Sheng, X., and Zhu, X. (2016). A prosthetic arm based on EMG pattern recognition. *2016 IEEE International Conference on Robotics and Biomimetics, ROBIO 2016*, pages 1179–1184.
- [Xu et al., 2018] Xu, W., You, X., and Zhang, C. (2018). Using Fermat number transform to accelerate convolutional neural network. *Proceedings of International Conference on ASIC*, 2017-Octob:1033–1036.
- [Yang et al., 2014] Yang, L., Tang, Y. Y., and Sun, Q. (2014). Implementation of 2D discrete wavelet transform by number theoretic transform and 2D overlap-save method. *Mathematical Problems in Engineering*, 2014(3).
- [Zhang et al., 2012] Zhang, D., Xiong, A., Zhao, X., and Han, J. (2012). Pca and lda for emg-based control of bionic mechanical hand. In *2012 IEEE International Conference on Information and Automation*, pages 960–965. IEEE.

[Zhu et al., 2012] Zhu, D., Huang, Q., Lee, Z. C., Zheng, Y., and Siek, L. (2012). A novel analog-to-residue converter for biomedical DSP application. *ISOC 2012 - 2012 International SoC Design Conference*, pages 371–374.